百元 GEO Platform 技術ホワイトペーパー

生成 AI 時代におけるブランド可視性の工学的実践

Vincent Lin（百元科技）

2026-04-18

概要

本書は百元科技（Baiyuan Technology）が 2024〜2026 年に開発した「百元 GEO Platform」の工程実践を記録する。百元 GEO Platform は生成エンジン最適化（Generative Engine Optimization、GEO）を対象とする SaaS システムであり、ブランドが ChatGPT、Claude、Gemini、Perplexity 等の生成 AI 応答内で正確かつ 継続的に言及されることを目的とする。本書は 7 次元 AI シテーション率スコア リングアルゴリズム、AI ボット向けシャドウドキュメント（AXP）配信機構、Schema.org 3 層エンティティ知識グラフ、およびハルシネーションの自動検知・修復を行うクローズドループシステムを包括する。CC BY-NC 4.0 ライセンスで 公開する。

目次

第 1 章—生成エンジン最適化（GEO）の時代背景と課題 **5**
目次 5
1.1 生成検索がユーザー習慣を乗っ取った 5
1.2 AI シテーション率：生死を決める新しい指標 6
1.3 GEO は SEO の延長ではなく、独立した分野である 6
1.4 市場の現状：海外先行、アジアは空白 7
1.5 本書を書いた動機と読み方 7
要点 8
参考文献 8

第 2 章—百原 GEO Platform：システム全景 **9**
目次 9
2.1 設計思想：モニタリングツールからクローズドループシステムへ 9
2.2 3 つのモジュール 9
2.3 中核データフロー 10
2.4 技術スタック 11
2.5 マルチテナント・データ分離 11
2.6 AI プラットフォーム対応範囲 12
要点 12
参考文献 12

第 3 章—7 次元 GEO 採点アルゴリズム **14**
目次 14
3.1 単一スコアでは足りない理由 14
3.2 7 次元の設計 14
3.3 重みの哲学 16
3.4 関数スケルトン 17
3.5 従来の SEO スコアとの比較 17
3.6 アルゴリズムの限界 17
要点 18

参考文献 18

第 4 章—Stale Carry-Forward：信号連続性とデータ鮮度の工学設計 19
目次 19
4.1 問題定義 19
4.2 直感的な解決策がすべて誤っている理由 19
4.3 Stale Carry-Forward の設計 20
4.4 設計上のトレードオフ 21
4.5 データ鮮度の哲学 22
4.6 関数スケルトン 22
4.7 他分野への応用可能性 23
要点 23
参考文献 23

第 5 章—複数プロバイダ AI ルーティング：先回り型フォールトトレラント設計 25
目次 25
5.1 単一ベンダー依存の構造的リスク 25
5.2 modelRouter のアーキテクチャ 25
5.3 3 つの工学的課題 26
5.4 再試行重ね掛けの罠 27
5.5 スイッチのトリガーと可観測性 28
5.6 副経路のヘルスマネジメント 29
5.7 関数スケルトン 30
要点 31
参考文献 31

第 6 章—AXP シャドウドキュメント：Cloudflare Worker で AI ボットにクリーンなコンテンツを配信 32
目次 32
6.1 同じ HTML で両方に仕えると両方が損する理由 32
6.2 AXP：シャドウドキュメントの構造 33
6.3 Cloudflare Worker 注入機構 33
6.4 AI ボット UA 一覧と検出戦略 34
6.5 SaaS 自社ブランドのパス競合 35
6.6 Sitemap 自動生成 36
6.7 JSON-LD フラット化の落とし穴 37
6.8 GSC インデックスの血涙ノート 37
6.9 シャドウ公開ファイルスイート：llms.txt / llms-full.txt / schema.json / feed.xml 38
6.10 RAG 知識ベース連携：ブランド専用 KB の自動化 39
要点 41
参考文献 41

第 7 章—Schema.org フェーズ 1：25 業種 × 三層 @id 相互リンク 42
目次 42
7.1 AI 時代における Schema.org の役割再定義 42
7.2 25 業種特化 @type の設計 42
7.3 三層 @id 相互リンクのナレッジグラフ 43
7.4 物理ビジネス vs オンラインサービス：フィールド重みの分岐 45
7.5 データ完全度アルゴリズム 46
7.6 Wizard + Edit 二入口設計 46
7.7 GBP URL パーサー 47
7.8 関数スケルトン 48
要点 49

第 8 章—GBP API 統合戦略：事実の主情報源から対外出力への単方向同期 50

目次 50
8.1 なぜ GBP は物理ビジネスの事実主情報源か 50
8.2 GBP API 申請ゲートとプロセス 50
8.3 代行管理モデルの選択 51
8.4 フィールドマッピング表 51
8.5 同期頻度とクオータ 52
8.6 Webhook がない場合の補完方法 53
8.7 フェーズ 1–4 ロードマップ 53
要点 54
参考文献 54

第 9 章—クローズドループ型ハルシネーション検知と自動修復 55
目次 55
9.1 なぜ検知だけでは不十分なのか 55
9.2 AI ハルシネーションの 5 分類 55
9.3 主機構：NLI 分類 + ChainPoll 56
9.4 中央共有 RAG：SaaS インフラストラクチャ 58
9.5 L1 LLM Wiki：アクティブ意味層 59
9.6 修復：ClaimReview 生成と多経路注入 61
9.7 2 層再スキャンループ 62
9.8 収束タイミングと受理基準 62
本章のまとめ 63
参考資料 63

第 10 章—フェーズ・ベースライン試験：固定質問セットによる AI 認識の縦断比較 64
目次 64
10.1 通常スキャンの縦断盲点 64
10.2 フェーズ・ベースライン試験の設計 65
10.3 通常スキャンからのデータパス隔離 65
10.4 4 つの観察軸 66
10.5 運用ガイダンス 67
本章のまとめ 67
参考資料 68

第 11 章—5 ブランド実地観察：6 週間の匿名化データ 69
目次 69
11.1 ブランドプロフィール（匿名化） 69
11.2 GEO スコア分布 70
11.3 プラットフォームカバレッジの非対称性 70
11.4 Schema.org 完成度とシテーション率 71
11.5 AXP 展開前後 71
11.6 顧客側の落とし穴 72
11.7 3 つの予想外の発見 73
11.8 商業化初月の検証 73
本章のまとめ 74
参考資料 74

第 12 章—限界、未解決問題、今後の課題 75
目次 75
12.1 プラットフォームができないこと 75
12.2 AI モデルバージョン変動の不予測性 76
12.3 未解決研究課題 77
12.4 ロードマップ 77
12.5 実務者と研究者への招待 78
本章のまとめ 78

参考資料 79

付録 A —用語集（完全版） 80
A 80
B 80
C 80
D 80
E 80
F 81
G 81
H 81
I 81
J 81
K 81
L 81
M 82
N 82
O 82
P 82
Q 82
R 82
S 82
T 83
U 83
W 83
標準参照表 83

付録 B —公開 API 仕様（抜粋） 84
共通規約 84
1. Brands 84
2. Scans 85
3. Entity（Schema.org ブランドエンティティ） 86
4. AXP（シャドウドキュメント） 87
5. Hallucinations 87
6. Baseline（フェーズ・ベースライン試験） 87
7. Webhooks（顧客受信可能） 88
エラーコード 88

付録 C —参考文献 89
1. 公式仕様と標準 89
2. 技術フレームワークと開発ツール 90
3. 業界調査と統計 90
4. 書籍・単行本 90
5. 関連学術研究（抜粋） 90
本書の引用方法 91

付録 D —図目録と制作ガイドライン 92
D.1 完全目録 92
D.2 制作スタイルガイド 93
D.3 制作ステータス 95

第 1 章—生成エンジン最適化（GEO）の時代背景と課題

> ユーザーはもはや答えを「検索」しない。答えを「尋ねる」ようになった。検索が生成に置き換わるとき、ブランドの可視性のルールは書き直さざるをえない。

目次

- 1.1 生成検索がユーザー習慣を乗っ取った
- 1.2 AI シテーション率：生死を決める新しい指標
- 1.3 GEO は SEO の延長ではなく、独立した分野である
- 1.4 市場の現状：海外先行、アジアは空白
- 1.5 本書を書いた動機と読み方
- 要点
- 参考文献

1.1 生成検索がユーザー習慣を乗っ取った

2023 年までは、「何か調べたい」という行為はほぼ「Google を開く」と同義だった。2024 年に入ってこの等式が揺らぎ、2025 年末にはついに成立しなくなった。

数字は明快である：

- **ChatGPT** のユーザークエリ処理量は 1 日あたり **25 億件** を超える（OpenAI 2025 Q3 開示）[1]
- **Perplexity** は月間 **7.8 億件**、年間成長率は 3 桁を維持する[2]
- **Google AI Overview** は 2024 年 5 月の全面ロールアウト後、従来の SERP の青いリンクに対するクリック率を、SimilarWeb と Digital Research Index の追跡データで平均 **34〜48%** 下落させた[3]。医療・法務・SaaS 選定など高情報密度領域では下落幅は **60%** に近い。

ブランドにとって結論は一行で終わる：

> 「ユーザーが自分を見つける」入口は、『10 個の青いリンク』から『AI が生成する一段落』に移行している。そしてその段落に登場しないブランドは、そのユーザーの意思決定パスから**事実上消滅**している。

これは予測ではなく、すでに起きた現実である。

図 1-1：検索チャネルの移行（概念図）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Information Retrieval Channel Share (2023–2025, illustrative)"
    x-axis ["2023 Q1", "2023 Q4", "2024 Q2", "2024 Q4", "2025 Q2", "2025 Q4"]
    y-axis "Share of Queries (%)" 0 --> 100
    bar [85, 78, 65, 52, 42, 35]
    line [5, 10, 18, 28, 36, 42]
```

図 1-1：棒グラフは「従来の Google SERP」の占有率、折れ線は「生成 AI（AI Overview を含む）」の合計占有率。数値は調査機関・地域により異なり、本図はトレンドを示すための概念図である。

---

[1] OpenAI. (2025). Usage & Revenue Update, Q3 2025. 公式四半期開示。
[2] Perplexity AI. (2025). Year in Review 2024: Search Volume & Engagement. 公式ブログ。
[3] SimilarWeb. (2025). The State of Generative Search: AI Overview Impact on Publisher Traffic. リサーチレポート。

1.2 AI シテーション率：生死を決める新しい指標

ユーザーが ChatGPT、Claude、Perplexity、Copilot、Gemini に以下のような問い合わせをするとき：

- 「おすすめの B2B マーケティング自動化ツールは?」
- 「東京都内でレビューの良い美容クリニックは?」
- 「中小企業向けの CRM SaaS ならどれがいい?」

AI は**具体的なブランド名を含む一段落の文章を生成する。**名前の挙がったブランドは候補リストに入り、名前の挙がらないブランドは**対話そのものに登場しない。**ユーザーが「他に候補はない?」と追加で尋ねることは稀である——2015 年の Google ユーザーが検索結果の 2 ページ目をめくらなかったのと同じ構造である。

この現象を定量化するのが **AI シテーション率**（AI Citation Rate）——代表的な意図クエリの集合に対して、AI がブランドを自発的に言及する割合である。クリック率でも、表示回数でも、順位でもない。純粋に、**「AI があなたを覚えていて、名前を口にするかどうか」**である。

その性質は従来の SEO 指標とは別物である：

性質	従来の SEO 指標	AI シテーション率
透明性	分解可能なルール（PageRank、Core Web Vitals）	ブラックボックス（公開ルールなし）
プラットフォーム間の差	Google が主、指標の多くが移植可能	乖離が大きい—ChatGPT / Claude / DeepSeek それぞれ独自ロジック
時間的安定性	アルゴリズム更新は四半期単位	モデル再訓練で週単位で変動
出力形態	クリック可能なリンク	自然言語の文（良くも悪くもなる）

つまり、AI 時代には**「リンクされること」**は重要ではなく、**「記述されること」**が重要である。記述の正確さ・好意的か否か・詳しさ、それそのものがブランド資産となる。

1.3 GEO は SEO の延長ではなく、独立した分野である

一行で線を引くとすれば：**SEO は Google に 1 位に載せてもらうこと、GEO は AI に言及してもらうこと**である。

図 1-2：SEO と GEO の中核差分

次元	SEO	GEO
成功の形	クリック可能な青いリンク	自然言語の文中に登場するブランド名
トリガー条件	キーワード一致 + 権威信号 + UX 信号	モデル訓練と検索拡張における「エンティティ関連強度」
操作レバー	コンテンツ、被リンク、構造化データ、Core Web Vitals	構造化実体、信頼できる情報源、ハルシネーション修正、AI ボット対応
検収指標	順位、CTR、滞在時間	シテーション率、ポジション品質、語り手のセンチメント、クロスプラットフォーム一貫性
時間軸	数週〜数ヶ月	モデル再訓練サイクル（通常は四半期単位）
主要な読者	人間のブラウザー	AI モデル + そのクローラー / 検索パイプライン

図 1-2：SEO と GEO は並列の分野であり、連続するものではない。GEO を SEO のサブ領域として扱う投資計画は必ず資源の誤配分を生む。

従来の SEO 実務者の多くはこう言う——「SEO をきちんとやれば、AI は自然に引用してくれる」。これは 2023 年には部分的に真だったが、2025 年には完全に偽である。理由は二つ：

第一に、AI の学習データは Google インデックスの射影ではない。主流大規模言語モデルは事前学習で Common Crawl、専門出版物、オープン知識基盤（Wikipedia、Wikidata）、および各ベンダー自社の検索拡張データを消化する。Google の順位はこのパイプラインの中で**一部の、そして間接的な**信号にすぎない。

第二に、構造化データの重みが AI 時代には飛躍的に大きくなった。AI がエンティティ（実体）を理解するのは H1/H2 やキーワード密度ではない。Schema.org JSON-LD、Wikidata triples、対応する知識グラフノードである。SEO スコアが満点でも Schema.org 構造がないサイトは、AI の目にはほぼ空白に映る。

GEO は SEO の次のバージョンではない。**入力・レバー・失敗モードがすべて異なる並列分野**である。この前提を受け入れなければ、関連する工学投資を計画することすらできない。

1.4 市場の現状：海外先行、アジアは空白

海外では 2024 年以降、第一世代の GEO ツールが立ち上がっている：

- **Profound**（米国）—Fortune 500 をターゲットにクロス AI ブランド監視とコンテンツ最適化
- **Otterly.ai**（欧州）—中小企業向けに AI シテーションダッシュボードと競合比較
- **AthenaHQ**（イスラエル）—エンタープライズ向けに RAG 連携とコンテンツ監査

いずれも英語市場にフォーカスしており、UI、AI プラットフォーム対応範囲、知識グラフ基盤は欧米中心である。日本語・繁体字中国語・韓国語など東アジア市場への対応、および中国発 AI（百度文心、DeepSeek、Moonshot Kimi、智譜）の対応は**ほぼ空白**である。

特に日本市場は、生成 AI 採用が 2025–2026 年に本格化したにもかかわらず、AI シテーション可視化の国産ツールが存在しない状態が続いている。多くのブランドオーナーとマーケティングエージェンシーは依然として「Google で自社名を検索して順位を目視確認する」というマニュアル運用に留まっている。**拡大中の市場空白**である。

1.5 本書を書いた動機と読み方

百原 GEO Platform は、この空白を埋めるために立ち上げた工学プロジェクトである。2024 年の初版プロトタイプから 2026 年現在の製品版まで、記録に値する工学経験を蓄積してきた——アルゴリズム、アーキテクチャ、フォールトトレランス設計、AI ボット向けコンテンツ配信、構造化エンティティ管理、ハルシネーション自動修正、そしてマルチテナント SaaS における持続可能なデータガバナンス。

本書は製品パンフレットでも、ユーザーマニュアルでもない。**エンジニアリング実践報告**——なぜこう設計したか、どの選択肢が行き止まりだったか、どのパターンが他チームに再利用可能か——を開示するものである。

想定読者は 3 種類：

1. **B2B 意思決定層** —「AI 時代のブランド可視性」の認識フレームワークを確立し、GEO を SEO のサブテーマと誤認する失敗を避けたい読者
2. **エンジニアリング責任者・アーキテクト** —「複数 AI プロバイダ耐障害」「信号連続性」「クローズドループ自動修正」などのパターンを借用したい読者
3. **開発者・技術者** —Schema.org、Cloudflare Worker、pgvector、BullMQ などのツールが実運用プロダクトでどのように組み合わされるかを見たい読者

以降の 11 章は、システム全景、中核アルゴリズム、対外可視性、品質保証クローズドループ、実戦データと反省の順に展開する。顧客個人情報・商業機密数字は集計化・匿名化して提示する。アルゴリズム詳細は骨格を完全開示するが、具体的な重み値は留保する——知識共有と商業的現実の間で我々が見出した均衡点である。

要点

- 生成 AI 検索はユーザーの情報取得チャネルを実質的に変えた。従来 SERP CTR は高情報密度領域で 34–60% 下落
- 「AI シテーション率」が GEO の中核指標であり、SEO 順位とは性質・メカニズム・操作レバーのすべてが異なる
- GEO と SEO は並列で継続ではない。GEO を SEO の下位テーマとして扱うと資源配分を誤る
- 海外には Profound / Otterly / AthenaHQ 等の先行ツールがあるが、日本・繁体字中国語市場には完成ソリューションがない
- 本書は百原 GEO Platform のエンジニアリング実践報告。意思決定者・エンジニアリング責任者・開発者の 3 類読者に向ける

参考文献

第 2 章—百原 GEO Platform：システム全景

百原 GEO Platform（以下「百原 GEO」または「本プラットフォーム」）は本書の主役である。百原科技（Baiyuan Technology）が開発・運用する。本章は以降の 10 章に対する地図索引として機能する。

目次

- 2.1 設計思想：モニタリングツールからクローズドループシステムへ
- 2.2 3 つのモジュール
- 2.3 中核データフロー
- 2.4 技術スタック
- 2.5 マルチテナント・データ分離
- 2.6 AI プラットフォーム対応範囲
- 要点
- 参考文献

2.1 設計思想：モニタリングツールからクローズドループシステムへ

第一世代の GEO ツールの多くは**モニタリング層**で止まっている——ダッシュボードで「あなたは今何点か」を示すだけ。モニタリングは診断であって治療ではない。低スコアを見た顧客は必ず問う：「で、どうすればいいんだ?」

百原 GEO の設計の出発点は、全プロセスを**クローズドループ**（Closed Loop）として捉えることだった：

図 2-1：クローズドループシステムの 6 段階

```
flowchart LR
    A[監視 <br/>Scan] --> B[診断 <br/>Diagnose]
    B --> C[最適化 <br/>Optimize]
    C --> D[配信 <br/>Deploy]
    D --> E[検証 <br/>Verify]
    E --> A
    E -.->| ハルシネーション検出 | F[修正 <br/>Remediate]
    F --> D
```

図 2-1：各段階は自動化可能・定量化可能・追跡可能でなければならない。「人間が一度は手で操作する必要がある」段階は、システムに刻まれた傷である。

この原則に基づき、プラットフォームを 3 つのモジュールに分割する。

2.2 3 つのモジュール

2.2.1 Scan Module —監視

ブランドを定期的に AI に問い合わせ、応答を取得・構造化・採点する。

中核責務

- 意図クエリ生成：ブランドの業種に応じて 20〜60 件の代表的質問を動的生成
- マルチプラットフォーム配信：同じ質問集合を 15 超の AI プラットフォーム / 検索エンジンに配信
- 応答抽出：自然言語回答から「ブランドが言及されたか」「言及ポジション」「感情トーン」「競合共起」を抽出

- スコア計算：7 次元アルゴリズムで GEO 総合点を合成（第 3 章）
- 信号維持：プラットフォーム障害時は Stale Carry-Forward を発動（第 4 章）

技術選定

- タスクキュー：**BullMQ** on Redis（再試行、優先度、レート制限）
- AI ルーティング：自社製 **modelRouter**、主経路は OpenAI 互換集約中継、副経路は各ベンダー直接（第 5 章）
- スキャンモード：通常（daily）+ センチネル（4h、第 9 章）+ Phase ベースライン（週次 / 隔週、第 10 章）

2.2.2 Visibility Module —対外可視性

AI がブランドを認識・引用しやすくする。

中核責務

- **構造化エンティティデータ管理**：Schema.org JSON-LD、25 業種特化 × 三層 `@id` 相互リンク（第 7 章）
- **AXP シャドウドキュメント**：AI ボット向けに純粋 HTML + JSON-LD + Markdown の「クリーンな」コンテンツを生成、人間用サイトと分離（第 6 章）
- **Cloudflare Worker 注入**：ネットワークエッジで AI ボット UA を検出し、シャドウドキュメントを動的に返す（またはオリジンにパススルー）
- **GBP 統合**：物理ビジネス向けに Google Business Profile を事実の主情報源として扱い、Schema.org へ単方向同期（第 8 章）
- **知識源構築**：Wikipedia / Wikidata / LinkedIn 等の AI クローラーが参照する権威プラットフォームと連携

2.2.3 Quality Module —品質保証

AI が抱くブランドへの誤認識を検出・修正し、ループを収束させる。

中核責務

- **ハルシネーション検出**：AI 応答から「ブランドに関する主張」を抽出し、グラウンドトゥルースと照合
- **ClaimReview 生成**：確定したハルシネーションごとに Schema.org `ClaimReview` ノードを生成
- **ナレッジベース同期**：修正情報を RAG に反映し、後続の検索でカバーする
- **2 層スキャンクローズドループ**
  - 第 1 層センチネル（4h / 検索型プラットフォーム）—修正が取得されたか高速に確認
  - 第 2 層フル（24h / 知識型プラットフォーム）—スコア + フィンガープリント検証を含む深度確認

2.3 中核データフロー

図 2-2：ブランドガバナンスの完全サイクル

```
flowchart TD
    U[ユーザーがブランド作成] --> W[ウィザードが Schema.org 記入を誘導]
    W --> G[AXP シャドウドキュメント + JSON-LD を生成]
    G --> CF[CF Worker が AI ボット向けにシャドウを配信]
    CF --> S[Scan Module が意図クエリを <br/>15 超の AI プラットフォームに配信]
    S --> R[応答が戻り → 採点]
    R --> D[ダッシュボードで 7 次元 <br/>+ 競合比較 + ギャップ分析を表示]
    R --> Q[Quality Module がハルシネーション検出]
```

```
Q --> C[ClaimReview を生成 <br/>AXP + RAG に注入]
C --> V[センチネル / フルで再スキャン検証]
V -->| 収束 | S
V -.->| まだ存在 | C
```

図 2-2：ループは「時間単位」（センチネル）または「日単位」（フル）で動く。ユーザーはダッシュボード上のスコア変動しか見えないが、背後でこの 8 ステップが連続的に循環している。

2.4 技術スタック

層	技術	主要役割
エッジ	Cloudflare Workers	AI ボット UA 検出、シャドウドキュメント注入、sitemap / robots 管理
フロントエンド	Next.js 16（Webpack）+ React 19 + TypeScript + Tailwind v4	ダッシュボード、ブランド管理、ウィザード、i18n（zh-TW / en / ja）
API	Node.js + Express 4 + Helmet + express-rate-limit + Zod	REST API、マルチテナント、JWT + 2FA
ワーカー	BullMQ 5 + Node.js タスクプロセス	スキャン、採点、AXP 生成、ハルシネーション検出、RAG 同期
AI ルーティング	自社製 modelRouter + OpenAI SDK + 各ベンダー直接 SDK	複数プロバイダ耐障害
データ	PostgreSQL 16 + pgvector + Redis 7	リレーショナル、ベクトル検索、キャッシュ、キュー
デプロイ	Docker Compose on AWS Lightsail（本番）/ ローカル Docker（UAT）	環境分離
RAG	全テナント共用セントラル RAG エンジン（内部 SaaS インフラ）	マルチテナント・ナレッジベース同期・検索

2026-04-18 時点でシステムは v2.19.4。マイナーバージョンを約 20 回発行、マイグレーション 139 件、フロントページ 60 超、API エンドポイント 180 超。

2.5 マルチテナント・データ分離

百原 GEO は B2B SaaS であり、同一データベースで複数顧客を同時に扱う。**データ分離は契約レベルの義務**であって「あったらいい」ものではない。我々は**二重保険**戦略を採る：

1. **PostgreSQL の Row-Level Security（RLS）**：`brands`、`scan_results`、`geo_scores` など中核テーブルに RLS ポリシーを設定し、`current_setting('app.org_id')` でフィルタ。アプリ層の SQL が間違っていてもデータは漏れない
2. **アプリ層フィルタ**：クエリ層 API は明示的に `org_id` を付加し、RLS と合わせて二重の検証を形成。RLS ポリシーが誤削除されてもアプリ層が分離を担保する

図 2-3：二重保険の模式図

```
flowchart LR
    R[HTTP Request] --> M[Auth Middleware<br/>org_id を解決]
    M --> C[Controller]
    C --> Q[Query Layer<br/> 明示的に WHERE org_id]
    Q --> PG[(PostgreSQL)]
    M -.->|SET app.org_id| PG
    PG --> RLS{RLS Policy<br/>current_setting でフィルタ}
    RLS --> D[(Data)]
```

図 2-3：アプリ層とデータベース層が独立に 1 回ずつフィルタする。単層の不具合ではクロステナント漏洩は起きない。

この設計は、早期にほぼ事故一歩手前だった事例から学んだものである——アプリ層のみに依存すると人間は必ず忘れる、RLS のみに依存するとポリシー変更を CI で検出できない。二重保険は両方の漏れが同時に発生しなければ事故に至らないようにする。

2.6 AI プラットフォーム対応範囲

本書執筆時点で、百原 GEO は **15 個の AI プラットフォーム** に対応し、3 群に分類している：

分類	数	代表プラットフォーム
グローバル汎用型	7	OpenAI GPT-4o / 4o-mini、Anthropic Claude、Google Gemini、Meta Llama、Mistral、xAI Grok、Cohere
中国語モデル	5	百度文心、DeepSeek、Moonshot Kimi、智譜 ChatGLM、阿里通義千問
検索型	3	Perplexity、ChatGPT Search、Google AI Overview（クローラー側）

プラットフォームを 1 つ追加するコストは「API を 1 つ繋ぐ」だけではない。モデル ID マッピング、extraParams 設計、timeout / retry 戦略、ハルシネーション検出テンプレート、スコアキャリブレーション、UI 対応を含め、**1 プラットフォームあたり 2〜4 エンジニアリング日**が必要である。

日本市場向けには、上記 15 に加えて日本固有の AI プラットフォーム（国産 LLM、国内エージェント）への対応を段階的に追加する予定である。

要点

- 百原 GEO は「クローズドループ」を設計の出発点とし、監視・診断・最適化・配信・検証のすべてを自動化する
- システムは Scan / Visibility / Quality の 3 モジュールに分かれ、それぞれが以降の章に対応する
- 技術スタックの核は Next.js 16 + Express + BullMQ + PostgreSQL 16 + pgvector + Redis
- マルチテナント分離は RLS + アプリ層フィルタの二重保険で単層の人的ミスを許容する
- 15 プラットフォーム対応（グローバル 7 + 中国語 5 + 検索型 3）、1 プラットフォーム追加に 2〜4 エンジニアリング日

参考文献

- 第 3 章—7 次元 GEO 採点アルゴリズム

- 第 4 章—Stale Carry-Forward
- 第 5 章—複数プロバイダ AI ルーティング
- 第 6 章—AXP シャドウドキュメント
- 第 7 章—Schema.org フェーズ 1
- 第 8 章—GBP API 統合
- 第 9 章—クローズドループ・ハルシネーション修正
- 第 10 章—Phase ベースラインテスト

第 3 章—7 次元 GEO 採点アルゴリズム

> 単一のスコアはどんな複雑系も比較可能な数字に圧縮する。しかし同時に、見るべき差異も一緒に圧縮してしまう。

目次

- 3.1 単一スコアでは足りない理由
- 3.2 7 次元の設計
- 3.3 重みの哲学
- 3.4 関数スケルトン
- 3.5 従来の SEO スコアとの比較
- 3.6 アルゴリズムの限界
- 要点
- 参考文献

3.1 単一スコアでは足りない理由

百原 GEO の 2024 年初版採点は単一指標 **Citation Rate**——「言及回数 ÷ クエリ総数」だけだった。シンプル、直感的、比較可能。しかし運用 3 ヶ月で、この指標が**体質の異なるブランドを同一スコアに圧縮してしまう**誤判定事例が大量に蓄積された。

v1 で両方 55 点を記録した実際のケース 2 例:

- **ブランド A**: 20 クエリ中 11 回言及。すべて回答の末尾、「他には □□ もあります」という付随的な言及。OpenAI と Anthropic の 2 社に集中
- **ブランド B**: 20 クエリ中 11 回言及。うち 8 回は回答の第 1〜2 文目。7 つの異なる AI プラットフォームにまたがり、具体的記述を伴う

両者の「AI に認識されているエネルギー」の差は極めて大きい。にもかかわらず、単一スコアは両者を同一セルに圧縮してしまう。ユーザーがこのスコアだけを見れば、アクション可能な情報がすべて失われる。

単一スコアの根本的な問題は: **GEO は多次元現象である。**言及回数・言及ポジション・言及プラットフォーム・トーン・記述深度・クロスプラットフォーム一貫性——これらはそれぞれ独立した信号であり、平均値に混ぜた瞬間に情報が消える。

3.2 7 次元の設計

百原 GEO v2 の採点は「AI エコシステム内のブランド状態」を 7 つの独立次元に分解する:

図 3-1: 7 次元レーダーの対比 (v1 vs v2、概念図)

```
%%{init: {'theme':'base'}}%%
graph LR
    A[Citation Rate<br/> シテーション率] --- B[Position Quality<br/> ポジション品質]
    B --- C[Query Coverage<br/> クエリカバレッジ]
    C --- D[Platform Breadth<br/> プラットフォーム広度]
    D --- E[Sentiment<br/> センチメント]
    E --- F[Content Depth<br/> コンテンツ深度]
    F --- G[Consistency<br/> 一貫性]
    G --- A
```

図 3-1：7 次元の関連を示す。隣接する次元には情報補完性があり、非隣接次元は独立に計算する。実際のレーダー図（0〜100）は PDF 版で実データとして提示する。

3.2.1 Citation Rate —シテーション率

定義：代表的意図クエリの中で、ブランドが自発的に言及される割合（0〜100）。

出発点：`mentioned_count / query_count`。

精緻化：

- 偽陽性の除去：競合の同名、類似プロダクトラインの誤認
- lemma マッチング：正式名称・略称・英語名・日英混在表記を同一言及とみなす
- プラットフォームごとにまずレートを計算し、それから加重平均する（「特定プラットフォームの回数が多い」ことが総合点を支配するのを防ぐ）

重みの割合：最大の単一次元ではあるが、意図的に 25%（4 分の 1）に抑える。v1 の単一指標状態に退化することを避けるため。

3.2.2 Position Quality —ポジション品質

定義：ブランドが AI 回答に登場するポジションの加重平均スコア。

ロジック：AI の回答を段落・文・箇条書きに分割し、各「言及」に対してポジション加重を与える：

位置	重み
第 1 文 / リスト第 1 項	1.0
冒頭 3 分の 1	0.8
中盤	0.5
末尾 / 追記	0.2

この次元は v1 時代によくあった疑問——「言及されているのに効果を感じない、なぜ?」——を解く。答えはしばしば「言及が遅すぎ、軽すぎる」。

3.2.3 Query Coverage —クエリカバレッジ

定義：言及される意図クエリタイプの多様性。

ロジック：意図クエリを「ベスト選択型」「比較型」「問題解決型」「入門推薦型」等に分類し、言及されたタイプ数 ÷ 総タイプ数。

この次元が露わにするのは：「ベスト X ツール」系の質問では言及されても、「A と B どちらを選ぶ?」の比較質問では完全に姿を消すブランドがある——異なる戦場には異なるコンテンツ戦略が必要である。

3.2.4 Platform Breadth —プラットフォーム広度

定義：ブランドがいくつの AI プラットフォームで自発的に言及されるか。

ロジック：言及プラットフォーム数 ÷ 監視プラットフォーム総数。

プラットフォームの偏食を露わにする。例えば OpenAI 系では言及されるが中国モデル（DeepSeek、Kimi）では完全に欠席する SaaS——これはランダム誤差ではなく、特定のコンテンツ可視性問題（訓練データ源、構造化データ対応範囲）を示している。

3.2.5 Sentiment Score —センチメントスコア

定義：言及テキストの感情傾向。

ロジック：独立した感情分類モデルで各言及を「ポジティブ / ニュートラル / ネガティブ」に分類し、0〜100 に集計。AI 本体の推論文をそのまま使わない（「A AI が A AI を自己評価する」問題を避けるため）。

この次元はブランド危機後に特に敏感になる。通常スキャンではニュートラルが 70〜80% を占めるが、50% に落ちると外部にチェック可能なネガティブコンテンツが発生したシグナルとなる。

3.2.6 Content Depth —コンテンツ深度

定義：AI がブランドに言及する際に添える記述の深度。

ロジック：「ブランドに関する自然言語テキスト」の長さ、実体密度（プロダクトライン・創業者・応用シナリオ等どれだけ関連事実を挙げるか）、構文複雑度を分析。

この次元は「名前を挙げられる」と「紹介される」の違いを分別する。B2B SaaS、教育機関、専門サービスにおいては、**Content Depth は Citation Rate 自体より重要**——深度ある記述こそがコンバージョンを生む。

3.2.7 Consistency —クロスプラットフォーム一貫性

定義：同一ブランドの上記 6 次元を異なる AI プラットフォームで集計した標準偏差の逆数（0〜100 に正規化）。

ロジック：一貫性の高いブランドは ChatGPT、Claude、Gemini、DeepSeek のどこでも似た語り口とスコアを得る。一貫性の低いブランドは、あるプラットフォームでは鮮明でも別のプラットフォームではぼんやりしている。

この次元は他の次元を変えないが、「結果の信頼性」を示すシグナルになる：一貫性が高いほどブランドの AI 認識が安定したエンティティ像に収束していることを意味し、低いほど AI 間で認識が分散している。

3.3 重みの哲学

7 つの次元は単純平均しない。重み配分は 3 つの原則に基づく：

1. **重要性** —Citation Rate と Content Depth は「ブランドが認識されるか」を最も直接的に左右する。重みは大きめ
2. **ノイズ感受性** —Sentiment と Consistency は単発応答の異常値に影響されやすい。重みは小さめ (ギザギザを防ぐ)
3. **操作可能性** —Platform Breadth と Query Coverage は「能動的行動」で直接改善可能な次元。一定の重みを与え、スコアが改善に反応するようにする

本章では**公式の骨格と重みのグルーピング（高 / 中 / 低）を開示するが、精密な数値は非公開**とする。これは秘密主義ではなく、**顧客が指標最適化に走って実質最適化を怠る事態を防ぐため**である：

> ある次元が 30% を占めることを知ってしまうと、その次元だけにリソースを投入し、スコアは上がるが AI の実際の認識は変わらないという結果になる。我々は「全体的なコンテンツ品質の改善」に集中してほしい。重みの分解には集中してほしくない。

この設計思想は検索エンジンアルゴリズムの歴史と一致する——Google も同じ理由で PageRank の精密な重みを公開しなかった。

3.4 関数スケルトン

```
// weights are resolved from config; values are deliberately not exposed.
function calcGEOScore(scanResults, brandId) {
  const dims = {
    citation:    computeCitationRate(scanResults, brandId),
    position:    computePositionQuality(scanResults, brandId),
    coverage:    computeQueryCoverage(scanResults, brandId),
    breadth:     computePlatformBreadth(scanResults, brandId),
    sentiment:   computeSentimentScore(scanResults, brandId),
    depth:       computeContentDepth(scanResults, brandId),
    consistency: computeConsistency(scanResults, brandId),
  };

  const weighted =
    dims.citation    * W_CITATION +
    dims.position    * W_POSITION +
    dims.coverage    * W_COVERAGE +
    dims.breadth     * W_BREADTH +
    dims.sentiment   * W_SENTIMENT +
    dims.depth       * W_DEPTH +
    dims.consistency * W_CONSISTENCY;

  return {
    total: Math.round(weighted),
    dimensions: dims,
    version: SCORING_VERSION,
  };
}
```

3.5 従来の SEO スコアとの比較

側面	SEO スコア	GEO スコア
入力データ	ページコンテンツ + 被リンク + UX メトリクス	AI 応答テキスト + エンティティ照合 + クロスプラットフォーム集約
結果形態	リンク順位	自然言語での言及
時間粒度	日次	日次 + センチネル 4h + Phase 週次
比較可能性	サイト間で可比	業種横断では厳密には不可比（意図クエリ空間が異なる）
バージョン感受性	低	高（AI モデルバージョンの影響大）

この表が強調することは 1 点：**GEO スコアは「ブランド状態指標」であって「品質順位指標」ではない。**業種ごとにスコアの基準線が異なり、業種横断で大小を比べても意味がない。

3.6 アルゴリズムの限界

この 7 次元システムができないことを誠実に列挙する：

- **商業結果を反映しない**：GEO スコアが高くてもコンバージョンが高いとは限らない。AI 可視性の代理指標であり、業績指標ではない

- **モデルバージョン再訓練の影響を受ける**：OpenAI や Claude が新版をリリースするとスコア全体が 3〜10 点動くことがある。外部変動であってブランド変動ではない
- **クエリ空間は主観的**：同業種の 2 つのクエリセットは異なるスコアを生みうる。固定ベースライン（第 10 章）で緩和するが根絶は不可能
- **中国語・日本語モデルの対応はまだ不完全**：非英語モデルの感情モデル・ポジション検出精度は英語モデルに及ばない

これらはアルゴリズムの失敗ではなく、GEO 本質への誠実な陳述である。「AI 認識を精密に定量化できる」と主張するツールはすべて疑うべきである。

要点

- 単一 Citation Rate は体質が異なるのに総合点が同じブランドを同一と誤判する
- 7 次元（Citation / Position / Coverage / Breadth / Sentiment / Depth / Consistency）はそれぞれ独立した信号と用途を持つ
- 重みは「重要性 / ノイズ感受性 / 操作可能性」の 3 原則で配分、精密な数値は指標囚人のジレンマを防ぐため意図的に非公開
- GEO スコアはブランド**状態**指標であり**品質**順位ではない。業種横断の大小比較は無意味
- スコアは AI モデルバージョン再訓練の影響を受ける。Phase ベースラインで縦断的に安定化する

参考文献

- 第 4 章—Stale Carry-Forward：信号連続性設計
- 第 10 章—Phase ベースラインテスト
- Google Search Central. How Search works. https://www.google.com/search/howsearchworks/
- Schema.org. ClaimReview schema. https://schema.org/ClaimReview

---

第 4 章—Stale Carry-Forward：信号連続性とデータ鮮度の工学設計

> GEO スコアが反映するのは「AI 認識におけるブランドの状態」であって、「データパイプラインの健全性」ではない。両者を混同すれば、スコアはその本来の意味を失う。

目次

- 4.1 問題定義
- 4.2 直感的な解決策がすべて誤っている理由
- 4.3 Stale Carry-Forward の設計
- 4.4 設計上のトレードオフ
- 4.5 データ鮮度の哲学
- 4.6 関数スケルトン
- 4.7 他分野への応用可能性
- 要点
- 参考文献

4.1 問題定義

外部 AI サービスに依存するシステムは、いかなる時点でも**部分的または全面的な失敗**に遭遇しうる。失敗の原因は多岐にわたる：

- ネットワーク層：パケットロス、CDN 経路切替、DNS ジッター
- API 層：5xx、429 レート制限、トークンクオータ枯渇
- モデル層：特定バージョン廃止、サーバー再起動、リージョン障害
- アカウント層：クレジットカード期限切れ、支払い遅延、請求システム遅延

これらの失敗はエンジニアリング上の**日常イベント**であって「異常例外」ではない。問題は：これらの失敗がスキャンパイプラインで発生したとき、**GEO スコアを汚染させてはならない**ことである。

シナリオを想像されたい。ブランド X の Citation Rate は過去 30 日間安定して 55 点を維持している。ある日未明、X をスキャンする 6 つの AI プラットフォームのうち 3 つが同時に失敗する。もし「失敗即 0」で当日のスコアに組み入れれば、X は 55 点から 27.5 点に落ちる。ユーザーはダッシュボードを見て「ブランドが急に失寵した」と誤認しパニックに陥る。だが実際には AI の X への認識は一切変わっていない——変わったのはデータパイプラインだけである。

これが**第一の誤り**である：パイプライン障害をブランド変化と取り違える。

4.2 直感的な解決策がすべて誤っている理由

上記問題への第一反応は通常以下の 3 種のいずれかである。3 つすべてに問題がある。

案 A：動的分母（失敗プラットフォームを除外）

発想：成功したプラットフォームだけを計算に入れ、失敗はスキップする。

問題：分母の変動がスコアを**鋸歯状に震動**させる。今日は 10 プラットフォーム全成功でスコア 60、明日は 6 つ成功でスコア 62（たまたま失敗したのが低スコアのプラットフォームだった）、明後日は再び 10 全成功でスコア 60 に戻る。トレンド図は完全に読めなくなり、ユーザーは「本当に改善したのか分母が変わっただけか」を判断できない。

案 B：失敗は 0% 扱い

発想：失敗プラットフォームの当日スコアを 0 に記録し、分母を安定させる。

問題：これはアルゴリズムに「このブランドは今日このプラットフォームに完全に存在しない」と告げることになるが、実態は「今日の状態を我々は知らない」である。**「言及されなかった」と「スキャンできなかった」は根本的に異なる状態**であり、混同すれば後続のトレンド分析・ハルシネーション検出・競合比較がすべて狂う。

案 C：沈黙で捨てる（記録しない）

発想：スキャン失敗時はこの時間帯を存在しなかったものとする。

問題：時系列データベースに**断層**が発生し、「週末でスキャンなし」「スキャン失敗」「スキャンしたが変化なし」の 3 状態を区別できなくなる。後付けのロジックでカバーするコストは高く、保守性も低い。

3 つの解は共通の根本問題を抱える：**いずれも「データ欠損」と「データゼロ」という根本的に異なる状態を区別していない。**

4.3 Stale Carry-Forward の設計

我々の採用する方式は**履歴から直近の成功値をキャリーフォワード**し、明示的に「停滞」フラグを立てることである：

図 4-1：同一障害事象に対する 3 戦略の信号表現

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "Score under Mid-Scan Platform Outage (Illustrative)"
    x-axis ["Day 1", "Day 2", "Day 3 (outage)", "Day 4", "Day 5"]
    y-axis "GEO Score" 0 --> 80
    line [55, 56, 27, 28, 56]
    line [55, 56, 55, 55, 56]
```

図 4-1：上の折れ線 = 案 B「失敗即 0」による偽暴落。下の折れ線 = Stale Carry-Forward が連続性を維持し、UI 上で isStale を明示する。

アルゴリズムの手順

1. **検出**：あるプラットフォームが現在のスキャンで失敗率 **100%**（全クエリが無応答またはタイムアウト）に達する
2. **履歴探索**：該当ブランド × プラットフォームのスキャン履歴から**最大 200 行遡り**、最も近い「非ゼロ成功」の sov_score を探す
3. **キャリーフォワード**：その履歴値を当日レコードに持ち込み、同時に 2 つのフラグを設定
   - `isStale = true`
   - `lastSuccessAt = <historical_timestamp>`
4. **UI で誠実に告知**：ダッシュボードで該当プラットフォームのスコアに赤いバッジを表示、ホバーツールチップに「□ N 時間前から音信不通、前回成功値を表示中」
5. **下流アルゴリズムは通常通り動作**：Consistency、トレンド分析、競合比較はこのキャリーフォワード値を使えるので断層が発生しない

図 4-2：プラットフォームデータの状態機械

```
stateDiagram-v2
    [*] --> Fresh
    Fresh --> Stale: 当日スキャン 100% 失敗
    Stale --> Fresh: 次回スキャン成功
    Stale --> Expired: 停滞が閾値超（例：72h）
    Expired --> Fresh: スキャン成功
    Expired --> Reset: 30 日経っても未回復
    Reset --> Fresh: ベースライン再構築
```

図 4-2：Fresh → Stale は通常の回復パス、Expired と Reset は例外処理で長期障害シーンのために予約されている。

4.4 設計上のトレードオフ

4.4.1 lookback が無限ではなく 200 行である理由

理由：稀な状況で期限切れデータを引用するのを避けるため。あるプラットフォームが長期にわたり応答しない場合、最も古い履歴は数ヶ月前まで遡りうる。その時点のスコアは現在のブランド状態と乖離している。200 行（毎日スキャンなら約 6〜7 ヶ月）をソフト上限として、「連続性の維持」と「データの関連性保持」の間で均衡を取る。上限を超えればキャリーフォワードせず null を記録し、UI に「データ再構築待ち」を明示する。

4.4.2 初回スキャンのブランドはどうするか

戦略：初回スキャンのブランドがプラットフォーム失敗に遭遇した場合、**キャリーフォワードしない**。履歴ベースラインがないから、0 や平均を強引に持ち込むのは偽データになる。フロント側は「初回スキャン—データ構築中」を表示し、「履歴不在」と「履歴あるが停滞」を明確に区別する。

4.4.3 Phase ベースラインテストとの相互作用

Phase ベースラインテスト（第 10 章）は**独立したデータ経路**を歩み、Stale Carry-Forward の影響を受けない。ベースラインテストの目的は縦断的な真の変化を追跡することにあり、いかなるキャリーフォワードもこの目的を汚染する。ベースラインテストがプラットフォーム失敗に遭えば、その Phase 結果は直接 `status = incomplete` とマークし、回復後に再実行する。

4.4.4 Stale がどれだけ続けば警告にすべきか

Stale マーク自体は警告ではない。しかし**同一プラットフォームが 72 時間を超えて停滞**したら、システムは通知をエスカレートする：

- フロントバッジを赤い常駐から赤い点滅に変更
- ツールチップ文言を「⊟ 長期音信不通、設定を確認してください」に変更
- ブランド管理者に email 通知をトリガー

7 日を超えても未回復なら、さらに `Expired` 状態に進み、UI で「データ期限切れ、現採点には含めず」を明示、同時にキャリーフォワードを停止する。

4.5 データ鮮度の哲学

キャリーフォワードの正当性はある前提に立つ：**AI 認識の変化は週単位である。**モデル再訓練、ナレッジグラフ更新、外部ニュースイベントの影響はいずれも時間単位の変動ではない。したがってデータ欠損時に昨日の値で今日を代替するのは**統計的に合理的**である。

しかしこの前提には境界がある：

- **高頻度変動指標**（ソーシャル声量、Google Trends など）では、キャリーフォワードは 24 時間を超えると不正確になる
- **低頻度安定指標**（Schema.org 完全度、GBP 検証状態など）では、キャリーフォワードは数週間でも許容しうる
- **進行中のイベント**（PR 危機、製品発表など）では、キャリーフォワードの平滑化効果はむしろリスクになる

本プラットフォームはキャリーフォワードを **AI シテーション率関連の次元**（Citation、Position、Sentiment）に限定し、構造化データ状態、フィンガープリント照合等の他指標には適用しない。

4.6 関数スケルトン

```
// Simplified illustration — actual implementation handles multi-platform fanout,
// lookback tuning, and emits observability events.
async function enrichWithStaleCarryForward(platform, brandId, currentResult) {
  const currentFailed = currentResult.successCount === 0;
  if (!currentFailed) {
    return { ...currentResult, isStale: false };
  }

  const lastSuccess = await db.query(`
    SELECT sov_score, position_quality, sentiment, scanned_at
      FROM scan_results
     WHERE brand_id = $1 AND platform = $2
       AND sov_score IS NOT NULL AND sov_score > 0
  ORDER BY scanned_at DESC
     LIMIT 1
    OFFSET 0
  `, [brandId, platform]);

  if (!lastSuccess.rows.length) {
    return { ...currentResult, isStale: false, reason: 'no_baseline' };
  }

  const historical = lastSuccess.rows[0];
  const ageHours = (Date.now() - historical.scanned_at.getTime()) / 3_600_000;

  if (ageHours > MAX_CARRY_FORWARD_HOURS) {
    return { ...currentResult, isStale: true, expired: true };
  }

  return {
    sov_score:        historical.sov_score,
    position_quality: historical.position_quality,
    sentiment:        historical.sentiment,
    isStale:          true,
```

```
    lastSuccessAt:    historical.scanned_at,
    staleAgeHours:    ageHours,
  };
}
```

図 4-3：フロントエンド isStale バッジの模式

```
  OpenAI GPT-4o       55 点    14 時間音信不通
  Anthropic Claude    62 点
  Google Gemini       48 点    14 時間音信不通
```

図 4-3：赤い丸とツールチップが明示する——「このスコアは前回成功値で、現在の値ではない」。ユーザーは誤認しえない。

4.7 他分野への応用可能性

Stale Carry-Forward は GEO 固有のパターンではない。「高頻度サンプリング、不安定ソース、時系列連続性が必要」な信号系に応用できる：

分野	応用シーン	調整すべきパラメータ
IoT センシング	センサー間欠断絶	lookback ウィンドウを「分単位」に
金融レート	取引所の短期断絶	不適用（金融のリアルタイム性は妥協できない）
ソーシャルモニタリング	API クオータ枯渇	lookback を短く、変動性が高いため
広告アトリビューション	Pixel 一時ロス	probabilistic matching と組み合わせ必要
サプライチェーン可視化	EDI 転送中断	キャリーフォワード期間を長めに（数日）

共通の前提は：**変化速度 « サンプリング速度**。この前提が崩れる領域（金融等）ではキャリーフォワードは成立しない。

要点

- Stale Carry-Forward の核心は**「ブランド状態」と「パイプライン健全性」の 2 種類の信号を分離**すること
- 3 つの直感的解決策（動的分母 / 失敗即 0 / 沈黙で捨てる）はいずれも時系列データの可読性を破壊する
- 設計は検出・履歴探索・タグ付け・UI での誠実告知の 4 手順。**UI が stale 状態を隠してはならない**
- lookback 上限、初回スキャン処理、Phase ベースラインとの分担が 3 つの主要トレードオフ
- このパターンは「変化速度 « サンプリング速度」の信号系全般に拡張可能

参考文献

- 第 3 章—7 次元 GEO 採点アルゴリズム
- 第 5 章—複数プロバイダ AI ルーティング
- 第 10 章—Phase ベースラインテスト

- Kleppmann, M. (2017). Designing Data-Intensive Applications. O’ Reilly.（第 8 章「分散システムの問題点」は部分障害処理の一般参考）

第 5 章—複数プロバイダ AI ルーティング：先回り型フォールトトレラント設計

> 単一 AI ベンダーに依存するシステムはすべて脆弱である。複数経路のフォールトトレランスはアーキテクチャの出発点であるべきで、事故後のパッチではない。

目次

- 5.1 単一ベンダー依存の構造的リスク
- 5.2 modelRouter のアーキテクチャ
- 5.3 3 つの工学的課題
- 5.4 再試行重ね掛けの罠
- 5.5 スイッチのトリガーと可観測性
- 5.6 副経路のヘルスマネジメント
- 5.7 関数スケルトン
- 要点
- 参考文献

5.1 単一ベンダー依存の構造的リスク

「どう耐障害するか」を議論する前に、**なぜ必要か**を明確にしておく。単一 AI ベンダーへの依存には、ベンダーの規模・ブランドを問わず回避不能な**構造的**リスクが 5 つある：

1. **共用トークンバケット** —全顧客・全モデルがひとつのレート制限プールを共有し、単一顧客のトラフィックスパイクが全体の減速に波及する
2. **モデル廃止 / バージョン切替** —ベンダーが旧モデル（`gpt-4-0314`、`claude-2` 等）を廃止する際、依存していた全機能を即座に切り替えねばならない
3. **リージョン障害** —単一クラウドリージョンの障害が該当地域の全ユーザーを遮断する
4. **アカウント / 請求問題** —クレジットカード期限切れ、決済遅延、API キーローテーション失敗が完全断絶を引き起こす
5. **ポリシー / 地政学的変動** —ベンダーのサービス範囲調整、特定地域の規制変更、モデル利用規約変更

強調しておくべきは：**この 5 種のリスクは「起きたら対処する」ものではなく「必ず起きる」ものである。**いかなる単一ベンダー依存も、これら 5 種のリスクを自社のサービス SLA に埋め込むことと等価である。SaaS にとって受け入れられない設計である。

5.2 modelRouter のアーキテクチャ

百原 GEO は**主経路 / 副経路**の二経路抽象層、`modelRouter` サービスを採用する：

図 5-1：二経路ルーティングアーキテクチャ

```
flowchart LR
    App[アプリ層 <br/>scan / eval / hallucination] --> MR[modelRouter]
    MR -->| 主経路 | Agg[OpenAI 互換 <br/> 集約中継]
    MR -->| 副経路 | D1[OpenAI 直接]
    MR -->| 副経路 | D2[Anthropic 直接]
    MR -->| 副経路 | D3[Google 直接]
    MR -->| 副経路 | D4[DeepSeek 直接]
```

```
    MR -->| 副経路 | D5[Moonshot 直接]
    MR -->| 副経路 | Dn[他ベンダー直接]
    Agg --> LLM1[GPT-4o / Claude / Gemini / ...]
    D1 --> LLM1
```

図 5-1：アプリ層は抽象インターフェース *modelRouter.complete()* を呼び出し、底層の経路選択はアプリに透明である。主 / 副判断はルーター内部に封じ込められる。

二経路の分業

経路	性質	用途
主経路（集約中継）	1 つの API エンドポイントが複数モデルを包摂、課金統一、導入コスト低	日常スキャン、採点、一般推論
副経路（ベンダー直接）	ベンダーごとに独立 API キー、独立課金、各自 SDK	主経路失敗時の自動切替、または特定ベンダー固有機能が必要な場合

主経路の利点は**一度接続すれば各所で使える**こと。副経路の利点は**独立性**。両者は `modelRouter` の**抽象インターフェース**で疎結合になり、アプリ層は現在の要求がどちらの経路を通っているかを知る必要がない。

5.3 3 つの工学的課題

二経路は単純そうに聞こえるが、実装すると 3 つの厄介な問題に遭遇する。

5.3.1 モデル ID が集約層と原生層で異なる

最も過小評価されやすい問題。同じモデルが集約中継とベンダー原生 API で異なる識別子文字列を持つことが多い：

ベンダー原生 ID	集約中継でよくある ID
`deepseek-chat`	`deepseek-v3`
`moonshot-v1-8k`	`kimi-k2`
`claude-3-5-sonnet-20241022`	`claude-3-5-sonnet`
`qwen-plus`	`qwen3-plus`
`meta-llama/Llama-3.3-70B-Instruct`	`llama-3.3-70b`

このマッピングを処理しないと：**主経路で成功したモデル名を直接 API に投げると即 404。**解決策はマッピングテーブル `DIRECT_MODEL_ID_MAP` の維持：

```
const DIRECT_MODEL_ID_MAP = {
  // aggregator ID → direct provider ID
  'deepseek-v3':          { provider: 'deepseek',  model: 'deepseek-chat' },
  'deepseek-r1':          { provider: 'deepseek',  model: 'deepseek-chat' },
  'kimi-k2':              { provider: 'moonshot',  model: 'moonshot-v1-8k' },
  'qwen3-plus':           { provider: 'alibaba',   model: 'qwen-plus' },
  'grok-2':               { provider: 'xai',       model: 'grok-2-latest' },
  'claude-3-5-sonnet':    { provider: 'anthropic', model: 'claude-3-5-sonnet-20241022' },
  // ... full mapping covers 15+ entries
};
```

モデル追加や廃止のたびに、このテーブルを同期更新しなければ、ある経路が静かに機能停止する。

5.3.2 extraParams の差異

異なるモデルの原生 API はそれぞれ固有の必須パラメータを持つ。集約層は通常これらに既定値を設定してくれるが、直接に切り替えるとこれらのパラメータは自前で与える必要がある:

モデル系列	必要な extraParams
Qwen3 系列	`enable_thinking: false`（でなければ推論過程を返しトークン大量消費）
DeepSeek-R1	既定で推論モード、応答が遅い。`temperature: 0.6` で出力安定
Claude（Anthropic SDK）	`max_tokens` **必須**（OpenAI 系列は省略可）
Gemini	`safetySettings` 既定が厳しすぎ、商業コンテンツが誤ブロックされないよう緩和が必要

実装上は各プロバイダごとに `DIRECT_EXTRA_PARAMS` テーブルを保持し、経路切替時に自動マージする。

5.3.3 特殊な推論モデルのトレードオフ

DeepSeek-R1 と **OpenAI o1/o3** は「推論型」モデルで、生成前に内部思考を行うため応答時間が 15〜30 秒になることが多く、我々のスキャナーの 25 秒タイムアウトを超える。

解決策は timeout を延ばす（全体スキャンペースを崩す）ことではなく、**フォールバック時に同ベンダーの非推論版に切り替える**:

主経路モデル	フォールバック直接モデル	妥協
`deepseek-r1`	`deepseek-chat`	reasoning_content フィールドを失う
`o1-mini`	`gpt-4o-mini`	推論チェーンを失うが、シテーション検出は推論に依存しない

GEO スキャンにとって、**シテーション検出は最終テキストさえあれば十分**。推論チェーンは加点であって必須ではない。reasoning_content を犠牲に完了率を取る合理的トレードオフである。

5.4 再試行重ね掛けの罠

これは実務で最もよく踏む落とし穴である。典型的システムには少なくとも 3 層の再試行がある:

```
Business Layer
                withRetry(fn, { retries: 3 })
Router Layer
                主経路失敗で副経路切替（= もう 1 回の試行）
SDK Layer
                OpenAI SDK 組込 maxRetries: 2
```

3 層を掛け合わせると、OpenAI SDK timeout 30s × 2 retries × withRetry 3 回 × 主副切替 1 回 = **最悪 7 分**かけてやっと失敗確定する。スキャナーは 1 リクエスト 25 秒以内の応答を期待しているのに、単一リクエストがワーカー全体を 10 倍以上塞ぐ結果になる。

正しい設計原則

> 最外層のみが「いつ諦めるか」を決める。内層はすべて `maxRetries: 0`、`timeout` を明示的に設定する。

具体的実装:

```
// SDK layer: no internal retry
const openai = new OpenAI({
  apiKey: API_KEY,
  maxRetries: 0,
  timeout: 20_000, // 20s hard limit
});

// Router layer: single failover, no loop
async function routeComplete(request) {
  try {
    return await primaryPath(request);
  } catch (err) {
    if (isRetryableError(err)) {
      return await fallbackPath(request);
    }
    throw err;
  }
}

// Business layer: caller decides retry policy
const result = await withRetry(
  () => modelRouter.complete(request),
  { retries: 2, backoff: 'exponential' }
);
```

3 層がそれぞれの責務を担当し、総経過時間の上限 = 20s × 2（主 + 副）× 3（外層再試行）= 120s。予測可能、モニタリング可能、制限可能。

5.5 スイッチのトリガーと可観測性

全てのエラーがスイッチを発動すべきとは限らない。スイッチしても無意味なエラーがある——副経路の枠を無駄に消費するだけになる。

スイッチ判断ツリー

```
flowchart TD
    E[Primary Path Error] --> C{Error Type}
    C -->|HTTP 5xx| S[副経路に切替]
    C -->|HTTP 429 rate-limit| S
    C -->|Timeout| S
    C -->|Network Error| S
    C -->|HTTP 401 auth| X[直接投げる <br/> 切替しても無意味]
    C -->|HTTP 400 param| X
    C -->|HTTP 402 payment| X
    C -->|Model not found| S
    S --> R[Record Event<br/>provider, model, reason, duration]
```

```
    X --> R
    R --> O[Emit metrics<br/>for observability]
```

図 5-2：スイッチ判断ツリー。5xx / 429 / timeout / ネットワークエラーは「一時的エラー」で切替可。401 / 400 / 402 は「恒久的エラー」で切替すべきでない。

可観測性に必要なフィールド

各スイッチイベントに以下を記録する（後続の分析・アラート用）：

フィールド	用途
`provider_primary`	主経路プロバイダ
`provider_fallback`	副経路プロバイダ（発動した場合）
`model_requested`	アプリ層が要求したモデル ID
`model_actual`	実際使用したモデル ID（フォールバック時は異なりうる）
`reason`	transient / permanent / timeout / model_not_found 等
`latency_primary_ms`	主経路の経過時間（timeout 待ちを含む）
`latency_fallback_ms`	副経路の経過時間
`status`	success_primary / success_fallback / both_failed

これらのメトリクスが蓄積されれば重要な問いに答えられる：「どのプロバイダが最も不安定?」「どのモデルのフォールバック発動率が高い?」「どの時間帯にトラブルが出やすい?」。このデータなしには顧客クレームに頼って debug するしかない。

5.6 副経路のヘルスマネジメント

副経路の最悪の状態は「失敗している」ことではなく、**「普段使われず、本当に必要な時に壊れていると気付く」**ことである。以下 3 つの仕組みでリスクを緩和する。

5.6.1 起動時 Smoke Test

ワーカー起動ごとに**全副経路プロバイダ**に最小リクエスト（`"ping"` または 1 トークン completion）を送る：

```typescript
async function smokeTestFallbacks() {
  const providers = Object.keys(DIRECT_CLIENTS);
  const results = await Promise.allSettled(
    providers.map(p => pingProvider(p, { timeout: 5_000 }))
  );
  return results.map((r, i) => ({
    provider: providers[i],
    ok: r.status === 'fulfilled',
    error: r.status === 'rejected' ? r.reason.message : null,
  }));
}
```

結果は startup log と `/health/fallbacks` エンドポイントに出力する。どこかの経路が失効すれば運用側がすぐ気付ける。

5.6.2 定期ヘルスチェック

30 分ごとに副経路に軽量チェックを行う（極低コストのプロンプト `"OK"` で 2 トークン応答）。3 回連続失敗で UI のプロバイダ一覧に「副経路利用不可」を表示する。

5.6.3 カバー不能なベンダー

全プロバイダに副経路が用意できるわけではない。よくある 3 種:

タイプ	例	処理戦略
クローズドソースで公開 API なし	一部社内企業モデル	UI で「副経路なし」を明示、主経路失敗は即 fail
顧客が自前 API キーを要する	一部有料フロンティアモデル	顧客が設定ページに自分で入力、空欄なら自動フォールバックしない
地政学的制限	一部国際跨ぎサービス	顧客所在地域により動的に有効化判断

透明性は「カバーしているふり」より重要。顧客に「このプロバイダには副経路がない」と告げる方が、事故時に気付かれるより遥かに良い。

5.7 関数スケルトン

5.7.1 modelRouter メインエントリ

```
export async function complete({ prompt, model, temperature, ...opts }) {
  const request = buildRequest({ prompt, model, temperature, ...opts });

  try {
    const result = await callPrimary(request);
    emitMetric({ status: 'success_primary', model });
    return result;
  } catch (err) {
    if (!isRetryableError(err)) {
      emitMetric({ status: 'permanent_fail', reason: err.code, model });
      throw err;
    }

    const fallback = resolveFallback(model);
    if (!fallback) {
      emitMetric({ status: 'no_fallback_available', model });
      throw err;
    }

    const fallbackReq = mapRequestForProvider(request, fallback);
    try {
      const result = await callDirect(fallback.provider, fallbackReq);
      emitMetric({
        status: 'success_fallback',
        provider_fallback: fallback.provider,
        model_actual: fallback.model,
      });
      return result;
```

```
    } catch (fbErr) {
      emitMetric({ status: 'both_failed', model });
      throw fbErr;
    }
  }
}
```

5.7.2 resolveFallback

```
function resolveFallback(aggregatorModelId) {
  // 1. Exact match in DIRECT_MODEL_ID_MAP
  if (DIRECT_MODEL_ID_MAP[aggregatorModelId]) {
    return DIRECT_MODEL_ID_MAP[aggregatorModelId];
  }

  // 2. Pattern match for versioned models
  //    e.g. "deepseek-r1-0528" → "deepseek-chat"
  for (const [pattern, target] of DIRECT_PATTERN_MAP) {
    if (pattern.test(aggregatorModelId)) return target;
  }

  // 3. No fallback available
  return null;
}
```

この 2 つの関数がルーティングアーキテクチャの核心。token counting、prompt assembly、response normalization はすべてこの上に層を積む。

要点

- 単一ベンダー依存は構造的リスクであって「余裕ができたら対応」オプションではない
- `modelRouter` は主 / 副二経路を抽象化し、アプリ層は底層選択に無感
- 3 つの核心課題：モデル ID マッピング、extraParams の差異、推論モデルのタイムアウトトレードオフ
- 再試行は最外層に集中すべきで、SDK と router 層は `maxRetries: 0` に、指数的経過時間を回避
- スイッチ判断はエラータイプ依存、transient なら切替、permanent なら直接投げる
- 副経路は能動的ヘルスマネジメントが必要（smoke test + 定期チェック + カバー不能ベンダーの透明な明示）

参考文献

- 第 2 章—システム全景
- 第 4 章—Stale Carry-Forward
- OpenAI. Node.js SDK retry configuration. https://github.com/openai/openai-node
- Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2nd ed.). Pragmatic Bookshelf.（Circuit Breaker、Bulkhead、Timeout の古典的参考）

第 6 章—AXP シャドウドキュメント：Cloudflare Worker で AI ボットにクリーンなコンテンツを配信

> 人間向けサイトと AI 向けコンテンツは同じ HTML であってはならない。同じ文書で両者に仕えようとすれば、両者とも損をする。

目次

- 6.1 同じ HTML で両方に仕えると両方が損する理由
- 6.2 AXP：シャドウドキュメントの構造
- 6.3 Cloudflare Worker 注入機構
- 6.4 AI ボット UA 一覧と検出戦略
- 6.5 SaaS 自社ブランドのパス競合
- 6.6 Sitemap 自動生成
- 6.7 JSON-LD フラット化の落とし穴
- 6.8 GSC インデックスの血涙ノート
- 6.9 シャドウ公開ファイルスイート：llms.txt / llms-full.txt / schema.json / feed.xml
- 6.10 RAG 知識ベース連携：ブランド専用 KB の自動化
- 要点
- 参考文献

6.1 同じ HTML で両方に仕えると両方が損する理由

現代のウェブサイトは人間向けに設計されている：

- クライアントサイドレンダリング（CSR）—JavaScript が走らないとコンテンツが出てこない
- 動的ロードのカード、カルーセル、モーダル
- Cookie 同意バナー、広告トラッカー、A/B テスト SDK
- 意味不明な `<div class="col-md-6">` が何十層もネスト
- 背景動画、WebGL、アニメーション

これらは**人間**にとっては UX だが、**AI クローラー**にとっては雑音である。AI ボット（GPTBot、ClaudeBot、PerplexityBot、Googlebot など）が現代ブランドページを取得すると、3 種類の失敗がよく起きる：

1. **JS 実行失敗またはタイムアウト** —多くの AI ボットは JS を実行しないか、制限付きでしか実行しない。SPA ページは `<div id="app"></div>` だけを取得する
2. **本体コンテンツ抽出失敗** —HTML ノイズが多すぎて、ブランド情報と UI 装飾を AI が区別できない
3. **構造化データ欠落** —Schema.org JSON-LD が動的生成の位置に置かれ、AI が取得できない

結果、AI のブランド認識は**誤り**か**薄っぺら**になる。解決策はサイト全体を AI に合わせて改造することではなく、**AI 専用のクリーンなシャドウコンテンツを別に用意すること**である。

図 6-1：同一ブランドの 2 種類の視点

```
flowchart LR
    Brand[ブランドデータ] --> H[人間版サイト <br/>React / Vue / Next.js<br/> 完全 UI / アニメ / 広告]
    Brand --> A[AI 版シャドウドキュメント <br/>Pure HTML + JSON-LD<br/> 純粋な意味、雑音なし]
    H -->| ブラウザ | User[ユーザー体験]
    A -->|AI ボット | LLM[モデル訓練と検索]
```

図 6-1：同じブランドデータから 2 つの表現を派生させる。人間版は体験最適化、AI 版は意味最適化。

6.2 AXP：シャドウドキュメントの構造

AXP（AI-ready eXchange Page）は百原がこの種のシャドウドキュメントに付けた名前。1 つの AXP ページは 3 つの層から成る：

図 6-2：AXP 文書の 3 層構造

```
flowchart TB
    subgraph AXP["AXP シャドウドキュメント"]
      HTML[" 純粋 HTML 意味骨格 <br/>h1 / h2 / p / ul / table"]
      JSONLD[" Schema.org JSON-LD<br/>Organization / Service / Person<br/> 三層 @id 相互リンク"]
      MD[" Markdown 生テキストブロック <br/>RAG チャンク化とベクトル化用"]
    end
    HTML --> AI[AI ボット消費]
    JSONLD --> AI
    MD --> AI
```

図 6-2：3 層が揃うことで異なるタイプの AI クローラーがそれぞれ必要な情報を取得できる。純粋 HTML は粗粒度の取得、JSON-LD はナレッジグラフ、Markdown は RAG 用。

3 層は同一 URL の応答内に共存する：HTML を本体とし、JSON-LD を `<script type="application/ld+json">` に、Markdown を `<script type="text/markdown" id="axp-markdown">` に格納する。

6.3 Cloudflare Worker 注入機構

AXP の配信方法は**エッジ注入**——CDN レベルでリクエストを傍受し、User-Agent に応じて返すコンテンツを決める。我々は Cloudflare Workers を採用する。

図 6-3：Worker ルーティング判断フロー

```
flowchart TD
    Req[HTTP Request] --> UA{User-Agent<br/> が AI ボットと一致?}
    UA -->|Yes| Cache{Worker Cache<br/> ヒット?}
    UA -->|No| Pass[オリジンにパススルー <br/> 顧客 origin]
    Cache -->| ヒット | Serve[キャッシュのシャドウ文書を返す]
    Cache -->| ミス | Fetch[geo.baiyuan.io API から <br/> 該当ブランドのシャドウ文書を取得]
    Fetch --> Store[Worker Cache に書込 <br/>TTL 15 分]
    Store --> Serve
    Pass --> Origin[顧客 Origin Server]
```

図 6-3：Worker はキャッシュを優先し、ミス時のみオリジンに AXP を取りに行く。人間のリクエストは直接パススルーされ、オリジンと同じレイテンシ。

Worker 擬似コード

```
export default {
  async fetch(request, env) {
    const ua = request.headers.get('user-agent') || '';
    const url = new URL(request.url);

    // 1. AI ボットではない：オリジンにパススルー
    if (!isAIBot(ua)) {
      return fetch(request); // proxy to customer origin
    }
```

```
    // 2. AI ボット: キャッシュを試す
    const cacheKey = `axp:${url.hostname}:${url.pathname}`;
    const cached = await env.KV.get(cacheKey);
    if (cached) {
      return new Response(cached, {
        headers: { 'content-type': 'text/html; charset=utf-8' },
      });
    }

    // 3. ミス: オリジンで AXP を取得
    const axpUrl = `https://api.geo.baiyuan.io/axp?host=${url.hostname}&path=${url.pathname}`;
    const axpRes = await fetch(axpUrl);
    if (!axpRes.ok) {
      return fetch(request); // AXP 取得失敗、オリジンにフォールバック
    }

    const body = await axpRes.text();
    await env.KV.put(cacheKey, body, { expirationTtl: 900 });
    return new Response(body, {
      headers: { 'content-type': 'text/html; charset=utf-8' },
    });
  },
};
```

重要な設計ポイント:

- AI ボットリクエストと人間のリクエストは**完全に異なる経路を通る**
- AXP 取得失敗時は**必ずオリジンにフォールバック**、顧客の AI トラフィックを 404 させない
- Cache TTL 15 分は「鮮度 vs バックエンド負荷」のトレードオフ

6.4 AI ボット UA 一覧と検出戦略

現在百原プラットフォームは **25 種類**の AI ボット UA を識別し、機能により 4 群に分類する:

図 6-4: AI ボット UA の分類

```
flowchart LR
    subgraph LLM[" 大規模言語モデルクローラー (10)"]
      GPTBot
      ClaudeBot
      GoogleExtended[Google-Extended]
      CCBot[CCBot / Common Crawl]
      MetaBot[FacebookBot / Meta-ExternalAgent]
      ByteBot[Bytespider]
      AnthropicBot[anthropic-ai]
      AppleBot[Applebot-Extended]
    end
    subgraph Search[" 検索型クローラー (6)"]
      Perplexity[PerplexityBot]
      ChatGPTUser[ChatGPT-User]
      PerplexityUser[Perplexity-User]
      YouBot[YouBot]
```

```
      PhindBot
    end
    subgraph Preview["Preview / Link 生成 (5)"]
      LinkedInBot
      FacebookExt[facebookexternalhit]
      TwitterBot[Twitterbot]
      DiscordBot[Discordbot]
    end
    subgraph RAG["RAG / 企業クローラー (4)"]
      CohereBot[cohere-ai]
      DiffBot[Diffbot]
      OmigoBot[Omigo]
    end
```

図 6-4：25 種類の AI ボットを群別で分類。百原プラットフォームは 4 群すべてに AXP 注入を有効化しているが、顧客ニーズに応じて admin で群ごとに無効化できる。

検出戦略

実装では**正規表現マージ**を採用、if-else 連打ではなく保守性優先：

```
const AI_BOT_REGEX = new RegExp(
  [
    'GPTBot', 'ChatGPT-User', 'OAI-SearchBot',
    'ClaudeBot', 'anthropic-ai', 'Claude-Web',
    'Google-Extended', 'GoogleOther',
    'PerplexityBot', 'Perplexity-User',
    'CCBot', 'Bytespider', 'FacebookBot',
    'Meta-ExternalAgent', 'Applebot-Extended',
    'cohere-ai', 'Diffbot', 'YouBot', 'PhindBot',
    'LinkedInBot', 'facebookexternalhit', 'Twitterbot',
    'Discordbot', 'Omigo', 'DuckAssistBot',
  ].join('|'),
  'i'
);

function isAIBot(ua) {
  return AI_BOT_REGEX.test(ua);
}
```

UA 一覧は四半期ごとに見直す。新出現のクローラー（`OAI-SearchBot` が 2025 年 7 月に初登場など）は即時追加が必要。

6.5 SaaS 自社ブランドのパス競合

実務上の特殊ケース：**SaaS プラットフォーム自身がその SaaS のユーザーでもあるとき**（dogfooding）、自社ドメインは「プラットフォーム利用者」（ログイン後にプロダクトを使う）と「ブランドサイト訪問者」（匿名で紹介を読む）の両方に対応する必要がある。

百原自社の `geo.baiyuan.io` がまさにこのシナリオ：

パス	人間ユーザー	AI ボット
/	マーケホーム（公開）	「百原科技」ブランドページの AXP
/dashboard	ログイン後ダッシュボード（プライベート）	403、AXP 化すべきでない
/features, /pricing	プロダクト紹介（公開）	対応サービスページの AXP
/login, /signup	ログイン / 登録（公開だがブランド情報なし）	注入せず、パススルー

判断ツリー

```
flowchart TD
    R[Request] --> B{AI ボット?}
    B -->|No| P1[オリジンにパススルー]
    B -->|Yes| P{パス分類}
    P -->| マーケ / ブランドページ | A[AXP 注入]
    P -->| 認証後機能 | X1[403 Forbidden を返す]
    P -->| 公開だがブランド内容なし | X2[オリジンにパススルー]
```

図 6-5：パス分類表は各ブランドの admin 設定ページで管理。表にない新パスはデフォルトでオリジンパススルー、保守的戦略を採る。

6.6 Sitemap 自動生成

AI ボットのクロール効率は `sitemap.xml` に依存する。AXP モードでは **AXP パスと完全一致する sitemap を動的生成**せねばならない。さもなくば「sitemap に載っているが Worker がそのパスで AXP を注入しない」という混乱が生じる。

v3.0.0 より、百原プラットフォームは**デュアルソースマージ**アーキテクチャで各顧客ドメインの sitemap を自動生成する：

デュアルソース

- **axp_pages**（GEO 最適化ページ）：/overview、/pricing、/faq、/comparison、/features、/fact-check
- **brand_content_pages**（実際のウェブサイトページ）：`sitemapScanner` が顧客オリジン sitemap をクロールして書き込んだもの

同一 URL が両ソースに存在する場合は **AXP バージョンが重複排除優先（dedup priority）**となる。

仕様準拠の出力

- 各 URL は `<loc>` と `<lastmod>` のみ出力——`<changefreq>` も `<priority>` も含まない（sitemap 仕様 §4.2 に準拠；両フィールドは主要検索エンジンに無視されており混乱を招く）

DENY_LIST フィルタリング

すべての URL は出力前に deny list を通過する。/login、/register、/logout、/dashboard/、/admin/、/api/、/cart、/checkout などのプライベートパスや非ブランドパスを除外。

ユーザー操作

AXP Panel → sitemap.xml セクション → 「オリジンから取得」ボタンをクリック。`POST /brands/:id/axp/sitemap/fetch` を呼び出し、設定を `scoring_configs` に書き込みつつ `sitemapScanner` をトリガーしてオリジン sitemap をクロール、結果を `brand_content_pages` に書き込む。プロセス全体は数秒で完了し sitemap が即座に更新される。

セキュリティ

`all-published-pages` 列挙エンドポイントは `INTERNAL_SITEMAP_SECRET` ヘッダーで保護され、競合ブランドの列挙を防止する。

robots.txt で `Sitemap: https://<domain>/sitemap.xml` を積極宣言。Sitemap も CF Worker が注入する。人間が `/sitemap.xml` を入力しても見える（SEO 通念で隠す必要なし）。

6.7 JSON-LD フラット化の落とし穴

Schema.org 仕様は**ネスト配列**を許容するが、実務では以下の問題にぶつかる:

図 6-6: 誤りと正しい例の並列

```
//  誤り: ネスト配列（一部の AI パーサーはブロック全体を拒否する）
{
  "@context": "https://schema.org",
  "@graph": [
    [
      { "@type": "Organization", "name": " 百原科技" }
    ],
    [
      { "@type": "Service", "name": "GEO スキャン" }
    ]
  ]
}

//  正しい: フラット配列
{
  "@context": "https://schema.org",
  "@graph": [
    { "@type": "Organization", "@id": "#org", "name": " 百原科技" },
    { "@type": "Service", "@id": "#svc-scan", "name": "GEO スキャン",
      "provider": { "@id": "#org" } }
  ]
}
```

図 6-6: エンティティ間の関係は配列ネストではなく *@id* 参照で表現する。Schema.org ツール検証の必須要件。

フラット化 + `@id` 連結を維持する利点:

- Google Rich Results ツールで検証通過
- Wikidata / Wikipedia の構造化データ抽出器と整合
- AI のナレッジグラフ構築がより安定

6.8 GSC インデックスの血涙ノート

2024 年から 2025 年にかけての実装で Google Search Console（GSC）インデックスで踏んだ落とし穴:

落とし穴	症状	根因	解決
`noindex` meta の誤上書き	GSC に「`noindex` で除外」表示	UAT 環境の `.env` を誤って PROD にデプロイ	環境変数に `strict` 検査を追加、起動時に不正組み合わせを拒否

落とし穴	症状	根因	解決
canonical のクロスドメイン	PROD ページの canonical が UAT ドメインを指す	同コードベースの 2 環境が canonical ロジックを共有	canonical を `request.hostname` から動的生成
Bot UA 漏れ	GSC 指数は変動するが特定の AI シテーションが消えた	新型 Bot が UA regex に未追加	四半期ごとに CF Worker log の未一致 UA を確認
Sitemap 不整合	GSC に `Discovered – currently not indexed` 警告	AXP ページは存在するが sitemap から漏れ	Sitemap 生成を同じソース（AXP インデックステーブル）から派生
HTTPS/HTTP 混在	robots.txt が HTTP では 200、HTTPS では 404	Worker が `http://` トラフィック未処理	301 → HTTPS 強制、robots も同期注入

これらの落とし穴は AXP 固有ではないが、**AXP がその深刻度を増幅する**——AI ボットの再クロール頻度は Googlebot より低く、1 回のミスが数週間後にやっと再クロール機会を得る。先に `check-prod-seo.sh` スクリプトを CI で走らせ 5 類の問題をチェックする方が、本番上がってから発見するより遥かにコスト安である。

6.9 シャドウ公開ファイルスイート：llms.txt / llms-full.txt / schema.json / feed.xml

`sitemap.xml` に加え、v3.0.0 プラットフォームはブランドごとに 4 つの追加公開 AI 可読ファイルを生成する：

ファイル	URL	用途
llms.txt	/llms.txt	AI クローラー向け Markdown 形式の簡潔なブランドサマリー。主要事実、機能、定価概要、連絡先、sitemap と schema.json へのリンクを掲載
llms-full.txt	/llms-full.txt	詳細な価格、機能一覧、仕様説明を含む拡張版
schema.json	/schema.json	スタンドアロンファイルとして配信する JSON-LD (Schema.org)。`@type` はブランド業種に応じて `Organization`・`SoftwareApplication`・`LocalBusiness` のいずれかを選択
feed.xml	/feed.xml	AXP ページの RSS 2.0 フィード——RSS をフォローする AI プラットフォームがコンテンツ更新を発見できる

4 ファイル共通の特性

- ブランド自社ドメイン（CF Worker Proxy 経由）と SaaS フォールバックパス `/c/:slug/` の両方で提供
- sitemap と同じ DB ソース（`axp_pages`、`ground_truths`、`brand_features`、`brand_services`）から生成
- 同じ DENY_LIST を適用
- ETag / Cache-Control / 304 による効率的な再クロールをサポート
- ブランドドメインの robots.txt に明示宣言：`# For AI/LLM crawlers: LLM-friendly site summary at https://<domain>/llms.txt`

llms.txt 標準の背景

llms.txt フォーマットは新興の llms.txt コミュニティ標準に準拠する——robots.txt がかつてボット通信を標準化したのと同様の位置づけである。PerplexityBot や OAI-SearchBot などの AI クローラーは、ドメインに `/llms.txt` が存在すると既にこれを取得することが知られている。

6.10 RAG 知識ベース連携：ブランド専用 KB の自動化

AXP ページの品質は「AI ボットに配信できるか」だけでなく、**コンテンツに含まれるブランド固有の知識量**に左右される。LLM が推論だけで生成したページは幻覚や曖昧な記述を含みやすい。ブランド自身の知識ベース（KB）から事実を注入することで、出力品質は桁違いに向上する。

6.10.1 ブランド分離：ブランドごとに専用 KB

百原は中央共用 RAG エンジン（§9.4）を採用しているが、各ブランドのドキュメントは `rag_kb_id` で区別された**専用の Knowledge Base（KB）**に格納される：

- 同一テナント内の複数ブランド（例：コスメブランドとフードブランド）の知識は混在しない
- AXP 生成時は `kbId` を指定してクエリを実行し、そのブランド固有の事実のみを取得することを保証
- ブランド削除時に対応する KB を一括削除でき、残留データリスクがない

```
flowchart LR
    subgraph Tenant[" 同一テナント"]
      BrandA[" ブランド A<br/>rag_kb_id: kb-aaa"]
      BrandB[" ブランド B<br/>rag_kb_id: kb-bbb"]
    end
    subgraph RAG[" 中央 RAG エンジン"]
      KBA["KB: kb-aaa<br/> (ブランド A 専用) "]
      KBB["KB: kb-bbb<br/> (ブランド B 専用) "]
    end
    BrandA -->|askWithKbId| KBA
    BrandB -->|askWithKbId| KBB
```

Fig 6-8: ブランドレベルの KB 分離。エンジンは共用、知識は汚染されない。

6.10.2 `seedBrandRAGKB`：AXP 有効化時の KB 自動作成とシード

新しいブランドが AXP を有効化（`enableAXP` API）すると、システムはバックグラウンドで非同期に 3 ステップを実行する：

1. **KB 作成**（存在しない場合）—`ragCreateKnowledgeBase` が新しい `kbId` を返し、`brand_rag_configs` に書き込む
2. **ブランドプロフィールをアップロード**（テキストドキュメント）—ブランド名・業種・概要・公式サイト・コアキーワードを構造化テキストとして KB のアンカー知識に
3. **公式サイトページ URL をアップロード**（最大 20 件、`geo_importance` 降順）—RAG バックエンドがクロール・ベクトル化し、静的プロフィール知識を補完

```
// enableAXP の setImmediate ブロック内（ノンブロッキング）
await initialCrawl(brandId, tenantId);   // AXP ページ生成
await seedBrandRAGKB(brandId, queryFn);  // 並行: RAG KB シード
```

設計上の考慮点：- **ノンブロッキング**—シード処理は AXP ページの即時レスポンスを遅延させない - **冪等性**—繰り返し呼び出しても新規ドキュメントを追加するのみで、KB の重複は生じない - **グレースフルデグラデーション**—URL のクロール失敗はサイレントにスキップし、他の URL の処理を継続

6.10.3 キーワード注入：ブランドの声を AXP コンテンツに

`brand.keywords`（ブランドが設定したターゲット GEO/SEO キーワード）は、`keywordsHint` サフィックスとしてすべての RAG クエリに注入される：

```
// hybridCoordinator.service.js
const keywordsHint = keywords.length
  ? `\n\n以下のターゲットキーワードを自然に組み込んでください（すべて登場する必要はありません）: ${keywords.j
  : '';
const question = PAGE_TYPE_QUESTIONS[pageType](brandName) + keywordsHint;
```

`pricing_summary`（定価ページ）と `product_features`（機能ページ）は「純粋なテーブル、キーワードゼロ」になりやすいため、RAG プロンプトはテーブルの前に**キーワードを豊富に含む導入段落**の生成を明示的に要求する：

1. 導入段落（2〜3 文）：ブランドのポジションを説明し、ターゲットキーワードを自然に組み込む
2. 完全な定価テーブル：知識ベースで確認済みのデータのみを掲載し、推測・捏造は不可

実測キーワードカバレッジ（`ILIKE` 部分文字列マッチ、5 ブランド × 6 ページタイプ、2026-04-21）：

ブランド	キーワード数	最低カバレッジ	最低カバレッジ率	大多数ページ
百原 GEO	13	pricing_summary	9/13	11–13/13
百原 RAG 知識庫	12	pricing_summary	7/12	11–12/12
愛上生活	10	pricing_summary	7/10	9–10/10
PIF	12	faq / pricing	7/12	10–12/12
百原科技有限公司	10	pricing_summary	1/10 ★	9–10/10

★ ブランド公式サイトに公開定価ページがないため、RAG が推測を正しく拒否。低カバレッジは想定内の動作。

6.10.4 `content_preview`：ページごとの品質監視シグナル

AXP ページ一覧 API に `content_preview` フィールドを追加：複数行 HTML コメントを除去した後の `content_md` 先頭 150 文字をページカードの要約として使用。

これにより実務上の問題が解決される：同一ブランドの 6 ページタイプすべてに同じ `fingerprint_phrase`（ブランド指紋フレーズ）が表示されていると、各ページが正しく生成されているかを確認できなかった。

```
-- 複数行 HTML コメントを除去し、先頭 150 文字を取得
LEFT(REGEXP_REPLACE(content_md, '<!--[\s\S]*?-->', '', 'g'), 150) AS content_preview
```

regex は `[\s\S]*?`（dotall モード）を使用する必要がある。`[^>]*` では複数行にまたがる HTML コメントを正しく除去できない。

要点

- 同一 HTML で人間体験と AI パース可能性を両立させるのは難しい。AXP は分離の必要設計
- AXP 3 層構造：純粋 HTML 骨格 + Schema.org JSON-LD + Markdown 生テキスト
- Cloudflare Worker がエッジで UA を検出、AI ボットと人間は完全に別経路
- 25 種類 AI ボット UA は正規表現マージで保守、四半期ごとに新型を確認
- SaaS 自社ブランドのパス競合は admin で分類表を保守、デフォルトは保守的パススルー
- Sitemap は動的生成で AXP パスと整合、JSON-LD は `@id` フラット化でネスト配列を避ける
- GSC インデックス問題は AI ボットで増幅されるため CI の pre-flight スクリプトでガード
- v3.0.0 より sitemap は AXP ページと実クロール済みページ（brand_content_pages）をマージ；llms.txt / llms-full.txt / schema.json / feed.xml が完全なシャドウ公開ファイルスイートを構成し、異なる AI クローラーの消費パターンに対応
- 各ブランドは独立した RAG KB（`rag_kb_id`）を持つ；AXP 有効化時に `seedBrandRAGKB` がブランドプロフィールと公式サイト URL を自動アップロード——手動作業不要
- `brand.keywords` を `keywordsHint` として RAG クエリに注入；定価ページと機能ページはキーワードを豊富に含む導入段落を必須とする；`content_preview` が `fingerprint_phrase` に代わるページ品質シグナルとなる

参考文献

- 第 7 章—Schema.org フェーズ 1：25 業種 × 三層 @id
- 第 8 章—GBP API 統合戦略
- Cloudflare. Workers Runtime Documentation. https://developers.cloudflare.com/workers/
- OpenAI. GPTBot User Agent. https://platform.openai.com/docs/gptbot
- Anthropic. ClaudeBot documentation. https://support.anthropic.com/en/articles/8896518

第 7 章—Schema.org フェーズ 1：25 業種 × 三層 @id 相互リンク

Schema.org は「タグを数個追加する」ほど単純ではない。業種特化も、エンティティ相互リンクも、自動生成もなければ、AI にとってほぼ存在しないに等しい。

目次

- 7.1 AI 時代における Schema.org の役割再定義
- 7.2 25 業種特化 @type の設計
- 7.3 三層 @id 相互リンクのナレッジグラフ
- 7.4 物理ビジネス vs オンラインサービス：フィールド重みの分岐
- 7.5 データ完全度アルゴリズム
- 7.6 Wizard + Edit 二入口設計
- 7.7 GBP URL パーサー
- 7.8 関数スケルトン
- 要点

7.1 AI 時代における Schema.org の役割再定義

Schema.org は 2011 年誕生、Google、Bing、Yahoo、Yandex が共同推進する構造化データ語彙である。元来の主用途は**従来型検索エンジン**に Rich Results（星評価、パンくず、FAQ 展開等）を生成させることだった。

2024 年以降、Schema.org の役割には 2 つの根本的変化が起きた：

1. **「検索エンジン装飾物」から「AI 訓練データの構造化源」へ** —主流 LLM は事前訓練で Common Crawl を消化する。Schema.org JSON-LD はその中で最も密度の高いエンティティデータ源である
2. **「加点項目」から「必須項目」へ** —Schema.org のないサイトは AI の目に「一塊のテキスト」、Schema.org のあるサイトは「識別可能なエンティティ」。視覚障害者にとっての「画像に alt があるか」と同レベルの違い

本書は Schema.org を**百原 GEO 最適化パスの第一のレバー**と位置づける。Schema.org 構造が整わないブランドは、他の次元をどう最適化しても AI 認識が安定しない。

7.2 25 業種特化 @type の設計

Schema.org 仕様には数百の `@type` があり、多くは特化度が高い（例：`MedicalClinic`、`VeterinaryCare`、`CafeOrCoffeeShop`）。**AI にとって、誤った `@type` を選ぶことは自分を誤った引き出しに入れることと等価**である——AI はナレッジグラフ内でエンティティを位置づける鍵次元として `@type` を使う。

百原プラットフォームは一般業種を **25 分類**に整理し、各分類に Schema.org `@type` 組をマッピングする：

図 7-1：25 業種分類表（物理 16 + オンライン 7 + フォールバック 2）

code	日本語名称	Schema.org `@type`
`medical_clinic`	美容クリニック	`MedicalClinic`, `LocalBusiness`
`dental_clinic`	歯科クリニック	`Dentist`, `LocalBusiness`
`general_clinic`	一般診療所	`MedicalOrganization`, `LocalBusiness`
`beauty_salon`	美容室 / サロン	`BeautySalon`, `LocalBusiness`
`fitness`	ジム / ヨガスタジオ	`HealthClub`, `SportsActivityLocation`

code	日本語名称	Schema.org `@type`
`restaurant`	レストラン / 居酒屋	`Restaurant`, `FoodEstablishment`
`cafe`	カフェ	`CafeOrCoffeeShop`
`legal_service`	法律事務所	`LegalService`, `ProfessionalService`
`accounting`	税理士 / 会計士事務所	`AccountingService`, `ProfessionalService`
`real_estate`	不動産仲介	`RealEstateAgent`, `ProfessionalService`
`auto_repair`	自動車整備	`AutoRepair`, `AutomotiveBusiness`
`education_offline`	塾 / 研修	`EducationalOrganization`, `LocalBusiness`
`veterinary`	動物病院	`VeterinaryCare`, `MedicalOrganization`
`lodging`	ホテル / 民宿	`LodgingBusiness`, `Hotel`
`retail_store`	小売店	`Store`, `LocalBusiness`
`financial_service`	金融サービス	`FinancialService`, `ProfessionalService`
`saas_application`	SaaS ソフトウェア	`SoftwareApplication`, `Organization`
`web_application`	Web ツール	`WebApplication`, `Organization`
`mobile_app`	モバイルアプリ	`MobileApplication`, `Organization`
`ecommerce`	純 EC	`OnlineStore`, `Organization`
`online_education`	オンライン学習プラットフォーム	`EducationalOrganization`
`news_media`	メディア / コンテンツサイト	`NewsMediaOrganization`
`online_professional`	オンラインコンサル	`ProfessionalService`, `Organization`
`other_physical`	その他の物理ビジネス	`LocalBusiness`
`other_online`	その他のオンラインサービス	`Organization`

図 7-1: 物理 16、オンライン 7、フォールバック 2。各分類は 2 つの *`@type`* (主 + 副) を指定、Schema.org が *`@type`* を配列にできる仕様を活用。

なぜ 25 分類でありそれ以上細分化しないか

Schema.org 仕様には数百のサブタイプがあるが、**過度な細分化はむしろ AI の認識率を下げる。**理由:

- AI モデルは訓練時に**一般的タイプ**の重みが高い (例: `Restaurant` > `FastFoodRestaurant`)
- 顧客記入時に**選択肢が多すぎると諦める**、25 は「カバー度」と「利便性」の均衡点
- 特化サブタイプは主 `@type` 以外に additional type として付加でき、全員に強制しない

7.3 三層 @id 相互リンクのナレッジグラフ

図 7-2: 三層エンティティのナレッジグラフ

```
flowchart TB
    subgraph L1[" 第 1 層: 主体"]
      Org["Organization / LocalBusiness<br/>@id = #org<br/>name / description / url / logo /<br/>address
    end
    subgraph L2[" 第 2 層: サービス"]
      Svc1["Service<br/>@id = #svc-1"]
      Svc2["Service<br/>@id = #svc-2"]
      SvcN["..."]
    end
    subgraph L3[" 第 3 層: 人員"]
      Emp1["Person / Physician<br/>@id = #emp-1"]
```

```
    Emp2["Person / Attorney<br/>@id = #emp-2"]
  end
  Svc1 -->|provider| Org
  Svc2 -->|provider| Org
  Emp1 -->|worksFor| Org
  Emp2 -->|worksFor| Org
  Svc1 -->|performer| Emp1
  Svc2 -->|performer| Emp2
  Org -.->|sameAs| Wiki[Wikipedia]
  Org -.->|sameAs| WD[Wikidata]
  Org -.->|sameAs| LI[LinkedIn]
  Org -.->|sameAs| GBP[Google Business Profile]
```

図 7-2：三層が *@id* で相互参照しナレッジグラフを形成。外部権威プラットフォームは *sameAs* で跨ナレッジベースリンクを確立。

なぜ blob ではなく三層なのか

実務でよくある誤りは、すべての情報を 1 つの `Organization` に詰め込むこと：

```
{
  "@type": "Organization",
  "name": " あるクリニック",
  "employees": [
    { "name": " 田中医師", "jobTitle": " 院長" }
  ],
  "services": [
    " ボトックス注射", " ヒアルロン酸注入"
  ]
}
```

この書き方の問題：AI は「田中医師」を独立参照可能なエンティティ（Person entity）として扱えない、「ボトックス注射」は文字列であってサービスエンティティ（Service entity）ではない。結果「田中医師に関する質問」がどの構造化データにも対応しない。

三層 @id の書き方はアドレス可能なエンティティを構築する：

```
{
  "@context": "https://schema.org",
  "@graph": [
    {
      "@type": ["MedicalClinic", "LocalBusiness"],
      "@id": "https://example.clinic/#org",
      "name": " あるクリニック",
      "sameAs": [
        "https://www.wikidata.org/wiki/Q...",
        "https://www.linkedin.com/company/..."
      ]
    },
    {
      "@type": "Physician",
      "@id": "https://example.clinic/#emp-1",
      "name": " 田中医師",
      "jobTitle": " 院長",
      "worksFor": { "@id": "https://example.clinic/#org" }
    },
```

```
    {
      "@type": "Service",
      "@id": "https://example.clinic/#svc-botox",
      "name": " ボトックス注射",
      "provider": { "@id": "https://example.clinic/#org" },
      "performer": { "@id": "https://example.clinic/#emp-1" }
    }
  ]
}
```

ユーザーが AI に「ボトックスを担当するのは誰?」と尋ねたとき、AI の推論パスには完全なエンティティチェーンがある。曖昧な文字列マッチングではなく。

7.4 物理ビジネス vs オンラインサービス：フィールド重みの分岐

`is_physical` フラグはフィールド完全度の重みテーブルを決定する。両者の AI シテーション率への影響次元は全く異なる：

図 7-3：フィールド重みの分岐

```
flowchart LR
    subgraph Physical[" 物理ビジネス (is_physical=true)"]
      P1[" 住所 15%"]
      P2["GBP Place ID 15%"]
      P3[" 営業時間 10%"]
      P4[" 電話 10%"]
      P5[" サービス項目 10%"]
      P6[" 従業員リスト 10%"]
      P7["url / name / logo 30%"]
    end
    subgraph Online[" オンラインサービス (is_physical=false)"]
      O1["url 20%"]
      O2["description 15%"]
      O3["logo_url 10%"]
      O4[" サービス / 機能 20%"]
      O5["sameAs 外部リンク 15%"]
      O6["FAQ 10%"]
      O7[" その他 10%"]
    end
```

図 7-3：物理ビジネスは「住所 + GBP」で 30%、オンラインサービスは「url + description」で 35%。同じアルゴリズム、2 つの重みテーブル。ユーザーの AI への主要意図タイプを正しく反映する。

設計根拠

- **物理ビジネス**：ユーザーが AI に尋ねる時は地域を伴うことが多い（例：「渋谷区でレビューの良い美容クリニック」）。AI は Schema.org から住所と営業情報を抽出する必要があり、これらが欠けると回答が「落地」しない
- **オンラインサービス**：ユーザーが尋ねるのは「機能型」質問（「最も良い CRM はどれ?」）。AI が必要なのは記述・差別化・同類比較、住所はむしろ関係ない

百原プラットフォームの UI は `is_physical` により動的にフィールドを表示 / 非表示する：物理類顧客は「住所」「営業時間」カードが見える、オンライン類顧客には表示しない。第 2 章 で言及した「Visibility Module」の具体化である。

7.5 データ完全度アルゴリズム

各フィールドには**重み**（0〜100）があり、記入すれば加点される。総完全度は各フィールド重みの加重平均。核心ロジック：

```
function computeCompletion(brand, industry) {
  const weights = industry.is_physical ? PHYSICAL_WEIGHTS : ONLINE_WEIGHTS;
  let score = 0;
  let maxScore = 0;

  for (const [field, weight] of Object.entries(weights)) {
    maxScore += weight;
    if (isFilledMeaningfully(brand, field)) {
      score += weight;
    }
  }

  return Math.round((score / maxScore) * 100);
}

// 単に非空チェックではなく「意味ある記入か」をチェック
function isFilledMeaningfully(brand, field) {
  const value = getField(brand, field);
  if (!value) return false;
  // プレースホルダーを弾く
  if (typeof value === 'string' && PLACEHOLDER_PATTERNS.test(value)) return false;
  // 関連テーブルは最低 1 件必要
  if (Array.isArray(value) && value.length === 0) return false;
  return true;
}
```

なぜ「非空」だけでは足りないか

初期実装はフィールドが非空かだけを判定していたが、顧客が `url` に `"https://"` だけ、`description` に「会社」とだけ書くプレースホルダーでスコアを稼ぐようになった。`isFilledMeaningfully` は追加で 3 つのチェックを行う：

1. **プレースホルダー正規表現** —`^(https?:\/\/)?$`、`^[a-zA-Z ]{1,3}$`、空白のみなど
2. **最小文字数閾値** —例えば description は 20 文字以上でカウント
3. **形式検証** —URL は解析可能、電話は E.164 形式準拠など

「形式上完全だが実質無用」な事例は特に顧客が GEO ツールに不慣れな初期に多い。UI は記入を止めないが、アルゴリズムはスコアに入れない。後続の最適化示唆を誤導しないため。

7.6 Wizard + Edit 二入口設計

図 7-4：入口フロー

```
flowchart TD
    Start{ユーザータイプ} -->| 新規ブランド | Wiz[Wizard<br/> 直線 7 ステップ]
    Start -->| 既存ブランド | Dash[Dashboard<br/> 完全度バナー]
    Wiz --> W1[Step 1: 基本情報]
```

```
    W1 --> W2[Step 2: 業種と記述]
    W2 --> W3[Step 3: 住所 / 位置 <br/>is_physical のみ]
    W3 --> W4[Step 4: 営業時間 <br/>is_physical のみ]
    W4 --> W5[Step 5: サービス項目]
    W5 --> W6[Step 6: 従業員リスト]
    W6 --> W7[Step 7: FAQ / SNS]
    W7 --> Done[完了]
    Dash -->|<80%| Alert[赤 / 琥珀警告]
    Dash --> Edit[/brands/id/entity<br/> 任意 Card に自由ジャンプ]
    Alert --> Edit
    Edit --> Save[保存即時更新 <br/> 完全度 %]
```

図 7-4：新ブランドは Wizard で初回カバー率を担保、既存ブランドは Edit で自由更新。両パスとも同じ Card コンポーネントを共有（DRY 原則）。

なぜ Wizard は全フィールド必須にしないか

Wizard の各ステップは「一旦スキップ」を許容する：

- **記入疲れ**はユーザーにプロセス全体を放棄させる。一度で 100% より、まず 60% を取る方がよい
- **誘導型 UI** は**強制型 UI** よりユーザーフレンドリー、progressive disclosure 原則に沿う
- Wizard 終了後、Dashboard は継続的にバナーで未記入フィールドを促し、**補完の第二機会**を形成

これはプロダクト哲学の選択である：**まずブランドを AI に存在させ、完璧はその後で追う。**

7.7 GBP URL パーサー

Google Business Profile（GBP）が提供する地点識別は 3 種類の ID があり、顧客はそのうち 1 種の URL しか手元にないことが多い：

ID タイプ	例 URL	用途
`place_id`	`https://www.google.com/maps/place/?q=place_id:ChIJ...`	Place Details API の primary key
`FTID`	`https://maps.google.com/maps?ftid=0x0:0xe6...`	Google Maps 内部識別
`CID`	`https://www.google.com/maps?cid=...`	Customer ID、短縮 URL 形態

図 7-5：GBP URL パーサー判断ツリー

```
flowchart TD
    In[任意の GMB URL を貼付] --> Split{URL タイプ}
    Split -->|place_id: を含む | P[place_id を抽出]
    Split -->|ftid= を含む | F[FTID を抽出 <br/>place_id に変換]
    Split -->|cid= を含む | C[CID を抽出 <br/>Places API クエリ → place_id]
    Split -->| 短縮 URL goo.gl| R[301 を解決 → Split に再入]
    Split -->| その他 | X[null を返す <br/> 完全 URL を貼るよう要求]
    P --> Done[Place ID を返す]
    F --> Done
    C --> Done
```

図 7-5：パーサーは 4 種類の URL 形式それぞれに分岐を持つ。解析不能な URL は明確にエラーを返し、推測しない。

なぜ CID は API クエリが必要か

CID は Google 内部の連番で、直接 Place ID に変換できない。パーサーは Google Places API `findPlaceFromText` で CID から逆引きする:

```
async function cidToPlaceId(cid) {
  const res = await fetch(
    `https://maps.googleapis.com/maps/api/place/findplacefromtext/json?` +
    `input=cid:${cid}&inputtype=textquery&fields=place_id&key=${API_KEY}`
  );
  const data = await res.json();
  return data.candidates?.[0]?.place_id ?? null;
}
```

この呼び出しは Google API クオータを消費する。パーサーは同一 URL に対して 24 時間キャッシュを持ち、重複消費を避ける。

7.8 関数スケルトン

generateBrandEntitySchema

```
function generateBrandEntitySchema(brand, industry) {
  const base = `https://${brand.primary_domain}`;
  const graph = [];

  // Layer 1: Organization / LocalBusiness
  graph.push({
    '@type': industry.schema_types, // 配列、例: ["MedicalClinic", "LocalBusiness"]
    '@id': `${base}/#org`,
    name: brand.name,
    url: brand.url,
    description: brand.description,
    logo: brand.logo_url,
    ...(industry.is_physical && {
      address: buildAddress(brand.location),
      telephone: brand.location?.telephone,
      openingHoursSpecification: buildHours(brand.hours),
      geo: buildGeo(brand.location),
    }),
    sameAs: buildSameAs(brand), // Wikipedia / Wikidata / LinkedIn / GBP
  });

  // Layer 2: Services
  for (const svc of brand.services ?? []) {
    graph.push({
      '@type': 'Service',
      '@id': `${base}/#svc-${svc.slug}`,
      name: svc.name,
      description: svc.description,
      provider: { '@id': `${base}/#org` },
    });
  }

  // Layer 3: Employees
```

```
  for (const emp of brand.employees ?? []) {
    graph.push({
      '@type': emp.specialized_type ?? 'Person', // Physician / Attorney / ...
      '@id': `${base}/#emp-${emp.slug}`,
      name: emp.name,
      jobTitle: emp.job_title,
      worksFor: { '@id': `${base}/#org` },
    });
  }

  return {
    '@context': 'https://schema.org',
    '@graph': graph,
  };
}
```

この関数は AXP 生成フロー（第 6 章）とクローズドループ・ハルシネーション修正（第 9 章）の共通底層基盤である。

要点

- AI 時代、Schema.org は「検索エンジン装飾物」から「AI 訓練データの構造化源」へ昇格
- 25 業種 enum（物理 16 + オンライン 7 + フォールバック 2）で一般ニーズを網羅、同時に利便性を維持
- 三層 `@id` 相互リンク（Organization / Service / Person）で blob データをアドレス可能エンティティに変換
- `is_physical` フラグが 2 つの重みテーブルを発動、物理は「住所 / GBP」重視、オンラインは「url / description」重視
- 完全度アルゴリズムがプレースホルダーと形式的記入を弾き、スコアのインフレを防ぐ
- Wizard が新規ブランドを誘導、Edit は既存ブランドに対応、Card コンポーネントを DRY で共有
- GBP URL パーサーは place_id / FTID / CID の 3 形式に対応

第 8 章—GBP API 統合戦略：事実の主情報源から対外出力への単方向同期

物理ビジネスの情報はウェブサイト、Facebook、LINE 公式アカウント、GBP、プラットフォーム profile に分散している。単一の事実源がなければ、AI が見るのは必ず継ぎ接ぎと矛盾である。

目次

- 8.1 なぜ GBP は物理ビジネスの事実主情報源か
- 8.2 GBP API 申請ゲートとプロセス
- 8.3 代行管理モデルの選択
- 8.4 フィールドマッピング表
- 8.5 同期頻度とクオータ
- 8.6 Webhook がない場合の補完方法
- 8.7 フェーズ 1–4 ロードマップ
- 要点

8.1 なぜ GBP は物理ビジネスの事実主情報源か

物理ビジネスにとって、Google Business Profile（GBP）は最も権威あるエンティティ同定ノードである。理由 3 つ：

1. **Google 検索 + Maps のユーザー接触面**：「渋谷区の美容クリニック」と検索したとき、GBP の地点カードは最初にクリックされる結果になることが多い
2. **AI 訓練データのパイプライン**：Google AI Overview、Perplexity、ChatGPT 等が地域クエリを扱うとき、GBP のビジネス情報を大量に参照する（直接または間接的に）
3. **Google プロダクト間の一貫性**：Google Maps、Search、Assistant、AI Overview がすべて同じ GBP データを共有

図 8-1：データフロー：GBP → Schema.org → AXP → AI

```
flowchart LR
    GBP[(GBP<br/> 事実主情報源)] -->|API 読取 | Sync[百原同期サービス]
    Sync -->| 正規化 | DB[(百原 DB<br/>brand_*)]
    DB -->| 生成 | JSONLD[Schema.org JSON-LD]
    JSONLD --> AXP[AXP シャドウドキュメント]
    AXP -->|CF Worker 注入 | AI[AI ボット]
    AXP -->|sitemap.xml| GB[Googlebot]
```

図 8-1：データは単方向に流れる。GBP 変更 → 百原 DB も変更 → JSON-LD 再生成 → AXP 更新 → AI がクロール。顧客は 1 箇所（GBP）で情報を保守するだけでよい。

核心原則：**同期は単方向。**百原プラットフォームは GBP に**書き戻さない**（少なくともフェーズ 1–2 では）、二重書込衝突と上書きリスクを避けるため。フェーズ 3 で初めて LocalPosts 書込を開放する（§8.7）。

8.2 GBP API 申請ゲートとプロセス

Google Business Profile API は公開登録で即使用できるものではなく、申請承認が必要である。申請プロセスで記録に値する関門：

段階	内容	期間	備考
1. 前提条件	Google Cloud プロジェクト所有 + 検証済み GBP 最低 1 つ	1 日	既存検証済みビジネスがなければ代行管理ブランドの GBP でも可
2. 申請フォーム提出	Use Case 記述、QPM 予測、データ用途	半日	Use Case は「ビジネスオーナー向け管理ツール」型に絞る
3. Google 審査	Google チームが申請適格性評価	7〜10 営業日	公式期間、実際はさらに長くなる場合あり
4. 承認有効化	API クオータ取得、関連 scope 開通	即時	デフォルト QPM 300、追加申請で引き上げ可

よくある却下理由

- Use Case 曖昧（「SEO ツール」はボーダーライン用途とみなされる）
- 検証済みビジネスなし（申請者に管理権がなければならない）
- 提出フォームを「公開フォーラム」に誤入力（公式文書構造が紛らわしい）

百原の戦略：**フェーズ 1（Schema.org 手動記入）段階で検証済み GBP ブランドを蓄積**、フェーズ 2 申請時に 5 件の検証事例を適格性証明として一括提出する。

8.3 代行管理モデルの選択

顧客の GBP を百原プラットフォームが読み取れるようにする方法は 2 種類:

図 8-2：2 つの代行管理モデルの対比

```
flowchart TB
    subgraph Manager[" モデル A: Manager 追加"]
      C1[顧客が GBP 画面で <br/> 百原を Manager として追加]
      C1 --> S1[百原は Service Account で <br/> 該当 Location を読取]
      S1 --> Pro1[" 一度設定すれば長期有効 <br/> 顧客が GBP UI を操作する必要 <br/> 百原に全 Location が
    end
    subgraph OAuth[" モデル B: OAuth 認可"]
      C2[顧客が百原画面で <br/>「Google 連携」クリック]
      C2 --> S2[OAuth Flow で <br/>refresh_token 取得]
      S2 --> Pro2[" ユーザー慣れた UX（FB 連携類似）<br/> 細粒度の権限制御 <br/> refresh_token の定期失効
    end
```

図 8-2：モデル A は単純だがユーザー操作ハードルが高い。モデル B は UI に慣れているが token 保守コストがある。

百原プラットフォームは**二軌並行**：UI はデフォルトで OAuth（B）に誘導するが、顧客の OAuth が権限取得できない場合（Workspace アカウントで管理制限がある等）は Manager 追加（A）にフォールバック誘導する。

8.4 フィールドマッピング表

GBP のデータ構造は Schema.org と完全一致しないため、**明確なマッピング表**が必要。よく使う 12 組の対応:

GBP フィールド	Schema.org property	変換ルール
`title`	`Organization.name`	直接マッピング、前後空白を除去
`storefrontAddress`	`Organization.address`	`PostalAddress` オブジェクトに組成 (複数行住所分解)
`primaryPhone`	`Organization.telephone`	E.164 形式正規化
`websiteUri`	`Organization.url`	解析可能 URL を検証
`regularHours.periods`	`openingHoursSpecification`	曜日 + 開閉時刻配列に変換
`categories.primaryCategory`	`@type` 選択	百原 25 業種 industry_code に対応
`profile.description`	`Organization.description`	最大 750 文字（GBP 制限）
`metadata.placeId`	`Organization.identifier` + `sameAs`	`identifier` に place_id、`sameAs` に Maps URL
`moreHours`	`specialOpeningHoursSpecification`	特別時間帯 (昼休み、祝日）分割
`attributes`	`amenityFeature` / `hasOfferCatalog`	属性タイプで異なる property に配分
`media.photos`	`image` / `logo`	1 枚目を logo、残りを image 配列に
`reviews`	`aggregateRating` + `review`	スコア数 + レビューサンプル（全量ではなく先頭 N 件)

各マッピングルールは `gbpToSchema.js` に純粋関数で実装。テスト時は固定 fixture で期待 JSON-LD 出力を検証する。

8.5 同期頻度とクオータ

GBP API のデフォルトクオータは **300 QPM**（queries per minute)。「データ鮮度」と「クオータ枯渇」の間で取捨する必要がある:

図 8-3：同期頻度マトリクス

フィールド類型	同期頻度	日次 QPS	説明
基本情報（name、address、hours）	1 日 1 回	極低	変動頻度低
営業時間変動	1 時間 1 回	低	臨時休業、特別祝日の即時反映が必要
写真、属性	1 日 1 回	低	視覚コンテンツ更新頻度は中
レビュー	10 分ごと	中	レビューは頻繁に増え、aggregateRating に影響大
Q&A	1 時間 1 回	低	ユーザー問答頻度はレビューより低

図 8-3：1 Location あたり 1 日約 150〜200 回のクオータを消費。300 QPM アカウントで約 2,000 Location 並行対応可。

ブランド数が単一アカウント QPM を超える場合の戦略：

1. **アカウント分散** —Additional QPM Quota を申請（Google の審査で引上可）
2. **バッチ分散** —急ぎでない同期を非ピーク時間に
3. **優先度分散** —完全度低いブランドを優先、高いブランドは頻度下げ

8.6 Webhook がない場合の補完方法

GBP API は **webhook を提供しない**[4]、顧客が GBP を変更しても即時通知がない。補完方法：

- **Notifications API + Pub/Sub**：Google が提供する「間接 webhook」。Location 変動時に Google Pub/Sub topic に push、百原が subscribe すれば数分内に変動シグナルを受信可
- **比較型同期**：読取時に `metadata.updateTime` を比較、時刻が更新されたときだけ JSON-LD を再生成、不要な下流処理を削減
- **顧客手動トリガー**：UI に「今すぐ同期」ボタンを提供、顧客が GBP を変更した直後に即プルを発動

3 層合算で「GBP 変更 → AI 可視」のレイテンシを **5〜10 分**内に圧縮できる。ほとんどのシナリオには十分。将来リアルタイム性の高い応用（「現在営業中か」クエリ等）が開放されれば、さらに最適化を検討する。

8.7 フェーズ 1–4 ロードマップ

図 8-4：GBP 統合の 4 フェーズ

```
flowchart LR
    P1[" フェーズ 1<br/> 進行中 <br/> ローカル Schema.org<br/> 手動記入 + GBP URL 貼付"]
    P2[" フェーズ 2<br/> API 承認待ち <br/>GBP API 読取 <br/> 基本情報自動同期"]
    P3[" フェーズ 3<br/> 計画中 <br/>LocalPosts 書込 <br/>ClaimReview 能動発行"]
```

[4]Google Business Profile API. Notifications Overview. https://developers.google.com/my-business/content/notification-setup

```
P4[" フェーズ 4<br/> 将来 <br/>Organization Account<br/> チェーンブランド一括管理"]
P1 --> P2 --> P3 --> P4
```

図 8-4：4 フェーズは「読 → 書 → 一括」と漸進展開。フェーズ 1 は API 承認に依存せず展開可、フェーズ 2 は API 承認後に有効化、フェーズ 3–4 はビジネスニーズで時期決定。

各フェーズのマイルストーン

フェーズ	機能	依存	状態
フェーズ 1	25 業種 Schema.org、手動記入、GBP URL から Place ID 抽出、Wizard+Edit、AXP 注入	外部依存なし	□ リリース済み (v2.19.x)
フェーズ 2	GBP API 読取：基本情報、営業時間、レビュー、写真の自動同期	GBP API 承認	□ 審査中
フェーズ 3	LocalPosts 書込（ブランド告知 / イベント）、Google に ClaimReview を能動発行	フェーズ 2 が 3 ヶ月安定稼働	□ 計画
フェーズ 4	Organization Account：チェーンブランドを 1 入口で複数店舗管理、一括編集、統計集約	Google が Organization Account for Partner を有効化	□ 将来

フェーズ 1 の完了は GBP API 承認を**待たない**——顧客が Maps URL を貼れば、プラットフォームが Place ID を抽出し DB に保存、Schema.org は Google Maps への `sameAs` を生成できる。これによりプラットフォームは Google 審査時程に塞がれず先にサービス開始できる。

要点

- GBP は物理ビジネスの事実主情報源、百原プラットフォームは単方向同期（GBP → Schema → AXP → AI）
- GBP API 申請は 7〜10 営業日、申請理由は「ビジネスオーナー管理ツール」に絞る
- 代行管理は二軌：OAuth 主、Manager 追加を fallback
- 12 組フィールド対応を明確定義、全マッピングは純粋関数でテスト可能に
- 同期頻度はフィールド変動率でレイヤ分け、300 QPM で約 2,000 Location 並行対応可
- GBP に webhook なし、Notifications API + Pub/Sub で補完、レイテンシ 5〜10 分
- 4 フェーズロードマップ：フェーズ 1 稼働中、フェーズ 2 承認待ち、フェーズ 3–4 はニーズ次第

参考文献

- 第 6 章—AXP シャドウドキュメント
- 第 7 章—Schema.org フェーズ 1
- 第 9 章—クローズドループ・ハルシネーション修正（ClaimReview 発行パス）

第 9 章—クローズドループ型ハルシネーション検知と自動修復

> 検知だけでは不十分である。「修復 → 検証 → 収束」のループがなければ、ハルシネーションは雑草のように再生する。

目次

- 9.1 なぜ検知だけでは不十分なのか
- 9.2 AI ハルシネーションの 5 分類
- 9.3 主機構：NLI 分類 + ChainPoll
- 9.4 中央共有 RAG：SaaS インフラストラクチャ
- 9.5 L1 LLM Wiki：アクティブ意味層
- 9.6 修復：ClaimReview 生成と多経路注入
- 9.7 2 層再スキャンループ
- 9.8 収束タイミングと受理基準
- 本章のまとめ
- 参考資料

9.1 なぜ検知だけでは不十分なのか

従来のブランドモニタリングツールは **「問題発見 → 顧客通知 → 顧客が自力で解決」**のループで動作する。SEO 時代にはこれで通用した。大半の問題が 「低露出」系であり、顧客はコンテンツ作成やリンク配置で改善できたからである。

AI ハルシネーションはこれとは異なる：

- 顧客は自社に関する AI の誤認識を**どう訂正すればよいか分からない**
- 顧客が 「正しい」コンテンツを書いても、AI が必ずしも再クロール・再学習するとは限らない
- 各 AI プラットフォームはデータパイプラインが異なる。一つを直しても他は直らない

結論：**問題を顧客に突き返すのは責任放棄である。**プラットフォーム側が検知から収束まで一貫して自動化しなければならない。

図 9-1：6 段階クローズドループ

```
flowchart LR
    A[1. 検知 <br/> 主張を抽出] --> B[2. 比較 <br/>Ground Truth と照合]
    B --> C[3. 判定 <br/> ハルシネーションか?]
    C -->|yes| D[4. 生成 <br/>ClaimReview]
    D --> E[5. 注入 <br/>AXP + RAG]
    E --> F[6. 再スキャン検証 <br/> センチネル + フル]
    F -->| 収束 | Done[クローズ]
    F -.->| 残存 | D
    C -->|no| Skip[アクション不要]
```

図 9-1：検知からクローズまでの 6 段階。各段階は局所的耐障害性を持ち、どの一段階が失敗しても他段階は破綻しない。

9.2 AI ハルシネーションの 5 分類

当プラットフォームはブランド関連の AI 誤認識を 5 カテゴリに分類し、それぞれに検知・修復戦略を持たせている。

図 9-2：ハルシネーション分類体系

```
flowchart TD
    Hallu[AI ハルシネーション]
    Hallu --> E1[1. エンティティ誤認]
    Hallu --> E2[2. 事業事実誤り]
    Hallu --> E3[3. 業種誤分類]
    Hallu --> E4[4. 時間/地理誤り]
    Hallu --> E5[5. 属性誤り]
    E1 -.->| 例 | EX1[" 日系企業を米系企業と誤認"]
    E2 -.->| 例 | EX2[" 製品ラインが競合のものと混同"]
    E3 -.->| 例 | EX3[" 歯科クリニックを美容外科に分類"]
    E4 -.->| 例 | EX4["2025 年開業を『10 年の老舗』と記述"]
    E5 -.->| 例 | EX5["B2B SaaS を一般消費者向けアプリと記述"]
```

図 9-2：5 分類は排他的ではなく、一回の AI 応答に複数種が含まれうる。修復の優先順位は影響度で付与する。

分類別の修復戦略

分類	優先度	主な修復手段
エンティティ誤認	P0	Schema.org `sameAs` 強化 + ClaimReview + Wikidata リンク
事業事実誤り	P0	AXP で正しい製品ラインを明記 + ClaimReview
業種誤分類	P1	`industry_code` 修正 + Schema.org `@type` + RAG 同期
時間/地理誤り	P1	`foundingDate` / `address` の明示 + ClaimReview
属性誤り	P2	記述強化、FAQ 追加、`audience` 修正

P0 はブランドそのものの認識を歪めるため最優先で修正する。P2 は意味的なニュアンスの差であり、一定量を蓄積してから一括対応してよい。

9.3 主機構：NLI 分類 + ChainPoll

単純な「主張抽出 → Ground Truth 照合」方式には 2 つの根本的な問題がある：

1. **GT カバレッジが低い** —全ブランド事実を人手でキーバリュー化することは不可能であり、未記載項目は偽陽性になる
2. **完全一致は厳しすぎる** —同一事実に複数の正しい表現がある（「2018 年設立」 vs 「2018 年創業」 vs 「創業 7 年」）。文字列比較は正しい表現を誤検知する

当プラットフォームの主検知機構は **NLI（自然言語推論）三値分類** である。GT 照合は、NLI に入力される 3 階層知識源の 1 つに過ぎない。

3 階層知識源（NLI 入力に統合）

```
flowchart TB
    Resp[AI 応答テキスト] --> Ext[原子的主張抽出 <br/>via LLM]
    Ext --> NLI[NLI 分類器]
    subgraph KS[" 知識源（権威性で 3 階層）"]
      K1["<b>L1 ブランド公式サイト </b><br/> ライブスクレイピング / DB 24h キャッシュ <br/> 一次情報、最高
      K2["<b>L2 RAG 知識ベース </b><br/> 顧客アップロード + 公式文書 <br/> 意味検索で回答"]
      K3["<b>L3 Ground Truth</b><br/> 手動キーバリュー <br/> 精密だがカバレッジ低"]
```

```
    end
    K1 --> NLI
    K2 --> NLI
    K3 --> NLI
    NLI --> Cls[分類 <br/>entailment / contradiction /<br/>neutral / opinion]
```

図 9-3：知識源は権威性で積層される。3 つすべてを単一コンテキストに統合して NLI に投入する。統合コンテキストが 500 文字未満の場合、当該スキャンでは検知をスキップする（証拠不足のため判定不能）。

NLI 四値分類

分類	意味	アクション
`entailment`	情報源が主張を**支持**する	事実正確、通過
`contradiction`	情報源が主張と**矛盾**する	**ハルシネーション認定**、修復フロー入り
`neutral`	情報源が主張を**肯定も否定もしない**	判定しない（重要：neutral はハルシネーションではない）
`opinion`	主観的価値判断（「最も優れた」「お勧め」）	スキップ、意見は事実ではない

NLI モデルは各主張に対し `confidence`（0.0〜1.0）と `severity`（critical / major / minor / info）も出力する。

「neutral ≠ ハルシネーション」が中核原理である理由

最もよくある設計上の誤りは「情報源が言及していないなら偽と見なす」である。しかしブランド公式サイトが全事実を網羅することは不可能で、AI が「従業員 50 名」と述べ、公式サイトに従業員数ページが無かったとしても、**それは数字が誤りであることを意味せず、検証できないだけ**である。`neutral` を `contradiction` として扱えば偽陽性のハルシネーションが氾濫し、不要な修復がトリガーされ、クローズドループ全体を毒する。最悪の場合、AI は次のクロール時に「訂正された」（捏造された）内容を読み、誤った事実を学習する。

ChainPoll：曖昧帯に対するセカンドオピニオン投票

`confidence` `[0.5, 0.8]`（曖昧帯）の主張には **ChainPoll 多数決** を発動する：

```
// 同一主張・同一プロンプトで LLM を 3 回呼び、多数決を取る。
async function chainpollVerify(claim, knowledgeContext) {
  const votes = { contradiction: 0, entailment: 0, neutral: 0 };
  const prompt = buildNLIPrompt(claim, knowledgeContext);

  const results = await Promise.allSettled([
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
    aiCall('hallucination_detect', prompt, { maxTokens: 20 }),
  ]);

  for (const r of results) {
    if (r.status !== 'fulfilled') continue;
    const text = (r.value.text || '').toLowerCase();
    if (text.includes('contradiction')) votes.contradiction++;
    else if (text.includes('entailment')) votes.entailment++;
    else votes.neutral++;
```

```
  }

  return pickMajority(votes); // 2-of-3 以上でのみ採用
}
```

ChainPoll は単一 LLM 分類のノイズ（分類器自体がランダム性を持つ LLM）を抑える。高信頼度（> 0.8）・低信頼度（< 0.5）の主張では ChainPoll は発動せず、曖昧帯のみに限る。これによりコストは有界である。

重大度の階層

主張が `contradiction` と分類されたあと、その `severity` が修復優先度を決める：

Severity	基準	修復スケジュール
`critical`	社名、製品カテゴリ、国/所在地が完全に誤り	即時修復、24h 以内に注入
`major`	中核機能、価格が誤り	24h 以内に修復
`minor`	二次的機能、仕様のずれ	次回スキャンでまとめて処理
`info`	表現が不正確	蓄積後にバッチ処理

この階層化により、**実際にビジネス損害を生む**ハルシネーションに資源を集中でき、軽微な言い回しの差異に溺れない。

9.4 中央共有 RAG：SaaS インフラストラクチャ

§9.3 の **L2 RAG 知識ベース** は**テナント毎にデプロイされない**。百元 SaaS 基盤全体が**単一の中央 RAG エンジンを共有する**（以降「Central RAG」）。これは静かだが極めて重要なアーキテクチャ決定である。

なぜテナント個別 RAG ではなく中央共有なのか

次元	テナント個別 RAG	中央共有（当方採用）
運用コスト	各テナントが独立デプロイ・スケール	単一クラスタで一括運用
推論コスト	各テナントが独自に embedding / LLM 実行	GPU / API 共有で分散
知識グラフ連携	テナント間相互検証不可	中央が匿名化クロスリファレンス可
アップグレード速度	テナント毎に個別	一度のアップグレードで全体同期
隔離リスク	自然に隔離される	テナント隔離を設計する必要がある（X-Tenant-ID + データ ACL）

当方は中央共有を採用した—エンジニアリング作業を支払って運用効率を買う選択である。テナント隔離は 3 層機構を積層している：

1. **HTTP ヘッダ** `X-Tenant-ID` —全クエリがテナント識別子を持ち、RAG エンジンがアクセス可能文書をフィルタ
2. **API キー署名** —全 SaaS バックエンド呼出が `X-RAG-API-Key` を保持、RAG 側が発呼元を検証
3. **RLS / 文書 ACL** —取込文書全てに所有テナントラベルを付与、クエリ時に強制フィルタ

この 3 層は §2.5 の RLS + アプリ層二重保険と類似する。単層のすり抜けが起きても、残る 2 層が同時に失敗しない限りテナント横断リークは発生しない。

ブランドレベル分離（2026-04-21 更新）：テナント内に複数ブランドが存在する場合、各ブランドはさらに専用の Knowledge Base ID（`rag_kb_id`）によって知識範囲を分離される。AXP 生成時はブランドの `kbId` を指定してクエリを実行し、同一テナント内の異なるブランド（例：コスメブランドとフードブランド）の知識が混在しないことを保証する。新ブランドが AXP を有効化すると、`seedBrandRAGKB` が自動的に KB を作成してブランドプロフィールと公式サイト URL をアップロードする——詳細は §6.10 を参照。

図 9-4：中央共有 RAG アーキテクチャ

```
flowchart LR
    subgraph Platform[" 百元 GEO SaaS（アプリケーション層）"]
      T1[テナント A バックエンド]
      T2[テナント B バックエンド]
      T3[テナント N バックエンド]
    end
    subgraph RAG[" 中央 RAG エンジン"]
      direction TB
      Gate["API Gateway<br/>X-Tenant-ID + X-RAG-API-Key 検証"]
      subgraph Core[" エンジンコア"]
        W["<b>L1 LLM Wiki</b><br/> 意味処理層"]
        V["<b>L2 RAG</b><br/> ベクトル検索"]
        W --> V
      end
      Gate --> Core
    end
    T1 -->|POST /api/v1/ask<br/>X-Tenant-ID: A| Gate
    T2 -->|POST /api/v1/ask<br/>X-Tenant-ID: B| Gate
    T3 -->|POST /api/v1/ask<br/>X-Tenant-ID: N| Gate
    V --> Ans[テナントスコープ回答]
```

図 9-4：全テナントが同一 RAG を共有するが、全クエリは Gateway で Tenant ID によりフィルタされる。

9.5 L1 LLM Wiki：アクティブ意味層

本アーキテクチャにおける Wiki は従来の 「文書ストア」ではない。**LLM によって能動的に維持される意味知識層**である。これこそハルシネーション検知を実用レベルに到達させるインフラであり、独立した節を割く価値がある。

9.5.1 なぜ生文書に直接ベクトル検索をかけないのか

直感的設計：顧客がサイト・FAQ・製品ページをアップロードし、埋め込み・保存、クエリ時に検索。この純受動的 RAG には 3 つの致命的欠陥がある：

- **文書間矛盾が解消されない** —製品ページは「顧客 100 万社」、FAQ は「50 万ブランドに提供」と異なる記述、ベクトル検索は両方を返し、下流の NLI が 2 つの矛盾する 「情報源」を見ることになる
- **時間次元が失われる** —同一内容の新旧バージョンがベクトル DB に共存し、「どちらが現在の事実か」を決定する仕組みが無い
- **集約が無い** —ベクトル検索は 「最も類似するパッセージ」のみを返すが、特定トピックに関するブランドの完全事実は十数個のパッセージに散在する可能性があり、検索はそれらを統合しない

これらは一般的 RAG アプリ（カスタマーサポートチャットボット等）では許容可能だが、**ファクトチェック**文脈では致命的である。

9.5.2 LLM Wiki の処理フロー

LLM Wiki は百元が自社開発した意味処理層であり、取込文書毎に以下を実行する:

図 9-5: LLM Wiki における文書ライフサイクル

```
flowchart TB
    In[新規文書取込 <br/> 顧客 UP / サイトスクレイプ / API push] --> Norm[正規化 <br/> 書式除去、平文抽出]
    Norm --> Segment[意味チャンキング <br/> トピック毎に分割]
    Segment --> Ext[エンティティ抽出 <br/> 主語 / 述語 / 目的語トリプル]
    Ext --> Compare{既存 Wiki と比較}
    Compare -->| 新規事実 | Add[Wiki に追加]
    Compare -->| 矛盾 | Conflict[衝突マーク <br/> 最新保持 + 旧版記録]
    Compare -->| 補完 | Merge[より完全な記述に統合]
    Add --> Reindex[増分コンパイル <br/> ベクトルインデックス +KG ノード更新]
    Conflict --> Reindex
    Merge --> Reindex
    Reindex --> Ready[L2 RAG 検索利用可能]
```

図 9-5: Wiki は文書を格納するだけでなく、「現在真と見なすべき事実集合」を能動的に維持する。新規文書毎に既存知識の再評価がトリガーされる。

9.5.3 増分コンパイル

従来の RAG システムは文書追加・修正時に 「ベクトルインデックス全体再構築」を要することが多く、頻繁な実行には高コストすぎる。LLM Wiki は **増分コンパイル** を採用する:

- **変更検知の粒度**: 事実トリプルレベル（例: `< 百元，設立年，2024>`）であり、文書レベルではない
- **影響範囲のみ再計算**: 新規事実が入るとき、関連トピックのベクトルと知識グラフノードのみ更新
- **バージョン管理**: 各事実が `valid_from` / `valid_to` タイムスタンプを持ち、タイムトラベルクエリ可能
- **ロールバック機能**: 取込文書が誤りを導入した場合、特定事実を旧版へロールバック可能

ハルシネーション検知にとってこれが意味するのは: ClaimReview 修復を Wiki に注入した**数分以内**に、NLI クエリが訂正事実を見始めるということである。夜間リビルドやダウンタイムを待つ必要が無い。

9.5.4 LLM Wiki の中核機能

本層がファクトチェックを支援するためのネイティブ機能:

- **文脈理解** —LLM が文書全体を読み（単一パッセージではなく)、跨パッセージ・跨文書の統合質問に回答
- **引用トレース** —回答にソースパッセージ ID が付与され、下流検証が可能
- **複数表現統合** —同一事実の異なる表現（「2024 年設立」 vs 「2024 年創業」）を同一ノードとして扱う
- **自動要約** —長文を自動要約し、下流の検索負荷を低減
- **衝突管理** —新旧情報源が食い違う場合、最新を保持し履歴をタグ付け（混在させない)

百元はこれらの機能を薄い REST API で公開し、ビジネス層を内部実装から切り離す。任意の LLM やベクトルエンジンの交換は本層内で完結し、ビジネスコードに影響しない。

9.5.5 L1 Wiki が L2 RAG に与える補助的役割

L2 RAG のベクトル検索は 「標準的な RAG」に見えるが、**インデックス内容**は L1 Wiki が処理済みの事実であり、生文書ではない。その帰結:

- 検索は 「Wiki で未処理の生パッセージ」を返さない

- 結果は特定 Wiki ノードまで遡れる ファクト ID を持つ
- NLI 分類器から見た情報源の品質は、生文書を読む場合より遥かに高い

要するに：**L1 Wiki は知識オントロジー、L2 RAG は知識検索。**L1 無しでは L2 は単なる高速文書検索ツールに過ぎない。

9.6 修復：ClaimReview 生成と多経路注入

ClaimReview Schema.org 例

```
{
  "@context": "https://schema.org",
  "@type": "ClaimReview",
  "datePublished": "2026-04-18",
  "url": "https://baiyuan.io/claims/founding-year",
  "claimReviewed": "Baiyuan Technology was founded in 2018",
  "itemReviewed": {
    "@type": "Claim",
    "appearance": "AI-generated response",
    "firstAppearance": "2026-04-16"
  },
  "reviewRating": {
    "@type": "Rating",
    "ratingValue": "1",
    "bestRating": "5",
    "alternateName": "False"
  },
  "author": {
    "@type": "Organization",
    "name": "Baiyuan Technology",
    "url": "https://baiyuan.io"
  }
}
```

ClaimReview は Schema.org 標準プロパティ—Google、Facebook、Twitter すべてがパースする。百元独自の発明ではなく、グローバルなファクトチェック生態系の相互運用可能な構成要素である。[5][6]

3 つの注入経路

```
flowchart LR
    CR[ClaimReview] --> P1[経路 A<br/>AXP シャドウドキュメント]
    CR --> P2[経路 B<br/>RAG 知識ベース]
    CR --> P3[経路 C<br/>GBP LocalPosts *Phase 3]
    P1 --> AI1[AI ボットがクロール]
    P2 --> AI2[RAG が検索]
    P3 --> AI3[Google AI Overview が参照]
    style P3 stroke-dasharray: 5 5
```

図 9-6：同一 ClaimReview が 3 チャネルへ同時投入され、AI 生態系が正しい事実を再学習する確率を最大化する。経路 C は GBP Phase 3 に条件付き。

[5] Schema.org. ClaimReview. https://schema.org/ClaimReview

[6] Google Search Central. Fact check article markup. https://developers.google.com/search/docs/appearance/structured-data/factcheck

---

9.7 2 層再スキャンループ

注入後、AI が実際に認識を更新したかをどう検証するか。再スキャンするしかない—しかし**頻度は低すぎず**（顧客が信頼を失う）**高すぎず**（コスト爆発）でなければならない。プラットフォームはスキャンを **2 層に分割**する。

図 9-7：2 層スキャンの役割分担

```
flowchart TB
    subgraph L1["Tier 1：センチネルスキャン（4h サイクル）"]
      T1[対象：検索型 AI<br/>Perplexity / ChatGPT Search /<br/>Google AI Overview]
      T2[軽量：指標クエリ 5 本をサンプリング]
      T3[目的：修復の採用を <br/> 迅速に確認]
    end
    subgraph L2["Tier 2：フルスキャン（24h サイクル）"]
      F1[対象：知識型 AI<br/>Claude / Gemini / DeepSeek /<br/>Kimi / GPT-4o / etc.]
      F2[深掘：インテントクエリ 20〜60 本 <br/>+ 7 次元スコア + 指紋比較]
      F3[目的：コーパス統合確認、<br/> 収束判定]
    end
```

図 9-7：Tier 1 は短サイクル、ライブクロールする AI を対象。Tier 2 は長サイクル、長い再訓練窓を持つ AI を対象。2 層合わせて完全な受理を構成する。

なぜ検索型と知識型で分けるのか

- **検索型 AI**（Perplexity、ChatGPT Search、AI Overview）はクエリ時にライブクロールする。AXP に注入された修復は数分以内に拾われ、**4 時間センチネル**で検証に十分である。
- **知識型 AI**（標準的な Claude / Gemini / DeepSeek）は事前学習コーパスや定期再訓練 RAG に依存する。修復が統合されるには**数日**を要し、**24 時間フルスキャン**が妥当である。

両方に均一サイクルを掛けると、検索型は検証遅延が過大（4 時間で分かるのに 24 時間待つのは無駄）、知識型は偽陰性（4 時間ではまだクロールされておらず、修復失敗と誤判定）となる。分割は最適化ではなく、**根本的に異なる 2 プラットフォームへの帰結**である。

Tier 1 と Tier 2 はデータストアを共有する

両層は同一 `scan_results` テーブルに書き込み、`scan_layer` カラムで区別する。下流分析（ダッシュボードトレンド、競合比較）は**統合か分離か**を必要に応じて選ぶ：

- **統合**シナリオ：ユーザが「本日のシテーション率」を見る—両層とも同一時間窓の信号を表す
- **分離**シナリオ：「修復収束速度」を分析—Tier 1 と Tier 2 は異なる時間スケールで収束、別個に計算

9.8 収束タイミングと受理基準

図 9-8：修復後の典型的収束曲線（イメージ）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "修復後のハルシネーション残存率（イメージ）"
    x-axis ["Day 0", "Day 1", "Day 3", "Day 7", "Day 14", "Day 21", "Day 30"]
    y-axis "Hallucination Rate (%)" 0 --> 100
    bar [80, 75, 60, 40, 22, 10, 3]
```

図 9-8：検索型 AI は通常 1〜3 日で収束、知識型 AI は 2〜4 週間。本図は集計イメージであり、個別ケースのタイムラインは大きく変動する。

収束判定規則

ハルシネーションが 「解決済み」と宣言されるのは、以下すべてを満たすときのみである：

1. **連続 N 回スキャン**（N=3〜5、厳格度で調整）で全対象プラットフォームから当該主張が不在
2. **同義表現**（類義語、翻訳、略語）も不在
3. **Tier 1 と Tier 2 の両方が検証済み** —片層パスでは不可

すべて満たされたとき、UI は当該ハルシネーションを 「□ 解決」とマークし、収束所要時間を記録する。

非収束時のハンドリング

期待収束時間を超えて残存するハルシネーションは、**スタボーン・ハルシネーション（頑固型）**にエスカレートされる：

- どのプラットフォームで失敗したかを分析（特定コーパス源に依存していないか?）
- ClaimReview に技術的誤りが無いか確認（死 URL、JSON-LD 構文エラー）
- 追加修復チャネルを検討—Wikidata 編集、Wikipedia 事実タグ、LinkedIn 会社概要更新

スタボーン・ハルシネーションの処理は通常 **人手介入** を要する。これは現行自動ループの境界である。この境界を正直に示すことは、完全自動化を装うより誠実である。

本章のまとめ

- クローズドループは 検知・比較・判定・生成・注入・検証 の 6 段階を自動化する
- AI ハルシネーションは 5 種類（エンティティ / 事実 / 業種 / 時間 / 属性）に分類され、優先度が付与される
- **主機構は NLI + ChainPoll** であり、単純な GT 照合ではない。GT は 3 階層知識源の 1 つ
- 「neutral ≠ ハルシネーション」は中核設計原理。ここでの偽陽性は修復サイクル全体を毒する
- 中央共有 RAG SaaS アーキテクチャ：全テナントが単一エンジンを共有、3 層テナント隔離
- **L1 LLM Wiki** はアクティブ意味層—衝突管理、増分コンパイル、バージョニング、要約
- L1 Wiki は知識オントロジー、L2 RAG は知識検索。L1 無しでは L2 は単なる高速文書検索
- ClaimReview は AXP + RAG + GBP LocalPosts の 3 経路に注入、Wiki 増分コンパイルで数分以内に反映
- 2 層スキャン（Tier 1 センチネル 4h + Tier 2 フル 24h）は検索型と知識型 AI の異なるサイクルを尊重する
- スタボーン・ハルシネーションは依然人手介入が必要—現行自動化の境界

参考資料

- 第 6 章—AXP シャドウドキュメント
- 第 7 章—Schema.org Phase 1
- 第 10 章—フェーズ・ベースライン試験

第 10 章—フェーズ・ベースライン試験：固定質問セットによる AI 認識の縦断比較

> 毎回異なる質問をすれば、スコアが動いたのがブランドのせいか質問のせいか永遠に区別できない。

目次

- 10.1 通常スキャンの縦断盲点
- 10.2 フェーズ・ベースライン試験の設計
- 10.3 通常スキャンからのデータパス隔離
- 10.4 4 つの観察軸
- 10.5 運用ガイダンス
- 本章のまとめ
- 参考資料

10.1 通常スキャンの縦断盲点

第 2 章の通常スキャンは、実ユーザの多様な質問をシミュレートするため**各実行時にインテントクエリを動的生成**する。これは「現在ブランドはどれくらいの頻度で言及されるか」という**横断的**質問には適するが、**縦断的**質問には答えられない：「先週と今週で、AI のこのブランドに対する認識はどう変化したか」。

二回のスキャンは異なるクエリセットを使用したため、スコア差分には少なくとも 3 つの原因候補がある：

1. **実際の変化** —AI のブランド認識が変化した（良くなった or 悪くなった）
2. **クエリシフト** —新しいクエリセットが旧セットよりたまたま多く / 少なく言及を誘発した
3. **ランダム性** —同一 AI に同一質問でも実行毎に応答は微妙に変動する

この 3 つを分離しない限り、トレンドラインは***ノイズ***である。実際の変化を抽出するには、**固定クエリセットによる反復試験**が必要である。

図 10-1：通常スキャン vs ベースライン試験

```
flowchart TB
    subgraph Regular[" 通常スキャン（横断）"]
      R1[実行毎に新規クエリ]
      R2['現在のスコアはいくらか'に回答]
      R3[時系列比較に不向き]
    end
    subgraph Baseline[" フェーズ・ベースライン（縦断）"]
      B1[固定クエリセット]
      B2[週次 / 隔週で反復]
      B3['認識はどう変化しているか'に回答]
    end
    Regular --- Baseline
```

図 10-1：2 つのスキャンタイプは異なる種類の質問に答える。補完的であり、代替不可能である。

10.2 フェーズ・ベースライン試験の設計

フェーズ定義とサイクル

- **Phase 1**（Day 0）：ベースライン確立
  - システムがブランドの業種に基づき**代表的インテントクエリ 20 本を生成**
  - 全質問と完全な AI 応答（`response_text`）を `baseline_test_runs.queries_json` および `baseline_test_responses` に格納
  - この 20 本のクエリは**恒久保存**され、あらゆるクリーンアップから隔離される
- **Phase 2**（Day 7 ±1）：初回再試験
  - Phase 1 の 20 本を再利用、全プラットフォームに再投入
  - 全応答を比較データとして格納
  - 「同一質問・異なる時点」のスコア差分を計算
- **Phase 3**（Day 14 ±1）：2 回目再試験
  - 同じ 20 本、3 回目の投入
  - データ点 3 つで短期トレンドラインを得る
- **追加フェーズ**（オプション）：顧客要望により Phase 4, 5（月次）へ延長

図 10-2：3 フェーズデータ構造

```
flowchart LR
    Q[" インテントクエリ × 20（固定）"] --> P1[Phase 1<br/>Day 0]
    Q --> P2[Phase 2<br/>Day 7]
    Q --> P3[Phase 3<br/>Day 14]
    P1 -->| 比較 | P2
    P2 -->| 比較 | P3
    P1 -->| 比較 | P3
    P1 --> R[response_text × platforms]
    P2 --> R
    P3 --> R
```

図 10-2：3 回の投入、1 クエリセット、3 完全応答セットの比較。

なぜ 20 本なのか

- **統計的観点**：20 本で主要インテント種別（best-of、比較、推薦、状況別）を網羅可能、それ以上は収穫逓減
- **コスト観点**：20 × 15 プラットフォーム × 3 フェーズ = ブランド毎 900 API コール、それ以上はクォータを圧迫
- **可視化観点**：20 本超のトレンドラインを 3 本並列で描くと読みにくい

20 本は経験則である。データが別の数値を支持するなら将来的に改訂可能である。

10.3 通常スキャンからのデータパス隔離

フェーズ・ベースライン試験は通常スキャンとオーバーラップしない**完全独立データパス**で動作する：

観点	通常スキャン	フェーズ・ベースライン
クエリ源	実行毎に動的生成	Phase 1 で固定
トリガ頻度	毎日 / 4h	スケジュール or 手動
メイン GEO スコアに算入?	Yes	**No**（別表示）

観点	通常スキャン	フェーズ・ベースライン
Stale Carry-Forward の対象?	Yes	**No**（失敗時は `incomplete` マーク）
データ保持	ローリング窓	`response_text` **永久保持**
Redis キャッシュ使用?	Yes（重複 API コスト削減）	**No**（毎回新鮮に問い合わせ）

なぜベースラインはキャッシュをバイパスするのか

通常スキャンは同一質問の最近応答をキャッシュし（AI の意見は数分で変わらない前提）、コストを削減する。しかしベースラインの目的は AI 意見の変化を測ることであり、キャッシュは測定そのものを破壊する。

なぜベースラインはメインスコアから除外されるのか

ベースライン結果をメインスコアに算入すると、Phase 2 と Phase 3 の再試験が**三重計上効果**（近接時間窓で同一ブランドが複数回計上される）を生み、ダッシュボードトレンドを汚染する。分離することでスコアの純度が保たれる。

10.4 4 つの観察軸

フェーズ・ベースラインデータの価値は 「スコア変化」だけではない—4 つの独立観察軸を支える。

図 10-3：4 軸変化観察マトリクス

```
flowchart TB
    subgraph Quant["1. 定量的"]
      Q1[シテーション率 上下]
      Q2[ポジション 前後]
      Q3[プラットフォームカバレッジ 広狭]
    end
    subgraph Qual["2. 定性的"]
      L1[記述表現の変化]
      L2[言語の深さ]
      L3[要点の追加・削除]
    end
    subgraph Comp["3. 競合的"]
      C1[共起する競合リスト]
      C2[競合の位置関係]
      C3[新規競合の出現]
    end
    subgraph Sent["4. センチメント"]
      S1[感情分布のシフト]
      S2[強い言葉の増減]
      S3[中立 → 肯定 / 否定 シフト]
    end
```

図 10-3：4 軸は直交する。定量は計算可能、定性は定性分析が必要、競合はグラフ差分、センチメントはスコアリングを要する。

軸別の分析アプローチ

定量的—差分を直接計算：スコア差、パーセント変化、トレンド傾き。

定性的—Phase 1 と Phase 2 の `response_text` を diff、「追加段落」「削除段落」「置換フレーズ」をハイライト。差分をビジュアルに顧客へ提示する。

競合的—各応答から全ブランドエンティティを抽出し、フェーズ間の集合を比較（新規 / 消失 / 継続）。時間方向の集合差分として扱う。

センチメント—文単位で感情分類を実行し、分布を比較。例：Phase 1 = 中立 80% / 肯定 15% / 否定 5%、Phase 2 = 中立 60% / 肯定 30% / 否定 10% → 明らかな感情分極化。

10.5 運用ガイダンス

フェーズ・ベースラインの発動タイミング

推奨トリガは 3 つ：

1. **顧客が上位プランに加入** —自動的に Phase 1 を作成、顧客はダッシュボードで縦断進化を閲覧
2. **顧客が主要コンテンツ改訂をリリース** —直後に手動で Phase 1 発動、改訂の効果を測定
3. **クローズドループ修復後**（第 9 章参照）—フェーズ試験を開始し、ハルシネーションの実収束を検証

ベースラインの無効化と再構築

以下のいずれかが発生したら、既存フェーズを延長せず**再構築**する：

- ブランドに**重大な事業変更**（M&A、スピンオフ、ピボット）が発生—旧クエリが通用しない
- AI プラットフォームが**メジャーバージョン**をリリース（GPT-5、Claude 4）—バージョン跨ぎのスコア直接比較は不可
- ベースライン運用が **6 ヶ月超**再構築されていない—質問ドリフトが蓄積

再構築時は新規 `baseline_cohort_id` を作成。旧コホートは履歴参照用に保持するが、新データ点を追加してはならない。

UI 提示の原則

- **通常ダッシュボードと混在させない** —ユーザが誤って Phase スコアを日次スコアに加算するのを防ぐ
- **専用** `/baseline` **ページ** で Phase 1→2→3 比較ビューを表示
- **主ビュー**：同一質問に対する 3 フェーズの応答を左中右カラムで並列表示、差分を色でハイライト
- **副ビュー**：4 観察軸にまたがる集計トレンド

本章のまとめ

- 通常スキャンは動的クエリ（横断「現在」に強い）、ベースラインは固定クエリ（縦断「進化」に強い）
- Phase 1→2→3 は 20 本の固定質問を 3 回再問い合わせ、`response_text` を恒久保持
- ベースラインは独立データパスを使用—メインスコアから除外、キャッシュバイパス、Stale Carry-Forward 対象外
- 4 観察軸：定量（スコア）、定性（テキスト）、競合（エンティティ集合）、センチメント（分布）
- 発動トリガ：上位プラン加入 / 主要コンテンツ改訂 / クローズドループ後の検証
- 再構築トリガ：事業変更 / AI メジャーバージョン / 6 ヶ月超

参考資料

- 第 2 章—システム全体像
- 第 3 章—7 次元スコアリングアルゴリズム
- 第 4 章—Stale Carry-Forward
- 第 9 章—クローズドループ型ハルシネーション修復

第 11 章—5 ブランド実地観察：6 週間の匿名化データ

理論だけでは不十分である。データが検証する。以下は 5 社のライブパイロットブランドを約 6 週間運用した集計観察であり、顧客名および識別可能な数値は匿名化している。

目次

- 11.1 ブランドプロフィール（匿名化）
- 11.2 GEO スコア分布
- 11.3 プラットフォームカバレッジの非対称性
- 11.4 Schema.org 完成度とシテーション率
- 11.5 AXP 展開前後
- 11.6 顧客側の落とし穴
- 11.7 3 つの予想外の発見
- 11.8 商業化初月の検証
- 本章のまとめ
- 参考資料

11.1 ブランドプロフィール（匿名化）

5 社のパイロットブランドは B2B、B2C、実店舗、純オンラインに跨がる：

コード	業種	タイプ	市場言語	開始時 GEO スコア
Brand A	B2B SaaS（マーケティングテック）	オンライン	中英バイリンガル	中位
Brand B	専門金融サービス	オンライン	主に英語	高位
Brand C	B2B SaaS（ナレッジマネジメント）	オンライン	中英バイリンガル	中位
Brand D	飲食チェーン、実店舗	実店舗	中国語	低位
Brand E	百元科技自身（ドッグフーディング）	オンライン	中英バイリンガル	低位（コールドスタート）

開始時 GEO スコアは低/中/高位で表示し、相対構造を保持しつつ絶対値は伏せている。

なぜこの 5 社を観察サンプルとしたか

- **Brand A / C** は「比較ペア」を構成：両者とも B2B SaaS、バイリンガル、類似の開始条件、下流最適化の対比
- **Brand B** は高位開始点—AI が既に熟知するブランドにも改善余地はあるか？
- **Brand D** はサンプル中唯一の実店舗—GBP + Schema.org LocalBusiness 組合せをテスト
- **Brand E** はコールドスタートの極端例：完全新規、外部参照ゼロ

5 サンプルでは**統計的主張はできない**。本章は結論ではなく観察を提示する—運用の実際の形を伝えることが目的である。

11.2 GEO スコア分布

6 週間を通じて全ブランドが全 7 次元で動きを見せた。

図 11-1：7 次元レーダー（Week 1 vs Week 6、匿名化集計）

```
%%{init: {'theme':'base'}}%%
graph TD
    W1["Week 1 集計平均 <br/>Citation: 中低 <br/>Position: 中 <br/>Coverage: 低 <br/>Breadth: 低 <br/>Ser
    W6["Week 6 集計平均 <br/>Citation: 中 <br/>Position: 中高 <br/>Coverage: 中 <br/>Breadth: 中 <br/>Ser
    W1 -.->|6 週間運用 | W6
```

図 11-1：全次元が改善。Coverage と Breadth の改善が最大。データは「低/中/高」階層で示し、具体値は省略。

観察された 3 パターン

1. **Position が先、Citation が後** —Schema.org 最適化後、AI がブランドを言及する位置が先に移動（最終段落 → 上位 1/3）。数週間遅れて言及数そのものが上昇する
2. **Coverage は Breadth より速く拡張** —インテントクエリ種別（比較、推薦）のカバー追加は、追加 AI プラットフォーム対応拡張より容易である
3. **Consistency が最後に収束** —ブランドが ChatGPT と DeepSeek で同じ姿に見えるようになるには通常 4〜8 週間を要する

11.3 プラットフォームカバレッジの非対称性

同一ブランドのシテーション率は AI プラットフォーム間で劇的に異なる。当方 5 ブランドの集計から見た相対強度：

図 11-2：プラットフォームカバレッジの非対称性（匿名化）

```
flowchart LR
    subgraph HighLang[" 英語強みブランド (Brand A/B/C/E) "]
      H1[ChatGPT  ]
      H2[Claude  ]
      H3[Perplexity  ]
      H4[DeepSeek  ]
      H5[Kimi  ]
    end
    subgraph LocalLang[" 中国語ローカルブランド (Brand D) "]
      L1[ChatGPT  ]
      L2[Perplexity  ]
      L3[AI Overview  ]
      L4[DeepSeek  ]
      L5[Kimi  ]
    end
```

図 11-2：□□ = 実質的に引用、□ = 言及あり、□ = ほぼゼロ。英語圏 B2B ブランドは US 発 AI で強い、中国語ローカルブランドは中国語モデルと Google AI Overview で強い。

示唆

- **自分の戦場でないプラットフォームへの過剰投資は避けよ** —Brand A は DeepSeek でシテーション率がゼロのまま、ChatGPT では堅調に引用された。DeepSeek の深掘りは無駄だった。英語市場の ChatGPT プレゼンスを深める方が優れる。
- **ローカル言語ブランドは中国語モデルを軽視すべきでない** —Brand D は当初 ChatGPT を主戦場と想定したが、台湾繁体中国語ユーザクエリでは実際には Kimi と DeepSeek の方がパフォーマンスが高かった。これが真の戦場である。
- **AI Overview の実店舗にとっての重要性は過小評価されがち** —Brand D のリーチの多くは Google AI Overview 経由（スタンドアロン AI アプリではなく）であり、GBP 品質と密結合であった。

日本市場への含意：Brand D のケースと同様、日本ローカルブランドも ChatGPT / Claude 偏重は危険である。ChatGPT 日本語 UI、Perplexity、Google AI Overview（日本語)、そして国内で比較的利用の多い Gemini を主戦場と見なすべきである。

11.4 Schema.org 完成度とシテーション率

図 11-3：完成度 × シテーション率差分（集計）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title " 完成度（%）vs 6 週間後のシテーション率差分（イメージ）"
    x-axis ["< 40%", "40-60%", "60-80%", "80-100%"]
    y-axis " シテーション率差分（相対）" 0 --> 100
    bar [10, 25, 55, 80]
```

図 11-3：完成度帯別の 6 週間シテーション率改善（相対値)。80% 超のブランドが最大のシテーション向上を示した。

観察

- **完成度はシテーション率改善と正相関** —ただし**線形ではない**：60% 未満では進行は遅く、60% 超で加速する
- これは**「認識可能性しきい値」**の存在を示唆する—AI がエンティティを能動的に引用し始めるには、一定臨界量の構造化事実が必要と思われる
- **80% 超は限界収益が逓減** —85% から 100% への押し上げは比例したシテーション向上をもたらさない

運用上の示唆

- **全ブランドを 60% 超えさせることを優先** —数社を 100% へ押し上げるより効率的
- 「100% 追求」の顧客不安は大半が無駄—80% 完成度と**コンテンツ品質**の組合せは、100% の機械的充填に勝る
- 完成度は目標ではなく—**AI に認識される手段**である。しきい値を越えたら他のレバーへ移るべきである

11.5 AXP 展開前後

5 社のうち 3 社（A / B / C）は Week 2-3 でのみ AXP 展開、残り 2 社（D / E）は Week 1 から稼働。このタイムラグが AXP の独立効果を観察させる。

図 11-4：AXP 前後の AI ボットトラフィック（匿名化集計）

```
%%{init: {'theme':'base'}}%%
xychart-beta
    title "AI ボット日次アクセス数（AXP 前後、相対インデックス）"
    x-axis ["Wk 1", "Wk 2", "Wk 3（AXP 展開)", "Wk 4", "Wk 5", "Wk 6"]
    y-axis "AI ボットアクセス（指数）" 0 --> 200
    bar [25, 30, 35, 95, 140, 165]
```

図 11-4：AXP 展開週に AI ボットトラフィックが急上昇する。指数 100 = 展開前平均。

観察

- **AXP 展開後 2 週間以内に AI ボットトラフィックが 3〜5 倍に増加**（大半は GPTBot、ClaudeBot、PerplexityBot）
- **シテーション率改善はトラフィックに約 2〜3 週間遅れる** —AI ボットがコンテンツを取込んだあと、コーパス統合サイクルが追加で必要
- **GSC インデックス化も同期して増加** —SEO 側のボーナス。AXP のクリーン HTML は従来検索にも親和的である

しかし—ボットトラフィック上昇はシテーション率上昇を保証しない。AXP コンテンツ自体が意味的に薄ければ、AI が扱う材料が無い。AXP は**「見てもらうためのインフラ」**であり、銀の弾丸ではない。

11.6 顧客側の落とし穴

顧客側運用から共通する 5 つの落とし穴が浮上した。

図 11-5：落とし穴分布（5 ブランド × 6 週間の集計）

```
pie title "顧客側落とし穴（5 ブランド、6 週間）"
    "industry_code 未入力" : 28
    "logo_url 欠落または破損" : 22
    "GBP 未接続（実店舗）" : 18
    "description 短すぎ（<20 文字）" : 17
    "sameAs 外部リンク未入力" : 15
```

図 11-5：業種分類の欠落が最頻出の落とし穴、Schema.org `@type` 選定に影響する。

共通根本原因

- **顧客はどのフィールドが AI にとって重要か知らない** —UI の優先度シグナルが無ければ、顧客は画面順に入力し高重み項目を取りこぼす
- **破損 logo_url は想定より頻発** —顧客は内部パス、一時的 CDN、移設済み画像ホストを貼付することが多く、AXP 生成が 404 を返す
- **実店舗業態は GBP 手動接続に慣れていない** —Brand D（唯一の実店舗パイロット）は GBP リンクに 3 週間を要し、うち半分が Place ID の誤解で失われた

UI フィードバックループ

これらの観察を受け、プロダクト変更を実施した：

- 完成度バナーは欠損項目を等価表示ではなく**「高重み欠損項目を強調」**する
- `logo_url` フィールドに**ライブ検証**を追加（ペースト時に HEAD リクエストで 200 を確認）
- 実店舗 GBP 接続に**スクリーンショット付きステップバイステップガイド**を組込み

11.7 3 つの予想外の発見

予期しなかったが記録に値する 3 つの観察。

1. バイリンガルブランドは言語跨ぎ一貫性に意外に脆弱

Brand A / C / E はいずれもバイリンガル（中 / 英）である。AI が同一ブランドを中国語クエリと英語クエリで記述する内容は**しばしば実質的に異なる**。例:

- 英語クエリ: “Acme is a B2B marketing-automation platform”
- 中国語クエリ:「Acme 是一家行銷顧問服務公司」(「Acme はマーケティングコンサル会社」)

これはハルシネーションではない—両記述とも部分的には真である。これは **AI が言語跨ぎで同一エンティティを集約する能力が未成熟**であるためである。ブランドオーナーへの示唆: 中国語と英語の Schema.org レコードは `sameAs` **で明示的に相互リンク**し、記述は独立執筆ではなく意味等価であるべきである。

日本市場への含意: ja / en（および必要なら zh-TW）のトリリンガル展開時も同一原理が適用される。日本語ブランドページと英語ブランドページを `sameAs` で明示結合しなければ、AI は別エンティティとして扱いかねない。

2. 「非引用」は「否定的引用」より難題

当方は否定的な AI 発言が最大の脅威と予想していた。運用の現実は逆であった:**「AI がブランドを全く言及しない」**方がより深刻な問題である。否定的言及は少なくとも AI がエンティティの存在を認識していることを証明し、ClaimReview で訂正可能である。完全不在は**ブランドが候補プールに入っていない**ことを意味—手掛かりが無い。

これは当方がシテーション率の重みを 25% に設定した（第 3 章参照）理由の説明である—重要な指標だが、総合を支配させると他次元が周縁化する。

3. 競合との共起は正のシグナルになりうる

通説:「同一 AI 応答に競合が出現するとビジビリティが希薄化する」。当方は逆を観察した。**適切な競合と並列に挙げられることはブランドのカテゴリアイデンティティを強化する**。Brand A は早期、著名な大手競合 2 社と共起した。シテーション率は中位だったが、エンドユーザの頭の中で「同じ階層」とブラケットされたことが、異常に強い下流コンバージョンへ転化した。

確証にはさらなるデータが必要である。しかし GEO における「敵と味方」の概念は、従来 SEO の「競合」概念と逆向きに作用する可能性を示唆する。AI 時代においては、**適切なブランドと並べられることが、単独で名指されることより重要であるかもしれない。**

11.8 商業化初月の検証

上記 5 ブランドは社内ドッグフーディングとパートナーパイロットの組合せであった。並行して、**百元 GEO は有料 SaaS として商業化初月で 3 社の商業顧客を成約**した。業界はパイロットと異なる:

業種	形態	主要ドライバ
美容クリニックチェーン	多拠点実店舗、競合密集カテゴリ	GBP 統合 + 実店舗 LocalBusiness Schema.org + 医療グレード・ハルシネーション検知

業種	形態	主要ドライバ
新興飲食チェーン	多拠点実店舗、急拡張期	拠点レベル AXP + Phase ベースラインで拡張期変化を捕捉
プレミアム・アロマテラピー / ヨガ	高 ACV、口コミ駆動	Content Depth + Sentiment を主次元（物語品質 > シテーション頻度）

観察

- **3 社は 3 つの異なる理由でプラットフォームを選択** —7 次元スコアリング仮説を検証：業界毎に優先次元が異なり、単一指標で全市場を満たすことはできない
- **美容チェーンと飲食チェーンは「多拠点」要件を共有** —Phase 4 Organization Account への事業側プッシュ（§8.7参照）が当初計画より早まった
- **プレミアム・アロマ / ヨガは Content Depth と Sentiment を副次から主要へ昇格** —高 ACV 業界は意思決定連鎖が長く、物語品質が頻度に勝る

これら顧客は執筆時点で新規契約であり、詳細運用データは将来改訂で公開する。ここでのポイント：**百元 GEO のエンジニアリング設計は社内だけでなく、有料の外部需要からも検証された。**

日本市場への含意：日系「美容クリニックチェーン」「飲食チェーン」「サロン業態」は上記台湾 3 業種と構造的に類似し、GBP × 多拠点 × 医療グレード検知という組合せはそのまま通用する想定である。note.com / Qiita での日本語発信は、この受容ペルソナを念頭に置く。

本章のまとめ

- 5 ブランド × 6 週間 = 観察であり結論ではない、サンプルサイズが統計推論を妨げる
- 全 7 次元が改善、Position は Citation に先行、Consistency が最後に収束
- プラットフォームカバレッジは言語・業種のバイアス—全プラットフォームではなく「自分の真の戦場」にフォーカス
- Schema.org 完成度はシテーション率と非線形正相関、60% が認識しきい値、80% 超は収益逓減
- AXP 展開はボットトラフィック 3〜5 倍、シテーション率改善は 2〜3 週遅れ、トラフィック ≠ シテーション率
- 顧客側 5 落とし穴（業種コード / ロゴ / GBP / 記述 / sameAs）、UI は「高重み欠損」強調でフィードバック
- 3 つの予想外の発見：バイリンガル一貫性の脆弱性、「非引用」が「否定的引用」より深刻、適切な競合共起が優位に転化
- 商業化初月：3 社成約（美容クリニックチェーン、新興飲食チェーン、プレミアム・アロマ / ヨガ）、それぞれ異なる次元がドライバ

参考資料

- 第 3 章—7 次元スコアリングアルゴリズム
- 第 6 章—AXP シャドウドキュメント
- 第 7 章—Schema.org Phase 1
- 第 9 章—クローズドループ型ハルシネーション修復

第 12 章—限界、未解決問題、今後の課題

できないことを明示的に列挙するツールは、万能を標榜するツールよりも信頼に値する。

目次

- 12.1 プラットフォームができないこと
- 12.2 AI モデルバージョン変動の不予測性
- 12.3 未解決研究課題
- 12.4 ロードマップ
- 12.5 実務者と研究者への招待
- 本章のまとめ
- 参考資料

12.1 プラットフォームができないこと

図 12-1：現行カバレッジマトリクス

能力	カバレッジ	ギャップ
モニタリング	完備	サポート済み 15 AI プラットフォームのみ、カスタムデプロイや私有 LLM には到達不可
スコアリング	完備	業種横断比較は意味を持たない、クエリ空間に主観性が残る
構造化データ	完備	多言語 Schema.org は zh-TW + en + ja のみ、韓国語・東南アジア言語は未対応
ハルシネーション検知	部分	知識源の品質に依存、源が疎な場合カバレッジ低下
ハルシネーション修復	部分	スタボーン（頑固型）は依然人手介入が必要
自動クローズドループ	部分	検索型は高速収束、知識型は低速、中間状態の完全フィードバックは困難
外部プラットフォーム検証	制限	LinkedIn、Crunchbase、G2、Capterra は公開 API 無し、手動のみ

能力	カバレッジ	ギャップ
GBP 統合	制限	Phase 2 API 承認待ち、現状は URL から Place ID 抽出のみ

図 12-1:「完備」= 機能網羅的、「部分」= 中核は存在するが既知ギャップあり、「制限」= 外部制約によりブロック。

具体的限界

- **全外部プラットフォームを 100% 検証不可** —LinkedIn、Crunchbase 等、公開 API 無しのプラットフォームは存在有無の手動チェックのみ可能で、コンテンツ正確性のプログラム比較は不可
- **GEO スコアは絶対値ではない** —業種毎にクエリ空間が異なるため、業種横断の数値比較は無意味である。妥当な比較は同業種・同時間窓・同クエリプール内のみ
- **中国語 AI のカバレッジは英語に遅れる** —非著名ブランドに対する中国語 LLM の認識深度は依然英語モデルに顕著に及ばない。L1 LLM Wiki の中国語事実構造化を強化することで緩和するが、完全には埋められない
- **Webhook 機構無し** —大半の AI プラットフォームは 「ブランドが言及された」イベントを発行しない、ポーリングによるレイテンシが発生

日本市場特有の限界：日本語 AI（特に日本語特化 LLM、例：Tsuzumi 等）のサポート優先度は現行で中位であり、海外 AI の日本語応答経由が主軸となる。これは今後のロードマップで拡張予定。

12.2 AI モデルバージョン変動の不予測性

これは**エンジニアリング側だけでは完全に解決できない**問題である。OpenAI が GPT-5 をリリース、Anthropic が Claude 4 をリリース、DeepSeek が新旗艦を投入したとき、全ブランドのスコアが同時に 3〜10 ポイント変動しうる。

バージョン変動の 3 分類

種類	例	方向
メジャーモデル更新	GPT-4o → GPT-5	多数のブランドが上昇（新しい学習データ）
安全性 / アラインメント強化	ーベンダが拒否率増加	多数のブランドが低下（拒否は引用を覆い隠す）
検索拡張 on/off	Claude がウェブ検索を追加 or 除去	ウェブ存在感によりブランド毎に方向が異なる

緩和策

百元はこれらの変動を防ぐことはできないが、3 つの機構で顧客への衝撃を緩和する：

1. **バージョン感度バナー** —追跡対象 AI プラットフォームでメジャーバージョン変更が検出されたら、UI に 「データは新モデルに適応中、短期ボラティリティが予期される」と表示
2. **フェーズ・ベースラインのバージョン跨ぎタグ付け** —モデルバージョン跨ぎのベースラインデータは**生値での直接比較が不可**、UI で区別
3. **重み保存型の履歴比較** —内部で 「特定バージョン下のスコア」をトレンド分析用に保持、バージョンジャンプをブランド変化と誤帰属しない

12.3 未解決研究課題

1. 真の否定的フィードバック vs ハルシネーション誤認識

AI が 「このブランドはカスタマーサービスが悪い」と述べたとき、それは:

- **ハルシネーション** —AI が無関係な情報源から当該主張を連想した
- **実レビュー** —実際のユーザネガティブレビューが学習コーパスに流入した

のどちらかである。対処は根本的に異なる: ハルシネーションは訂正すべき、実フィードバックは隠蔽ではなくサービス改善に繋げるべきである。百元の現行自動化は**両者を確実に区別できず**、情報源判断のため人手介入を要する。これはクローズドループの真の穴である。

2. 因果 vs 相関

顧客がコンテンツを改訂、3 週間後にシテーション率上昇。これは:

- **因果** —コンテンツ改訂が直接的に AI 認識を向上させた
- **相関** —別イベント（ニュース、有料配置、季節性）が改善を駆動した

厳密な因果証明には A/B テスト基盤（同一ブランドの半分を改訂、半分未改訂）が必要—商業的には実施不可能。これは **GEO 分野共通の研究ギャップ**である。

3. ロングテールクエリのカバレッジ戦略

動的インテントクエリ生成は 20〜60 本で主要インテント種別を網羅する。しかし**ロングテールクエリ**(極めて特定的、稀な利用者質問）は列挙不可である。顧客が 「うちのユーザが XX と聞いたのに AI が当社を言及しなかった」と述べたとき、それは:

- 捕捉し得ないロングテールを偶然の抽出が見逃した
- 系統的なカバレッジギャップ

の区別が現状では困難である。現行はケースバイケース処理。将来の 「顧客提供インテントクエリ」機能は助けになりうるが、「顧客は自社に都合の良い質問しか尋ねない」バイアスを導入する。

12.4 ロードマップ

図 12-2: 今後の課題依存グラフ

```
flowchart LR
    subgraph Short[" 短期 (6 ヶ月以内) "]
      A1[GBP API Phase 2-3<br/> 読み書き]
      A2[多言語 Schema.org<br/>ja / ko 拡張]
      A3[可視化アップグレード <br/>Phase ベースライン表示]
    end
    subgraph Mid[" 中期 (6〜12 ヶ月) "]
      B1[AI プラットフォーム拡張 <br/>Mistral / Cohere 深化 <br/>+ Claude Projects]
      B2[言語跨ぎ sameAs<br/> 自動化]
      B3[競合共起アドバイザ]
    end
    subgraph Long[" 長期 (12 ヶ月超) "]
      C1[因果推論研究 <br/>A/B 方法論]
      C2[私有 LLM エンティティモニタ]
```

```
    C3[マルチテナント・カスタムインテントクエリ]
  end
  A1 --> B1
  A2 --> B2
  A3 --> C3
```

図 12-2：3 フェーズロードマップ。各フェーズは前段階に依存する。具体的タイミングは外部要因（Google、特定 AI ベンダ）に依存する。

短期フォーカス

- **GBP API Phase 2-3** —最大の保留項目。Phase 2（読取）は承認から 1〜2 ヶ月以内を見込む。Phase 3（LocalPosts 書込）は Phase 2 が 3 ヶ月安定した後。
- **多言語 Schema.org** —日本・韓国市場顧客需要に対応。言語モデルは既存 API を再利用、i18n コンテンツと `@type` マッピングの拡張のみ必要。
- **Phase ベースライン可視化** —最も頻繁に要望される UX 項目。

長期目標

- **因果推論研究** —学術機関と連携し、GEO の A/B 方法論を公表、分野の共有基盤とする。
- **私有 LLM モニタリング** —企業顧客の内部 AI アプリ（サポートボット、内部知識アシスタント）に対するエンティティ認識監査—現行 SaaS に隣接するプロダクトライン。

12.5 実務者と研究者への招待

本書は GEO を議論・共進可能な分野として成立させる試みであり、単一ベンダの閉じた経験にすることを避ける。そのために：

- **引用・改変・翻訳を歓迎する**（CC BY-NC 4.0 ライセンス下で、任意の章について）
- **誤記・質問・追加を歓迎する** —GitHub Issues （テンプレート提供済）
- **共同研究提案を歓迎する** —特に因果推論、言語跨ぎエンティティアラインメント、スタボーン・ハルシネーション領域
- **アーキテクチャパターンの再利用を歓迎する** —Stale Carry-Forward、マルチプロバイダ耐障害、中央共有 RAG、クローズドループ修復は汎用エンジニアリング設計である

GEO は非常に黎明期である。本書はこの分野における最初期に公開された技術文書の一つとなることを目指す—後続のチームが各穴を独立再発見するのではなく、我々が既に這い出した穴から出発できるように。

日本市場の実務者へ：2025-2026 年は日本で AI 導入が加速する時期であり、日本語 GEO 分野における先発優位の窓が開いている。本書が note.com の CMO/CDO 読者層、Qiita のエンジニア読者層の双方に、共通の技術語彙を提供できれば幸いである。

本章のまとめ

- 百元 GEO には明示的な限界がある：外部プラットフォーム検証の制限、中国語/日本語モデルカバレッジの遅れ、Webhook 機構無し
- AI モデルバージョン変動は不予測、バナー・ベースライン分離・重み保存比較で緩和
- 3 つの未解決問題：真否定 vs ハルシネーション、因果 vs 相関、ロングテールクエリカバレッジ
- ロードマップは 短期（GBP API 完成）/ 中期（言語跨ぎ sameAs）/ 長期（因果推論研究）に分割
- 実務者への招待：書籍引用、GitHub での議論、研究提案、アーキテクチャ再利用

参考資料

- 第 3 章—7 次元スコアリング（スコア限界の議論）
- 第 8 章—GBP API フェーズ・ロードマップ
- 第 9 章—クローズドループ修復（自動化の境界）
- 第 11 章—実地観察と予想外の発見
- リポジトリ README —貢献と引用

付録 A —用語集（完全版）

アルファベット順。各エントリは **定義 + 初出章 + 関連章** を提供する。

A

- **@id (Schema.org)** —JSON-LD 一意識別子文字列、`@graph` 内またはドキュメント間でエンティティ参照に使用。初出：§6.7。中核利用：§7.3。
- **AI ボット** —LLM 学習または検索用にウェブコンテンツを取込むクローラの総称（GPTBot、ClaudeBot、PerplexityBot 等）。初出：§6.3。
- **AI シテーション率** —代表的インテントクエリのうち AI が能動的にブランドを言及する割合（0〜100%）。本書の中核指標。初出：§1.2。詳述：§3.2.1。
- **aggregateRating** —Schema.org プロパティで集計レビュースコアを保持。当プラットフォームでは GBP から同期。§8.4。
- **AXP (AI-ready eXchange Page)** —百元造語。AI ボット向けに提供される「シャドウドキュメント」—純 HTML + Schema.org JSON-LD + Markdown、人向けサイトから疎結合。詳解：第 6 章。
- **アプリ層フィルタ** —アプリケーション層クエリレベルのフィルタリング、マルチテナント設計では RLS と並ぶ第二防衛線。§2.5。

B

- **BullMQ** —Redis バックエンドの Node.js ジョブキューライブラリ、バックグラウンドタスク基盤の中核。§2.4。
- **ブランドエンティティ** —Schema.org Organization / LocalBusiness ノードを根とする構造化ブランドデータの総称。第 7 章。

C

- **ChainPoll** —同一主張を同一プロンプトで LLM に 3 回送信し多数決を取る検証手法。NLI 信頼度 0.5〜0.8 の曖昧帯で使用。§9.3。
- **CID (Customer ID)** —Google Maps 内部数値識別子、Places API 呼出無しでは Place ID に変換不可。§7.7。
- **ClaimReview** —主張が真か否かを注釈する Schema.org プロパティ。プラットフォームがハルシネーション訂正を公的宣言するために使用。§9.6。
- **クローズドループ** —検知 → 比較 → 判定 → ClaimReview 生成 → AXP/RAG 注入 → 再スキャン の循環。第 9 章。
- **Cloudflare Workers** —AI ボット UA 検知および AXP 注入に用いるエッジ実行環境。§6.3。
- **Common Crawl** —公開大規模ウェブスクレイプ、大多数のメジャー LLM の主要事前学習源。§1.3。
- **Consistency（一貫性）** —7 次元の一つ、他 6 次元のプラットフォーム跨ぎ標準偏差の逆数。§3.2.7。
- **Content Depth（コンテンツ深度）** —7 次元の一つ、ブランド言及に伴う記述の深さ。§3.2.6。
- **Core Web Vitals** —Google のページ体験指標（LCP、FID、CLS）。SEO では重要だが GEO ではほぼ無関係。§1.3。

D

- **DIRECT_MODEL_ID_MAP** —アグリゲータモデル ID とベンダネイティブモデル ID の変換表。§5.3.1。
- **ドッグフーディング** —自社プロダクトの社内利用。事例の Brand E が該当。§11.1。

E

- **entailment / contradiction / neutral / opinion** —NLI 4 値分類出力。`contradiction` のみハルシネーション認定、`neutral` は明示的にハルシネーションではない。§9.3。

- **extraParams** —ベンダ固有の必須 API パラメータ、例：Qwen3 は `enable_thinking: false` を要求。§5.3.2。

F

- **フィンガープリント** —AXP または Schema.org アーティファクトのハッシュ、AI が更新内容を取込んだかの判定に使用。第 9 章関連。
- **FTID (Feature ID)** —Google Maps 内部ロケーション識別子。§7.7。

G

- **GBP (Google Business Profile)** —Google のビジネス掲載情報、実店舗の信頼源。第 8 章。
- **GEO (Generative Engine Optimization)** —生成 AI 応答内でのブランドの言及頻度・位置・記述正確性を向上させる学問と実践。第 1 章。
- **GPTBot** —OpenAI のウェブクローラ、ChatGPT 学習コーパス収集用。§6.4。
- **Ground Truth (GT)** —事実の地盤真実。本書では候補主張比較用の信頼参照セット（ライブスクレイプされた公式サイト / RAG / 手動 GT の 3 階層）。§9.3。

H

- **ハルシネーション** —現実と一致しない AI 生成コンテンツ。本書では 5 分類：エンティティ誤認、事業事実誤り、業種誤分類、時間/地理誤り、属性誤り。§9.2。

I

- **industry_code** —プラットフォームが使用する 25 業種分類 enum。§7.2。
- **インテントクエリ** —ブランド業種に基づき動的生成される代表的ユーザ質問。§2.2.1。
- **is_physical** —ブランドエンティティに付与する実店舗 vs オンライン事業フラグ、Schema.org フィールド重みを駆動。§7.4。
- **isStale** —Stale Carry-Forward 下でプラットフォームスコアが履歴持越値であることを示すマーカー。§4.3。

J

- **JSON-LD** —JavaScript Object Notation for Linked Data、Schema.org の主流シリアライゼーション形式。§6.2, 第 7 章。

K

- **kgmid (Knowledge Graph Machine ID)** —Google ナレッジグラフ・エンティティ識別子。`sameAs` で使用。
- **ナレッジグラフ** —エンティティと関係性の構造化ネットワーク。Schema.org の `@id` 相互リンクで構築。§7.3。

L

- **Layer 1 センチネルスキャン** —4 時間サイクルの軽量検証、検索型 AI を対象。§9.7。
- **Layer 2 フルスキャン** —24 時間サイクルの深掘スキャン、7 次元スコア + フィンガープリント比較。§9.7。
- **LLM Wiki** —百元造語。LLM によって能動的に維持される意味知識層、RAG アーキテクチャ内 L1 に位置。§9.5。
- **LocalBusiness** —実店舗に汎用される Schema.org `@type`。§7.2。

M

- **modelRouter** —自社開発のマルチ AI プロバイダルーティングサービス、プライマリ/フォールバック 2 経路抽象を提供。第 5 章。

N

- **NLI（自然言語推論、Natural Language Inference）**—2 パッセージ間の含意関係を分類するタスク、当方のハルシネーション検知主機構。§9.3。

O

- **OpenAI 互換 API** —複数 AI ベンダを OpenAI SDK インタフェースでラップするアグリゲータエンドポイント。§5.2。

P

- **pgvector** —PostgreSQL のベクトル埋込・類似検索拡張。§2.4。
- **フェーズ・ベースライン試験** —固定 20 問セットを異なる時点で再試問する縦断試験。第 10 章。
- **Place ID** —Google Maps 一意ロケーション識別子、Places API の主キー。§7.7。
- **Position Quality（位置品質）**—7 次元の一つ、AI 応答内でブランドが出現する位置の加重平均。§3.2.2。
- **Platform Breadth（プラットフォーム広度）**—7 次元の一つ、少なくとも 1 回言及したプラットフォーム数 ÷ 監視対象総プラットフォーム数。§3.2.4。

Q

- **Query Coverage（クエリ網羅）**—7 次元の一つ、ブランドが言及されるインテントクエリ種別の多様性。§3.2.3。
- **QPM (Queries Per Minute)** —API クォータ単位、GBP API デフォルトは 300 QPM。§8.5。

R

- **RAG（検索拡張生成、Retrieval-Augmented Generation）**—検索で拡張する生成。当プラットフォームは中央共有 RAG SaaS アーキテクチャを採用。§9.4。
- **RLS (Row-Level Security)** —PostgreSQL の行レベルアクセス制御、マルチテナント隔離の第一防衛線。§2.5。

S

- **sameAs** —他プラットフォーム（Wikipedia、Wikidata、LinkedIn、GBP）上の同一エンティティを指す Schema.org プロパティ。§7.3。
- **Schema.org** —Google、Bing、Yahoo、Yandex が共同推進する構造化データ語彙。第 7 章。
- **Sentiment Score（センチメントスコア）**—7 次元の一つ、言及の集計トーン（肯定 / 中立 / 否定）。§3.2.5。
- **センチネルスキャン** —Layer 1 参照。
- **SERP (Search Engine Results Page)** —古典的 SEO の中核戦場。§1.1。
- **サイトマップ** —クローラのコンテンツ発見を助ける XML URL リスト。§6.6。
- **スモークテスト** —サービス起動時にフォールバック経路上で実行される最小限検証、実到達可能性を保証。§5.6.1。
- **Stale Carry-Forward** —百元造語。プラットフォームのスキャンが 100% 失敗したとき、履歴から直近成功値を持ち越し stale とマーク、信号連続性を維持する。第 4 章。

T

- **Tenant ID (X-Tenant-ID)** —マルチテナント HTTP ヘッダ、中央共有 RAG のテナントフィルタに使用。§9.4。

U

- **UA (User-Agent)** —HTTP リクエストヘッダ、AXP 注入が AI ボット vs 人間を判定する基礎。§6.4。

W

- **Webhook** —サーバ発イベント通知。GBP API は**提供しない**。§8.6。
- **Wikidata** —公開ナレッジグラフ、Schema.org `sameAs` の重要ターゲット。§7.3。
- **ウィザード** —新規ブランド向けの線形 7 ステップ Schema.org オンボーディングフロー。§7.6。

標準参照表

標準	役割	章
Schema.org	構造化データ語彙	第 6, 7, 9 章
JSON-LD 1.1	Linked Data シリアライゼーション	第 6, 7 章
OpenGraph	リンクプレビュー用メタタグ	第 6 章
robots.txt	Robots Exclusion Protocol	第 6 章
sitemaps.xml	Sitemap Protocol 0.9	§6.6

付録 B —公開 API 仕様（抜粋）

本付録は百元 GEO Platform の**公開向け** REST エンドポイントを抜粋したものであり、第三者統合・研究・監査参照を目的とする。内部管理者エンドポイント（admin、monitoring、billing）は含まない。

共通規約

ベース URL

`https://api.geo.baiyuan.io/v1`

認証

全エンドポイントは `Authorization: Bearer <token>` ヘッダを要求する。トークンは顧客パネルの「API キー管理」ページから取得する。

レスポンスエンベロープ

全レスポンスは以下の形式を取る：

```
{
  "status": "success" | "error",
  "data":   { ... },           // status=success のとき
  "error":  "description"      // status=error のとき
}
```

レートリミット

- デフォルト 60 req/min、エンタープライズプランで引き上げ可能
- 超過 → HTTP 429 と `Retry-After` ヘッダ（待機秒数）

---

1. Brands

GET `/brands`

現アカウントに見えるブランドを列挙。

クエリパラメータ：

- `limit`（1〜100、デフォルト 20）
- `offset`（デフォルト 0）
- `industry_code`（オプション、25 分類 enum は §7.2 参照）

レスポンス：

```
{
  "status": "success",
  "data": {
    "brands": [
      {
        "id": "brd_01H...",
        "name": "Example Brand",
        "industry_code": "saas_application",
        "url": "https://example.com",
        "completion": 78,
```

```
        "current_geo_score": 62
      }
    ],
    "total": 42,
    "limit": 20,
    "offset": 0
  }
}
```

GET `/brands/:id`

単一ブランドの詳細を取得。

POST `/brands`

新規ブランド作成。**Body**: `name`（必須）、`url`、`industry_code`、`description` 等。

PUT `/brands/:id`

ブランド更新。Body 意味論は `Content-Type` に依存、merge-patch 対応。

---

2. Scans

POST `/scans`

スキャンを即時トリガ（スケジュール無し）。

Body:

```
{
  "brand_id": "brd_01H...",
  "platforms": ["chatgpt", "claude", "perplexity"],
  "intent_count": 20,
  "scan_layer": "full"
}
```

レスポンス（202 Accepted）:

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
    "estimated_duration_seconds": 180,
    "queue_position": 3
  }
}
```

GET `/scans/:id`

スキャンステータス問い合わせ。

レスポンス:

```
{
  "status": "success",
```

```
  "data": {
    "scan_id": "scn_01H...",
    "state": "queued" | "running" | "completed" | "failed",
    "progress": 0.75,
    "started_at": "2026-04-18T09:15:00Z",
    "completed_at": null,
    "results_url": "/scans/scn_01H.../results"
  }
}
```

GET `/scans/:id/results`

スキャン結果取得。

レスポンス（抜粋）:

```
{
  "status": "success",
  "data": {
    "scan_id": "scn_01H...",
    "scan_layer": "full",
    "geo_score": 62,
    "dimensions": {
      "citation":    { "value": 55, "isStale": false },
      "position":    { "value": 70, "isStale": false },
      "coverage":    { "value": 60, "isStale": false },
      "breadth":     { "value": 58, "isStale": false },
      "sentiment":   { "value": 72, "isStale": false },
      "depth":       { "value": 50, "isStale": false },
      "consistency": { "value": 65, "isStale": false }
    },
    "platforms": {
      "chatgpt":    { "sov": 60, "position": "top-third" },
      "claude":     { "sov": 58, "position": "top-third" },
      "perplexity": { "sov": 48, "position": "middle" }
    }
  }
}
```

---

3. Entity（Schema.org ブランドエンティティ）

GET `/brands/:id/entity`

ブランドの完全な Schema.org データ取得（Organization、Services、Employees、Hours、Location、Social Links）。

GET `/brands/:id/entity/schema.json`

生成済み Schema.org JSON-LD の取得（顧客サイトに直接埋込可能）。

レスポンス: `Content-Type: application/ld+json`、body は `{"@context": "https://schema.org", "@graph": [...]}`。

GET `/brands/:id/entity/completion`

完成度スコア取得。

レスポンス:

```
{
  "status": "success",
  "data": {
    "total_percent": 78,
    "breakdown": {
      "basic_info":  { "percent": 100, "weight_pct": 25 },
      "location":    { "percent":  80, "weight_pct": 15 },
      "services":    { "percent":  60, "weight_pct": 15 },
      "employees":   { "percent":  40, "weight_pct": 10 }
    },
    "missing_high_priority": ["description", "logo_url"]
  }
}
```

---

4. AXP（シャドウドキュメント）

GET `/axp?host=<domain>&path=<path>`

本エンドポイントは **Cloudflare Worker から呼び出される**、統合者が直接呼ぶ必要は無い。指定ブランド・パスのシャドウドキュメント HTML を返す。

---

5. Hallucinations

GET `/brands/:id/hallucinations`

検出済ハルシネーション列挙。

クエリパラメータ: `severity`（critical/major/minor/info）、`status`（open/resolving/resolved）。

POST `/brands/:id/hallucinations/:hid/repair`

修復フロー発動。

---

6. Baseline（フェーズ・ベースライン試験）

POST `/brands/:id/baseline`

新規フェーズ・ベースラインコホート作成。

POST `/brands/:id/baseline/:cohortId/run`

既存ベースラインの再実行。

GET `/brands/:id/baseline/:cohortId/compare`

Phase 1 vs Phase 2 / 3 の比較取得。

---

7. Webhooks（顧客受信可能）

百元プラットフォームは顧客設定済 Webhook URL（パネルで設定）へ以下イベントを**プッシュ**する:

イベント	トリガ
`scan.completed`	スキャン完了
`hallucination.detected`	新規ハルシネーション検出（severity >= major）
`hallucination.resolved`	ハルシネーション収束
`completion.threshold_crossed`	完成度が 60% / 80% / 100% のしきい値を跨ぐ
`gbp.sync_failure`	GBP 同期が連続 3 回失敗

ペイロード形式:

```
{
  "event": "scan.completed",
  "brand_id": "brd_01H...",
  "scan_id": "scn_01H...",
  "occurred_at": "2026-04-18T09:30:00Z",
  "data": { ... }
}
```

全 Webhook リクエストは `X-Baiyuan-Signature` ヘッダ（HMAC-SHA256）を持つ、顧客は処理前に検証すべきである。

エラーコード

HTTP	Code	説明
400	`invalid_parameter`	パラメータ形式または値が不正
401	`unauthenticated`	トークン無し、または無効
403	`forbidden`	トークン有効だがリソースアクセス権無し
404	`not_found`	リソース不存在
409	`conflict`	状態衝突（例：スキャン進行中、再トリガ不可）
422	`unprocessable_entity`	データ検証失敗
429	`rate_limited`	クォータ超過
500	`internal_error`	内部エラー—再試行、またはサポート連絡

付録 C —参考文献

本書で引用した全外部情報源。4 カテゴリに整理する。

1. 公式仕様と標準

Schema.org と Linked Data

- Schema.org. Schema.org Vocabulary Specification. https://schema.org/docs/schemas.html
- W3C. JSON-LD 1.1: A JSON-based Serialization for Linked Data. https://www.w3.org/TR/json-ld11/
- Schema.org. ClaimReview. https://schema.org/ClaimReview
- Schema.org. LocalBusiness. https://schema.org/LocalBusiness
- Schema.org. Organization. https://schema.org/Organization
- Schema.org. Service. https://schema.org/Service
- Schema.org. Person. https://schema.org/Person

Google エコシステム

- Google Business Profile API. Overview. https://developers.google.com/my-business
- Google Business Profile API. Notifications Overview. https://developers.google.com/my-business/content/notification-setup
- Google Maps Platform. Places API documentation. https://developers.google.com/maps/documentation/places/web-service
- Google Search Central. How Search works. https://www.google.com/search/howsearchworks/
- Google Search Central. Fact check article markup. https://developers.google.com/search/docs/appearance/structured-data/factcheck
- Google Search Central. Google-Extended and generative AI training. https://developers.google.com/search/docs/crawling-indexing/overview-google-crawlers

AI ボット User Agent

- OpenAI. GPTBot User Agent Documentation. https://platform.openai.com/docs/gptbot
- OpenAI. OAI-SearchBot User Agent. https://platform.openai.com/docs/bots
- Anthropic. ClaudeBot documentation. https://support.anthropic.com/en/articles/8896518
- Perplexity. PerplexityBot. https://docs.perplexity.ai/guides/bots
- Common Crawl. CCBot User Agent. https://commoncrawl.org/ccbot

ウェブプロトコル

- IETF. RFC 9309 —Robots Exclusion Protocol. https://www.rfc-editor.org/rfc/rfc9309.html
- sitemaps.org. Sitemap Protocol 0.9. https://www.sitemaps.org/protocol.html
- W3C. HTTP/1.1. https://www.rfc-editor.org/rfc/rfc7230

オープンライセンス

- Creative Commons. CC BY-NC 4.0 Legal Code. https://creativecommons.org/licenses/by-nc/4.0/legalcode
- Citation File Format. CFF 1.2.0 Specification. https://github.com/citation-file-format/citation-file-format

---

2. 技術フレームワークと開発ツール

- Cloudflare. Workers Runtime Documentation. https://developers.cloudflare.com/workers/
- Next.js. App Router & Server Components. https://nextjs.org/docs
- PostgreSQL. Row Security Policies. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- pgvector. Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector
- BullMQ. Queue documentation. https://docs.bullmq.io/
- OpenAI. Node.js SDK. https://github.com/openai/openai-node
- Anthropic. TypeScript SDK. https://github.com/anthropics/anthropic-sdk-typescript

---

3. 業界調査と統計

- OpenAI. (2025). Usage & Revenue Update, Q3 2025. 公式四半期開示。(第 1 章で引用)
- Perplexity AI. (2025). Year in Review 2024: Search Volume & Engagement. 公式ブログ。(第 1 章で引用)
- SimilarWeb. (2025). The State of Generative Search: AI Overview Impact on Publisher Traffic. 調査レポート。(第 1 章で引用)
- Google. (2024). Generative AI in Search: Let Google do the searching for you. 公式発表。

---

4. 書籍・単行本

- Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O’Reilly Media. ISBN 978-1449373320. (邦訳:『データ指向アプリケーションデザイン』オライリー・ジャパン)
  - 第 8 章 “The Trouble with Distributed Systems” は第 4 章 Stale Carry-Forward パターンの一般的参照枠である。
- Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (第 2 版). Pragmatic Bookshelf. ISBN 978-1680502398.
  - Circuit Breaker、Bulkhead、Timeout パターンは第 5 章 modelRouter の設計参照である。
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution. ISBN 978-1942788003.
  - 第 9 章のクローズドフィードバック設計は、第三の道の 「継続的学習」章に示唆を受けている。

---

5. 関連学術研究（抜粋）

本書は包括的な文献レビューを実施していない。以下は著者が読んだ代表的研究であり、読者は GitHub Issues を通じて追加を提案されたい。

LLM ハルシネーション検知

- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023.
- Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. EMNLP 2023.

ナレッジグラフとエンティティリンキング

- Pan, J. Z., Razniewski, S., Kalo, J.-C., et al. (2024). Large Language Models and Knowledge Graphs: Opportunities and Challenges. TGDK 2024.
- Hogan, A., Blomqvist, E., Cochez, M., et al. (2021). Knowledge Graphs. ACM Computing Surveys, 54(4).

多源 NLI

- Nie, Y., Chen, H., & Bansal, M. (2019). Combining Fact Extraction and Verification with Neural Semantic Matching Networks. AAAI 2019.
- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: A Large-scale Dataset for Fact Extraction and VERification. NAACL 2018.

構造化データと LLM

- Yang, J., Chen, H., Yan, Y., et al. (2024). Exploring the Role of Structured Data in Large Language Model Fact Verification. Preprint.

本書の引用方法

正式な引用形式は `CITATION.cff` および README の引用セクション に記載されている。

```
@techreport{lin2026baiyuangeo,
  author      = {Lin, Vincent},
  title       = {Baiyuan GEO Platform: A Whitepaper on Building a SaaS for Generative Engine Optimizatio
  institution = {Baiyuan Technology},
  year        = {2026},
  url         = {https://github.com/baiyuan-tech/geo-whitepaper},
  note        = {v1.0-draft}
}
```

付録 D —図目録と制作ガイドライン

日本語版における全図をここにインデックス化する。`.md` ファイルに直接埋込まれた Mermaid ダイアグラムは GitHub 上でネイティブ描画され、その他の静的画像（ある場合）は `assets/figures/` に配置される。

D.1 完全目録

ID	種類	主題	章
図 1-1	xychart-beta	2023〜2025 年の情報チャネル移行	第 1 章
図 1-2	表	SEO vs GEO 中核的差異	第 1 章 §1.3
図 2-1	flowchart	6 段階クローズドループシステム	第 2 章 §2.1
図 2-2	flowchart	ブランドガバナンス全体サイクル	第 2 章 §2.3
図 2-3	flowchart	マルチテナント RLS + アプリ層二重保険	第 2 章 §2.5
図 3-1	graph	7 次元レーダー隣接関係	第 3 章 §3.2
図 4-1	xychart-beta	プラットフォーム障害時の 3 戦略	第 4 章 §4.3
図 4-2	stateDiagram	プラットフォームデータ状態機械（Fresh/Stale/Expired/Reset）	第 4 章 §4.3
図 4-3	ASCII	フロントエンド isStale バッジ	第 4 章 §4.6
図 5-1	flowchart	modelRouter 2 経路アーキテクチャ	第 5 章 §5.2
図 5-2	flowchart	切替判断ツリー	第 5 章 §5.5
図 6-1	flowchart	同一ブランドの 2 つのビュー	第 6 章 §6.1
図 6-2	flowchart	AXP 3 層構造	第 6 章 §6.2
図 6-3	flowchart	Worker ルーティング判断フロー	第 6 章 §6.3
図 6-4	flowchart	25 種 AI ボット UA の 4 グループ分類	第 6 章 §6.4
図 6-5	flowchart	SaaS 自社ブランドのパスカテゴリ判断ツリー	第 6 章 §6.5
図 6-6	コードブロック	JSON-LD フラット vs ネスト配列	第 6 章 §6.7
図 7-1	表	25 業種分類表	第 7 章 §7.2
図 7-2	flowchart	3 層エンティティ・ナレッジグラフ	第 7 章 §7.3
図 7-3	flowchart	実店舗 vs オンラインの重み差異	第 7 章 §7.4
図 7-4	flowchart	ウィザード + 編集の 2 エントリ	第 7 章 §7.6
図 7-5	flowchart	GBP URL パーサ 4 分岐判断ツリー	第 7 章 §7.7
図 8-1	flowchart	一方向データフロー：GBP → Schema → AXP → AI	第 8 章 §8.1
図 8-2	flowchart	2 つのホスティングモデル (Manager / OAuth)	第 8 章 §8.3
図 8-3	表	同期頻度マトリクス	第 8 章 §8.5

ID	種類	主題	章
図 8-4	flowchart	GBP 統合 4 フェーズ・ロードマップ	第 8 章 §8.7
図 9-1	flowchart	6 段階クローズドループ	第 9 章 §9.1
図 9-2	flowchart	ハルシネーション 5 分類体系	第 9 章 §9.2
図 9-3	flowchart	3 階層知識源の NLI 統合	第 9 章 §9.3
図 9-4	flowchart	中央共有 RAG アーキテクチャ	第 9 章 §9.4
図 9-5	flowchart	LLM Wiki 文書ライフサイクル	第 9 章 §9.5
図 9-6	flowchart	ClaimReview 3 経路注入	第 9 章 §9.6
図 9-7	flowchart	2 層スキャンの役割分担	第 9 章 §9.7
図 9-8	xychart-beta	ハルシネーション残存率収束曲線	第 9 章 §9.8
図 10-1	flowchart	通常スキャン vs ベースライン	第 10 章 §10.1
図 10-2	flowchart	Phase 1/2/3 データ構造	第 10 章 §10.2
図 10-3	flowchart	4 軸変化観察マトリクス	第 10 章 §10.4
図 11-1	graph	5 ブランド 7 次元レーダー (Wk 1 vs Wk 6)	第 11 章 §11.2
図 11-2	flowchart	プラットフォームカバレッジの非対称性（英語 vs ローカル）	第 11 章 §11.3
図 11-3	xychart-beta	完成度 × シテーション率差分	第 11 章 §11.4
図 11-4	xychart-beta	AXP 前後の AI ボットトラフィック	第 11 章 §11.5
図 11-5	pie	顧客側落とし穴分布	第 11 章 §11.6
図 12-1	表	現行カバレッジマトリクス	第 12 章 §12.1
図 12-2	flowchart	今後の課題依存（短/中/長期）	第 12 章 §12.4

合計：13 章、44 図（Mermaid 優位、ASCII アート・表・コードブロックを少量含む）。

D.2 制作スタイルガイド

1. 統一スタイル

以下のパレットを使用する（`%%{init: {'theme':'base'}}%%` + 個別 `style` オーバーライドで適用）：

用途	Hex
プライマリ（ノード境界、強調）	`#1e40af`（深青）
アクセント（強調、矢印）	`#ea580c`（明橙）
警告（エラー、失敗経路）	`#dc2626`（赤）
成功（修復、収束）	`#16a34a`（緑）
ニュートラル（背景、補助）	`#64748b`（スレート）

Mermaid `style` 構文で特定ノードに適用できる：

```
flowchart LR
    A[node]
    style A fill:#1e40af,color:#fff
```

2. Mermaid を静的画像より優先

- **技術フロー**（flowchart / stateDiagram / sequenceDiagram）—全て Mermaid

- **データ関係**（graph、ER）—Mermaid
- **トレンドチャート**（xychart-beta）—Mermaid
- **円グラフ**（pie）—Mermaid

理由：

- バージョン管理下で差分可能、読者がコピー・改変可能
- 外部画像ホスト依存無し（ホストは失敗しうる）
- AI クローラが Mermaid ソースコードを読取可能、図自体が構造化データ

3. 静的画像（SVG/PNG）を使うべき場合

以下の場合のみ `assets/figures/` 配下に配置：

- Mermaid で表現できない複雑な視覚（深くネストされたレイアウト、イラスト）
- プロダクトスクリーンショット（UI フレーム、顧客パネル例）
- Recharts 対話性が必要なチャート（PDF では静的 PNG になる）

全静的画像は：

- ソースファイルを保持（`.fig`、`.drawio`、Figma リンク）
- デフォルトは SVG（スケーラブル、軽量）
- 命名：`fig-< 章 >-< 番号 >-<slug>.svg`、例：`fig-11-03-completion-citation.svg`

4. alt テキスト

全静的画像に記述的 alt テキストを付与：

`![図 X-Y 主題：具体的内容 <br>* 図 X-Y：詳細キャプション *](assets/figures/fig-XX-YY-slug.svg)`

alt テキストは画像を見られない読者に図の要点を伝達すべきである（視覚障害ユーザと AI クローラの両方に対応）。

5. 多言語版は独立制作

- 中国語版の図は英語版へ**翻訳しない**、英語慣習で再描画する
- 日本語版は日本語で独立描画、日本語フォント・慣習・フレーズを使用
- データ単位・日付形式・文字方向はロケール毎に調整
- 全言語版は図番号を共有（図 1-1 中国語 / 図 1-1 英語 / 図 1-1 日本語の並列）

6. プロダクトスクリーンショットの匿名化

UI スクリーンショットは：

- 顧客ブランド名を伏字化（「Brand A」、「Example Brand」を使用）
- 実数値をぼかし or 置換（例：`$12,345` → `$XX,XXX`）
- 内部 URL パスを伏字化（構造のみ示し、具体的ドメインは示さない）
- メールアドレスをマスク（`user@***.com`）

7. Pandoc PDF ビルド時の Mermaid

Mermaid は GitHub Markdown、GitLab、Obsidian、VS Code Preview 上でネイティブ描画される。**Pandoc PDF 生成**には前処理ステップが必要：

- CI が `mermaid-cli` でコードブロックを SVG に事前描画
- または Pandoc フィルタ `pandoc-mermaid` で自動処理

本リポジトリの `build-pdf.yml` は v1.0-draft で用いられたプレースホルダ置換手法を含む（`.github/workflows/build-pdf.yml`参照）。

D.3 制作ステータス

ステータス	図数	比率
□ `.md` 内完成	44	100%
□ プロダクトスクショ要取得	0	0%
□ 日本語版再描画	0（日本語版全編公開済）	—